الفصل الرابع

المصفوفات وأنواعها

٤.١- المصفوفات الأحادية الأبعاد :

٤.٢- ترتيب عناصر المصفوفات:

٤.٣- المصفوفات ثنائية الأبعاد:

٤.٤- ترتيب المصفوفات ثنائية الأبعاد:

٤.٥- النصوص (string):

١.٤- المصفوفات الأحادية الأبعاد :

قبل أن نتحدث عن المصفوفات علمنا أن المتغيرات تحجز مكان في الذاكرة تخزن فيه قيمة المتغير .
لو أخذنا هذا الجزء من الذاكرة وافترضنا ان المتغير (x=15) مخزن في الموقع (18126).

مواقع خلايا الذاكرة	
الموقع	محتواه
18125	***data***
18126	*15*
18127	*data*

هذا هو موقع المتغير x

مهم

كما تلاحظ من الرسم أن المتغير (x) حاجز مكان في الذاكرة وخازن قيمته بداخلة.
فما هي المصفوفات. هي مجموعة خلايا متتالية في الذاكرة تحجز لغرض خزن معلومات معينة في داخلها كأن نخزن في داخلها أرقام أو أحرف وتبقى هذه القيم المخزنة داخل المصفوفة حتى نغلق البرنامج إذا لم نغيرها داخل البرنامج .
في مصفوفات يجب الإعلان عن عدد المواقع التي نحتاجها في العمل في بداية البرنامج حتى يحجزها المترجم للمصفوفة ولا يخزن قيم أخرى في داخلها و تبقى محجوزة فقط لعناصر المصفوفة. ويكون الإعلان عليها هكذا

هيكلية
```
Type  arrrayname[size of array]
```

- ***(arrrayname)*** هو اسم المصفوفة الذي سنتعامل معه في البرنامج أي اسم ممكن أن نسمي المصفوفة
- **(size of array)** هو حجم الذي ستشغله المصفوفة في الذاكرة وقد يكون أي رقم حسب احتياجك
- **(Type)** هو نوع المصفوفة التي سوف نعرفها قد تكون حرفية أو رقمية. **لو عرفنا مصفوفة من نوع (integer) جميع عناصرها تكون (integer) ولا يجوز تخزين أحرف في داخلها**

مثال: تعريف مصفوفة من نوع (integer) حجمها خمسة واسمها (first_array).

هيكلية
```
int   first_array [5];
```

لنفرض أن حجز أول عنصر في المصفوفة بدء بموقع (18126) ويكون شكل مواقع الذاكرة هكذا

مواقع خلايا الذاكرة		
مواقع عناصر المصفوفة	الموقع	محتواه
	18125	***data***
first_array [0]	***18126***	
first_array [1]	***18127***	
first_array [2]	***18128***	
first_array [3]	***18129***	
first_array [4]	***18130***	
	18131	*data*

هذه المواقع محجوزة للمصفوفة
first_array
وبذالك لا تستطيع اي متغيرات اخرى استخدامها

مهم

*لو تلاحظ أن أول عنصر للمصفوفة وهو **(first_array [0])** يحجز الموقع (18126) وتخزن فيه قيمة أول عنصر ويليه بقية العناصر بالتتالي أي **(first_array [1])** يحجز الموقع (18127) وبالتتالي....؟*

- عنونة عناصر المصفوفة تبدأ من الصفر كما تلاحظ أول موقع بالمصفوفة هو **(first_array [0])**

هيكلية
```
first_array [index]
```

(index) هو عنوان الموقع الذي نريد أن نصل إلى محتوياته في داخل المصفوفة لنعدل عليها أو نطبعها.

- مثلا أردنا وضع (79) بثالث موقع في المصفوفة نكتب هكذا

كود
```
first_array [2]=79;
```

لو تلاحظ أن **(index=2)** وليس (٣) لأنه كما قلنا **تسلسل عناصر المصفوفة يبدأ من الصفر وليس من الواحد** أي لو كان لدينا مصفوفة حجمها خمسة عناصر فتسلسل (index) يكون من الصفر إلى الأربعة.
لذالك بعد أن وضعنا رقم (79) بالموقع الثالث يصبح شكل المصفوفة في مواقع الذاكرة هكذا

مواقع خلايا الذاكرة

مواقع عناصر المصفوفة	الموقع	محتواه
	18125	***data***
first_array [0]	***18126***	
first_array [1]	***18127***	
first_array [2]	***18128***	*79*
first_array [3]	***18129***	
first_array [4]	***18130***	
	18131	*data*

محتويات الموقع الثالث

هذه المواقع محجوزة للمصفوفة **first_array**

- لو أردنا إدخال قيمة (٩٠) في الموقع الرابع في المصفوفة فيكون الكود

كود
```
first_array [3]=90;
```

- لو أردنا طباعة محتويات الموقع الثالث

الكود بلغة C	الكود بلغة C++
Printf("%d", first_array [2]);	cout<< first_array [2];

لو أردنا تعريف مصفوفة من نوع float نفس طريقة مصفوفة السابقة وكذالك بقية الأنواع تعرف بنفس الطريقة

هيكلية
```
float   first_array [5];
```

نستطيع أعطاء عناصر المصفوفة قيم معينة كقيم أولية وقت التعريف عن المصفوفة هكذا

كود
```
int  first_array [5]={34,26,43,23,54};
```

فيكون شكل المصفوفة في مقطع الذاكرة وهي مخزن فيها العناصر هكذا

مواقع خلايا الذاكرة		
مواقع عناصر المصفوفة	الموقع	محتواه
	18125	***data***
first_array [0]	***18126***	*34*
first_array [1]	***18127***	*26*
first_array [2]	***18128***	*43*
first_array [3]	***18129***	*23*
first_array [4]	***18130***	*54*
	18131	*data*

هذه المواقع محجوزة للمصفوفة **first_array** ومحتوياتها

إي أن العنصر الأول في المصفوفة تكون قيمته (first_array [0]=34) وبقية المواقع بالتسلسل كما في الشكل .

مثال: برنامج ندخل مصفوفة أحادية الأبعاد مكونة من ستة عناصر من شاشة التنفيذ ونطبعها.؟

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
{1. int i,first_arry[6];
2.for (i=0;i<6;i++)
3.scanf("%d",& first_arry[i]) ;
4.prin ("the Content of array is .\n");
5.for (i=0;i<6;i++)
6.prin ("%d\t", first_arry[i]);}
	#include<iostream.h>
	main()
	{ 1.int i,first_arry[6];
	2.for (i=0;i<6;i++)
	3.cin>> first_arry[i] ;
	4.cout<<"the Content of array is .\n";
	5.for (i=0;i<6;i++)
	6.cout<< first_arry[i]<<"\t";}

توضيح الخطوات:

١.خطوة رقم (١) تم تعريف متغير للعداد وتم تعريف مصفوفة أرقام حجمها ستة

٢.خطوة رقم (٢) هوا عداد يعد من صفر إلى خمسة وكل عدة يتم طلب في خطوة (٣) إدخال عنصر جديد من المصفوفة كما هوا مبين في الشكل ، عندما يكون (i=0) فان (first_arry[i]) معناه (first_arry[0]) أي يطلب من المستخدم إدخال العنصر الأول ويستمر بطلب الإدخال إلى أخر عنصر وهو (i=5) (كما هو بالشكل بالأسفل)

```
(Inactive C:\TCWIN45\BIN\NONAME01.EXE)
first_arry[0]   first_arry[1]   first_arry[2]   first_arry[3]   first_arry[4]
first_arry[5]
```

٣.خطوة رقم (٣) هناك طريقتين للقراءة **أما ضغط مفتاح (enter) وإدخال عنصر جديد من عناصر المصفوفة أو ضغط المسطرة وإدخال عنصر جديد** . خطوة رقم (٤) هي رسالة تقول انه سوف يتم طباعة محتويات المصفوفة

٥.خطوة رقم (٦) عداد يعد من صفر إلى الخمسة ليطبع جميع عناصر المصفوفة في الخطوة رقم (٦). أي عندما يعد العداد رقم صفر ستنفذ خطوة رقم (٦) طابعة (first_arry[0]) تم عندما يعد العداد رقم واحد ستنفذ خطوة رقم (٦) طابعة (first_arry[1]) تم عندما يعد العداد رقم اثنان ستنفذ خطوة رقم (٦) طابعة (first_arry[2]) ويستمر بطباعة جميع عناصر المصفوفة من محتويات أول موقع إلى محتويات أخر موقع .

وتكون شاشة التنفيذ هكذا

```
(Inactive C:\TCWIN45\BIN\NONAME01.EXE)
8 45 34 2 9 5 3
the Content of array is .
8        45       34       2        9        5
```

لو غيرنا عداد خطوة رقم(٥) وجعلناه يتناقص من خمسة إلى الصفر سوف يطبع أولا (first_arry[5]) ثم (first_arry[4]) ويستمر إلى الصفر *طابعا المصفوفة بالمقلوب* (من أخر عنصر إلى أول عنصر)

أي يصبح عداد خطوة رقم (٥) هكذا

كود

```
5.for (i=5;i>=0;i--)
```

ستكون شاشة التنفيذ بعد الإدخال هكذا

```
(Inactive C:\TCWIN45\BIN\NONAME01.EXE)
43 54 23 7 4 34
the Content of array is .
34       4        7        23       54       43
```

أبدال بين قيم موقعين: لإبدال بين قيمة موقعين في المصفوفة نحتاج إلى متغير ثالث من نفس نوع المصفوفة حتى نخزن بهِ نتيجة احد الموقعين ثم نبدل لان في حال عدم وجود متغير ثالث لا نستطيع الإبدال وستضيع احد القيمتين

كود الإبدال بين قيمة المتغير (first_arry[5]) والمتغير (first_arry[2])

```
int item;   //هوا المتغير المؤقت الذي سنخزن به احد قيم الموقعين
Item= first_arry[5];
first_arry[5]= first_arry[2];
first_arry[2]=item;
```

مثال: برنامج ندخل مصفوفة أحادية الأبعاد مكونة من خمسة عناصر ونجمعها .؟

تحليل :خطوات الإدخال هي اعتيادية وتقريبا ثابتة في المصفوفات فمصفوفة حجمها خمسة نحتاج عداد يعد من الصفر إلى الأربعة ثم بعد الإدخال نجمع جميع عناصر المصفوفة مع بعضها

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
{ 1.int i,array1[5],sum=0;
2.for (i=0;i<5;i++)
3.scanf("%d",& array1[i]);
4.for (i=0;i<5;i++)
5.sum=sum+array1[i];
6.prin ("sum of array item=%d",sum);}
	#include<iostream.h>
	main()
	{ 1.int i,array1[5],sum=0;
	2.for (i=0;i<5;i++)
	3.cin>> array1[i] ;
	4.for (i=0;i<5;i++)
	5.sum=sum+array1[i];
	6.cout<< "sum of array item="<<sum;}

توضيح الخطوات:

١.خطوة رقم (١) تم تعريف متغير للعداد وتم تعريف مصفوفة أرقام حجمها خمسة ومتغير للجمع وأعطيناه صفر كقيمة ابتدائية كما قلنا سابقا يجب تصفير المتغير الجمع قبل الجمع

٢.خطوة رقم(٢ و٣) إدخال المصفوفة كما تلاحظ العداد يعد من الصفر إلى الأربعة لان حجم المصفوفة خمسة

٣.خطوة رقم (٤) عداد يمر على عناصر المصفوفة عنصر عنصر لكي يجمعها مع بعضها أي عندما يكون قيمة العداد هي (i=0) سيجمع (sum=sum+array1[0]) ويستمر بالجمع مع القيم السابقة كما هي هذا الشكل

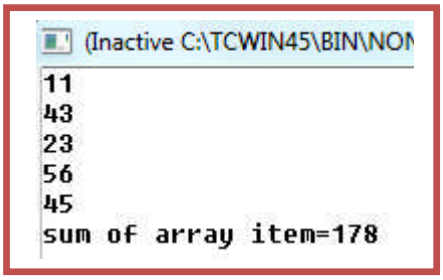
هذا الشكل هو تتبع للعمليات التي تحدث في خطوة رقم (٤) وخطوة رقم (٥)

توضيح عملية جمع في المصفوفة إذا فرضنا عناصر المدخلة هي (11,43,23,56,45) كما في شاشة التنفيذ

```
When i=0
sum=sum+array1[0];  → sum=0+11 →sum=11
When i=1
sum=sum+array1[1];  → sum=11+43 →sum=54
When i=2
sum=sum+array1[2];  → sum=54+23 →sum=77
When i=3
sum=sum+array1[3];  → sum=77+56 →sum=133
When i=4
sum=sum+array1[4];  → sum=133+45 →sum= 178
```

لو تلاحظ كيف يجمع نتائج الجمع السابقة مع العناصر الجديدة.....!

٤.خطوة رقم (٦) يطبع نتيجة الجمع



مثال: برنامج ندخل مصفوفة أحادية الأبعاد مكونة من سبعة عناصر جد اكبر رقم.؟

تحليل :خطوات الإدخال هي اعتيادية وتقريبا ثابتة في المصفوفات فمصفوفة حجمها سبعة نحتاج عداد يعد من الصفر إلى ستة. ثم بعد الإدخال نجد اكبر رقم بين عناصر المصفوفة كيف.؟ نسند قيمة أول عنصر في المصفوفة لمتغير وثم نقارن قيمة هذا المتغير مع بقية العناصر أي عنصر في المصفوفة نجد أن قيمته اكبر من قيمة متغيرنا نبدل قيمة متغيرنا بقيمة عنصر المصفوفة ونستمر حتى النهاية وكل ما وجد متغيرنا قيمة أعلى منه أخذها وإذا اقل منه لاتهمه ويستمر إلى النهاية ويكون في التالي حاملا أعلى قيمة.

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
{ 1.int i,array1[7],max;
2.for (i=0;i<7;i++)
3.scanf("%d",& array1[i]) ;
4.max=array1[0];
5.for (i=0;i<7;i++)
6.if (array1[i] > max)
7.max=array1[i];
8.prin ("max number in array1 is=%d",max);}
	#include<iostream.h>
	main()
	{ 1.int i,array1[7],max;
	2.for (i=0;i<7;i++)
	3.cin>> array1[i] ;
	4.max=array1[0];
	5.for (i=0;i<7;i++)
	6.if (array1[i] > max)
	7.max=array1[i];
	8.cout<< "max number in array1 is="<<max;}

وهذا توضيح لما سيحصل في خطوات رقم (٥ و٦و٧)

جعلت الخطوات التي تنفذ ذو لون عميق والتي لا تنفذ ذات لون رفيع

توضيح (٥ و٦و٧) إذا فرضنا العناصر المدخلة هي (11,32,22,32,43,31,23) كما في شاشة التنفيذ

4. max=array1[0]; → max=11

5.When i=0
6. (array1[0]=11) ليست اصغر من (max=11) هذه الخطوة لا تحقق الشرط لان قيمة المتغير
7. (٦) لا تنفذ هذه الخطوة إذا لم يتحقق الشرط في خطوة

5.When i=1
6. (array1[1]=32) اصغر من (max=11) هذه الخطوة تحقق الشرط لان قيمة المتغير
7.max =array1[1] → max=32 //٣٢ اخذ المتغير قيمة جديدة اكبر منه لتصبح قيمته

5.When i=2
6. (array1[2]=22) ليست اصغر من (max=32) هذه الخطوة لا تحقق الشرط لان قيمة المتغير
7. (٦) لا تنفذ هذه الخطوة إذا لم يتحقق الشرط في خطوة

5.When i=3
6. (array1[3]=32) ليست اصغر من (max=32) هذه الخطوة لا تحقق الشرط لان قيمة المتغير
7. (٦) لا تنفذ هذه الخطوة إذا لم يتحقق الشرط في خطوة

5.When i=4
6. (array1[4]=43) اصغر من (max=32) هذه الخطوة تحقق الشرط لان قيمة المتغير
7.max =array1[4] → max=43 //٤٣ اخذ المتغير قيمة جديدة اكبر منه لتصبح قيمته

5.When i=5
6. (array1[5]=31) ليست اصغر من (max=43) هذه الخطوة لا تحقق الشرط لان قيمة المتغير
7. (٦) لا تنفذ هذه الخطوة إذا لم يتحقق الشرط في خطوة

5.When i=6
6. (array1[6]=23) ليست اصغر من (max=43) هذه الخطوة لا تحقق الشرط لان قيمة المتغير
7. (٦) لا تنفذ هذه الخطوة إذا لم يتحقق الشرط في خطوة

توضيح الخطوات:

١.خطوة رقم (١) تم تعريف متغير للعداد وتم تعريف مصفوفة أرقام حجمها سبعة ومتغير لخزن اكبر رقم وخطوة رقم (٢ و٣) هي إدخال للمصفوفة

٢.خطوة رقم (٤) سندنا قيمة أول موقع إلى متغير (max) حتى نقارنه مع بقية المواقع حتى أي قيمة تكون اكبر منه في المقارنة نبدلها بقيمة (max) حتى نحصل على اكبر قيمة.

٣.خطوة رقم (٥)عداد يمر على جميع عناصر المصفوفة لكي نقارن في خطوة رقم (٦) عناصر المصفوفة بقيمة المتغير ومتى ما كان احد عناصر المصفوفة قيمته اكبر نبدل قيمة (max) في خطوة رقم (٧) بالقيمة الأكبر منه

لو تلاحظ أن قيمة الموقع الخامس هي اكبر قيمة من بين القيم (شاهد التوضيح وكيفية تغير قيمة (max)

Array1[0]	Array1[1]	Array1[2]	Array1[3]	Array1[4]	Array1[5]	Array1[6]	تغير قيمة المتغير (max) عند مقارنته
11	*32*	*22*	*32*	*43*	*31*	*23*	
Max=11	*Max=32*	*Max=32*	*Max=32*	*Max=43*	*Max=43*	*Max=43*	

*

٤.خطوة رقم (٨) يطبع اكبر رقم

```
(Inactive C:\TCWIN45\BIN\NONAME0
11
32
22
32
43
31
23
max number in array1 is=43
```

مهم

لو أردنا إيجاد اصغر رقم في المصفوفة فقط نبدل الشرط في الخطوة رقم (٦) إلى اصغر هكذا

كود

```
6.if (array1[i] < max )
```

وطريقة التتبع هنا كلما يجد رقم في المصفوفة اصغر من قيمة المتغير يسند المتغير للقيمة الأصغر

مهم

لو أردنا ضرب كل عنصر من عناصر المصفوفة في (٢)

كود

```
array1[i]=2*  array1[i];
```

كأنما نقول قيمة العنصر الجديد تساوي قيمته القديمة مضروبة في (٢) ونمرر العداد على جميع العناصر

وكذالك بقية العمليات الرياضية تعامل نفس الطريقة كالقسمة. مثلا نقسم كل عنصر على ٤

كود
```
array1[i]=  array1[i] /4;
```

ولا توضع فقط هذه الجملة وحدها يجب وضع قبلها عداد يمر على جميع عناصر المصفوفة لكي يقسم جميع العناصر عنصر عنصر لو تلاحظ خطوة رقم (٢) وخطوة رقم (٣) في المثال التالي.

مثال: مصفوفة من خمسة عناصر إضافة قيمة خمسة لكل عنصر في المصفوفة

البرمجة بلغة c
```
#include<stdio.h>
main()
{ 1.int i,array1[5]={10,15,30,32,21};
2.for (i=0;i<5;i++)
{3. array1[i]= array1[i]+5;
4.prin   ("%d\t", array1[i] );}
}
```

البرمجة بلغة c++
```
#include<iostream.h>
main()
{ 1.int i,array1[5]={10,15,30,32,21};
2.for (i=0;i<5;i++)
{3. array1[i]= array1[i]+5;
4.cout<< array1[i]<<"\t";}
}
```

توضيح الخطوات:

١.خطوة رقم (٣) نضيف فيها قيمة خمسة لكل عنصر من عناصر المصفوفة

لو كان قيمة العداد هي (i=3) فيكون تنفيذ الخطوة رقم (٣) هكذا
```
array1[3]= array1[3]+5; → array1[3]=32+5 →  array1[3]=37
```

٢.وضعنا خطوة رقم (٤) داخل قوس العبارة التكرارية حتى يطبع مباشرة قيمة كل موقع بعد الإضافة .

٢.٤- ترتيب عناصر المصفوفات :

ترتيب الأرقام أو الأحرف تصاعديا أو تنازليا يكون بمقارنة كل عنصر من عناصر المصفوفة مع العناصر الذي تليه في المصفوفة فعلى سبيل المثال إذا أردنا ترتيب العناصر تصاعديا فنقارن كل عنصر في المصفوفة بالعناصر الذي تليه فإذا وجدنا رقم اقل من قيمة العنصر نبدل بين العنصريين (مثلا إذا كان array[0]=50,array[3]=2) فعند البدء المقارنة نجد القيمة في الموقع الرابع من المصفوفة هو (٢) وفي الموقع الأول هو (٥٠) وقيمة الموقع الرابع اقل لذالك سنبدل قيمة الموقع الرابع بقيمة الموقع الأول ونقارن بقية المواقع بقيمة الموقع الأول الجديدة وهو أصبح (٥٠) ونستمر بالإبدال إلى أن نصل في المقارنة إلى أخر رقم بالمصفوفة ثم نأخذ ثاني عنصر بالمصفوفة ونقارنه ببقية العناصر ثم الثالث إلى إن نصل إلى أخر عنصر نجد المصفوفة مرتبة وشاهد هذه خطوات تفصيلية للترتيب

لنفرض أن لدينا مصفوفة من خمسة عناصر ونريد ترتيبها تصاعديا وكانت القيم المخزنة بالمصفوفة كما في الشكل

مهم

array[0]	array[1]	array[2]	array[3]	array[4]
45	42	85	6	64

عند الترتيب التصاعدي نضع الرقم الاقل الى الامام والاعلى الى الخلف في المقارنة والتنازلي بالعكس

للترتيب كما قلنا نقارن كل موقع بجميع المواقع التي تليه هكذا..؟

١.مقارنة الموقع الأول ببقية المواقع التي تليه:

١.مقارنة من الموقع الأول مع الموقع الثاني

array[0]	array[1]	array[2]	array[3]	array[4]
45	42	85	6	64

وجدنا إن قيمة الموقع الثاني اقل من قيمة الموقع الأول لذالك سنبدل بينهما

٢.مقارنة الموقع الأول بالموقع الثالث

array[0]	array[1]	array[2]	array[3]	array[4]
42	45	85	6	64

وجدنا إن قيمة الموقع الثالث أعلى من قيمة الموقع الأول لذالك لا نبدل بينها

٣.مقارنة الموقع الأول بالموقع الرابع

array[0]	array[1]	array[2]	array[3]	array[4]
42	45	85	6	64

وجدنا أن قيمة الموقع الرابع اقل من قيمة الموقع الأول لذالك سنبدل بينهما

٤.مقارنة الموقع الأول بالموقع الخامس

array[0]	array[1]	array[2]	array[3]	array[4]
6	45	85	42	64

وجدنا أن قيمة الموقع الخامس أعلى من قيمة الموقع الأول لذالك لا نبدل بينهما

٢.مقارنة الموقع الثاني ببقية المواقع التي تليه:

١.مقارنة بين الموقع الثاني مع الموقع الثالث

array[0]	array[1]	array[2]	array[3]	array[4]
6	45	85	42	64

وجدنا أن قيمة الموقع الثالث أعلى من الثاني لذالك سوف لا يبدل بينهما.

٢.مقارنة الموقع الثاني مع الموقع الرابع

array[0]	array[1]	array[2]	array[3]	array[4]
6	45	85	42	64

وجدنا أن قيمة الموقع الرابع اقل من الموقع الثاني لذالك سنبدل بينها

٣.مقارنة الموقع الثاني بالموقع الخامس

array[0]	array[1]	array[2]	array[3]	array[4]
6	42	85	45	64

وجدنا أن قيمة الموقع الخامس أعلى من قيمة الموقع الثاني لذالك سوف لا يبدل بينهما

٣.مقارنة الموقع الثالث ببقية المواقع التي تليه:

١.مقارنة الموقع الثالث بالموقع الرابع

array[0]	array[1]	array[2]	array[3]	array[4]
6	42	85	45	64

وجدنا قيمة الموقع الرابع اقل من قيمة الموقع الثالث لذالك سنبدل بينهما

٢.مقارنة الموقع الثالث بالموقع الخامس

array[0]	array[1]	array[2]	array[3]	array[4]
6	42	45	85	64

وجنا قيمة الموقع الخامس اقل من قيمة الموقع الرابع لذالك سنبدل بينهما

٣.مقارنة الموقع الرابع ببقية المواقع التي تليه: يقارن فقط بالموقع الخامس لأنه أخر موقع وسنجد أن قيمة الموقع الخامس اقل من قيمة الموقع الرابع لذالك سنبدل بينهما

array[0]	array[1]	array[2]	array[3]	array[4]
6	42	45	85	64

قيمة الموقع الخامس اقل من قيمة الموقع الرابع لذال سنبدل بينهما وستصبح المصفوفة مرتبة بشكل التالي

array[0]	array[1]	array[2]	array[3]	array[4]
6	42	45	64	85

ألان لنحول الكلام الذي كتباه والمخططات إلى مثال

مثال: برنامج لترتيب عناصر المصفوفة يدخلها المستخدم تصاعديا..؟

تحليل:كيف نرتب برمجيا؟علمنا انه بالترتيب نقارن كل موقع بجميع المواقع التي تليه لذالك سنحتاج إلى عدادان الأول خاص بالمرور على كل موقع مرة واحدة وعداد داخلي يقارن كل عنصر مرة عليه العداد الخارجي بجميع العناصر التي تليه في المصفوفة وأينما وجد رقم اقل منه يبدل بينهما.

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h> int main() {1.int array[5],i,j; 2. int item=0; 3.prin ("Here is the Array befor sorted enter it\n") ; 4. for (i=0;i<5;i++) 5.scanf("%d",&array[i]) ; 6. for (i=0;i<5-1;i++) 7.for (j=i; j<5;j++) 8.if (array[j] <array[i]){ 9.item =array[j]; 10.array[j]=array[i]; 11. array[i]= item ;} 12. prin ("Here is the Array a er sorted\n"); 13.for (i=0;i<5;i++) 14.prin ("%d\n",array[i]);}	#include<iostream.h> int main() {1.int array[5],i,j; 2. int item=0; 3.cout<<"Here is the Array befor sorted enter it\n"; 4. for (i=0;i<5;i++) 5.cin>>array[i] ; 6. for (i=0;i<5-1;i++) 7.for (j=i; j<5;j++) 8.if (array[j] <array[i]){ 9.item =array[j]; 10.array[j]=array[i]; 11. array[i]= item ;} 12.cout<<"Here is the Array a er sorted\n"; 13.for (i=0;i<5;i++) 14.cout<<array[i]<<"\n";}

توضيح الخطوات:

١.خطوة رقم (١) عرفنا مصفوفة حجمها خمسة ومتغير للعداد الخارجي (i) ومتغير أخر للعداد الداخلي (j)

٢.خطوة رقم (٢) عرفنا متغير لتخزين قيمة احد المتغيرين عند الإبدال لكي لا تضيع قيمة احدهما

٣.خطوة رقم (٣) هي رسالة طباعيه تطلب من المستخدم إدخال المصفوفة وخطوة رقم (٤ و ٥)قراءة المصفوفة

٤.خطوة رقم (٦) هوا عداد خارجي يبدأ بالعد من الصفر إلى الموقع القبل الأخير لكي يرتبهم ولا يعد الموقع الأخير لان الموقع الأخير مرتب أصلا من قبل الخطوة التي تسبقه لذالك نرى شرط التوقف في العداد هو(i<5-1).
أي في كل عدة لهذا العداد تتكرر الخطوات رقم (٧ و٨و٩و١٠و١١)

٥.خطوة رقم (٧) هوا عداد يبدأ بالعد من قيمة العداد الخارجي إلى أربعة أي إذا عد العداد الخارجي صفر سيعد العداد الداخلي من صفر إلى أربعة لكي يقارن الموقع الأول بجميع المواقع التي تليه. وإذا عد العداد الخارجي واحد سيعد العداد الداخلي من واحد إلى أربعة لكي يقارن المواقع الثاني بجميع المواقع التي تليه ويستمر إلى موقع الرابع

٦.خطوة رقم (٨) هوا شرط أذا كان قيمة العنصر اقل من قيمة احد المواقع التي تليه يبدل بينهم في خطوة رقم (٩ و١٠و١١)

٧.خطوات رقم (١٢و١٣و١٤) هوا عملية طباعة للمصفوفة بعد الترتيب هكذا

```
(Inactive C:\TCWIN45\BIN\NONAME01.EXE)
Here is the Array befor sorted enter it
45
42
85
6
64
Here is the Array after sorted
6
42
45
64
85
```

القيمة الوسطية بعد الترتيب تقع منتصف عناصر المصفوفة وبما إن حجم مصفوفتنا خمسة فالقيمة الوسطية هي الموقع الثالث أي (array[2])

مهم

لو أردنا ترتيب المصفوفة تنازليا نفس الطريقة فقط نبدل الشرط في خطوة رقم (٨) إلى

كود

```
8.if (array[j] >array[i])
```

مهم

لو أردنا ترتيب فقط المواقع الثلاث الأولى ولاتهمنا البقية فقط نغير شرط التوقف في خطوة رقم (٦ و٧) نبدل رقم خمسة ب ثلاثة هكذا (وضعنا ثلاثة عند شرط التوقف لكي يرتب فقط الموقع الثلاثة الأولى)

كود

```
6. for (  i=0;i<3-1;i++)
 7.for ( j=i; j<3;j++)
```

مهم

لو أردنا ترتيب المواقع الزوجية في المصفوفة ترتيبا تصاعديا فقط نغير هاتان الخطوتان

كود

```
6. for (  i=0;i<5-1;i=i+2)
 7.for ( j=i; j<5;j=j+2)
```

مثال: برنامج لمعرفة هل الرقم متناظر (أي يقرأ من الجهتين) **ويكون عدد الأرقام خمسة مثلا ١٢٣٢١**

تحليل:لكي نعرف هل الرقم متناظر هنا في مصفوفة خماسية يجب أن يساوي محتويات الموقع الأول محتويات الموقع الخامس والثاني يساوي الرابع **(المثال التالي ليس يتحقق فقط من تناظر الأرقام بل الحروف أيضا)**

البرمجة بلغة c	البرمجة بلغة c++

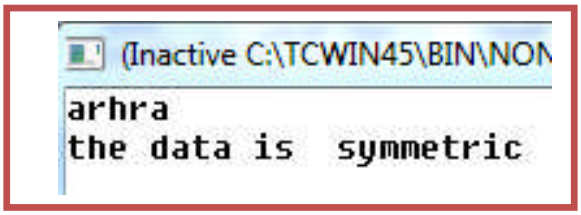
#include<stdio.h>
#include<conio.h>
main()
{1.int i, bol=0, a[5];
2.for(i=0;i<5;i++)
3.a[i]=getche();
4.for(i=0;i<5;i++)
5.if(a[i]!= a[5-i-1])
6.bol=1;
7. if(bol==0)
8. prin ("\nthe data is symmetric");
9.else
10.prin ("\nthe data is no symmetric");}
	#include<iostream.h>
	#include<conio.h>
	main()
	{1.int i, bol=0, a[5];
	2.for(i=0;i<5;i++)
	3.a[i]=getche();
	4.for(i=0;i<5;i++)
	5.if(a[i]!= a[5-i-1])
	6.bol=1;
	7. if(bol==0)
	8. cout<<"\nthe data is symmetric";
	9.else
	10.cout<<"\nthe data is no symmetric";}

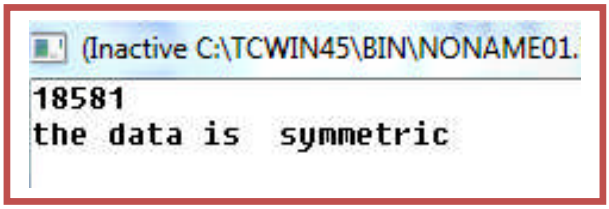
توضيح الخطوات:

١.خطوة رقم (٣) هي دالة لإدخال حرف وليس رقم ورغم أنهما ستدخل بشكل حرف ومداخلاتنا قد تكون أرقام فلا يهم ففي المقارنة فبدلا ما يقارن الرقم (١) يقارن الاسكي كود له أي (٥٠) فلا يضر لكن للجمالية استخدمنا هذه الدالة

٢.خطوة رقم (٥) يقارن العناصر الأول بالأخر والثاني بالقبل الأخير وهكذا ماداما متساويين يكون المتغير (bol=0) فإذا كانا غير متساويين ويتحقق شرط خطوة رقم (٥) وينفذ خطوة (٦) ستتحول قيمته إلى واحد

٣.خطوة رقم (٧) يتحقق من قيمة المتغير (bol) إذا بقت ثابتة حسب تعريفها طبع رسالة إن الرقم متناظر وإذا تغيرت إلى واحد يطبع رسالة أن الرقم غير متناظر





مثال: مصفوفة حجمها (٥) اسمها (a) ضع العناصر الزوجية لها في مصفوفة (b) والأعداد الفردية في (c)

تحليل: الموضوع بسيط جدا ليدك ثلاث مصفوفات نفس الحجم أو بالأحرى نفس حجم المصفوفة (a) لأنها قد تكون جميع الإعداد فردية أو زوجية.وبعد أن ندخل المصفوفة (a) نكون شرط إذا كان عددها فردي نضعه في (c) وإذا كلن زوجي نضعها في (b) ولكل مصفوفة عداد خاص بها.

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
{1.int count_a, count_b=0, count_c=0,a[5],b[5], c[5];
2.for(count_a=0; count_a<5; count_a++)
3.scanf("%d",&a[count_a]);
4.for(count_a=0; count_a<5; count_a++)
5.if (a[count_a] %2==0)
{6.b[count_b]=a[count_a];
7.count_b= count_b+1;}
8.else
{9.c[count_c]=a[count_a];
10.count_c= count_c+1;}
11.prin ("items in array b is\n");
12.for(count_a=0; count_a< count_b; count_a++)
13.prin ("%d\t", b[count_a]);
14. prin ("\nitems in array c is\n");
15.for(count_a=0; count_a< count_c; count_a++)
16. prin ("%d\t", c[count_a]);}
	#include<iostream.h>
	main()
	{1.int count_a, count_b=0, count_c=0,a[5],b[5], c[5];
	2.for(count_a=0; count_a<5; count_a++)
	3.cin>>a[count_a];
	4.for(count_a=0; count_a<5; count_a++)
	5.if (a[count_a] %2==0)
	{6.b[count_b]=a[count_a];
	7.count_b= count_b+1;}
	8.else
	{9.c[count_c]=a[count_a];
	10.count_c= count_c+1;}
	11.cout<<"items in array b is\n";
	12.for(count_a=0; count_a< count_b; count_a++)
	13.cout<< b[count_a]<<"\t";
	14.cout<<"\nitems in array c is\n";
	15.for(count_a=0; count_a< count_c; count_a++)
	16.cout<< c[count_a]<<"\t";}

توضيح الخطوات:

١.خطوة رقم (١) عرفنا عداد للمصفوفة (a) وهو (count_a) و عداد للمصفوفة (b) وهو (count_b) و عداد للمصفوفة (c) وهو (count_c) وعرفنا المصفوفات بنفس الأحجام

٢.خطوة رقم (٢و٣) هي قراءة للمصفوفة (a) من شاشة التنفيذ

٣.خطوة رقم (٤) هوا عداد لاختبار عناصر المصفوفة (a) تتبعه الخطوات رقم (5—11) كلها داخله التكرار

٤.خطوة رقم (٥) هوا شرط التحقق إذا كان رقم المصفوفة زوجية سيوضع في مصفوفة (b) في خطوة رقم (٦) ونزود عداد هذه المصفوفة في خطوة رقم (٧) (لكل مصفوفة عدادها لكي نميز بينهم بالإضافة حتى تكون الإضافة صحيحة)

٥. خطوة رقم (٨) هوا شرط التحقق إذا لم يكون رقم المصفوفة زوجي سيوضع في مصفوفة (c) في خطوة رقم (9) ونزيد عداد هذه المصفوفة في خطوة رقم (10) (لكل مصفوفة عدادها لكي نميز بينهم بالإضافة حتى تكون الإضافة صحيحة)

مهم

٦.خطوة رقم (١٢) هو عداد يبدأ بالصفر وينتهي بأقل من عداد المصفوفة (b) لكي نطبع فقط المواقع التي تحتوي على عناصر في هذه المصفوفة ولانطبع المواقع الفارغة اعتمادا على عدادها الذي يزداد بعد كل إضافة في خطوة رقم (7)

٧. خطوة رقم (١٥) هو عداد يبدأ بالصفر وينتهي بأقل من عداد المصفوفة (c) لكي نطبع فقط المواقع التي تحتوي على عناصر في هذه المصفوفة ولانطبع المواقع الفارغة اعتمادا على عدادها الذي يزداد بعد كل إضافة في خطوة رقم (١٠)

هذا ما سيظهر في شاشة التنفيذ

```
(Inactive C:\TCWIN45\BIN\NONAM
4 5 3 2 4 5
items in array b is
4        2        4
items in array c is
5        3
```

مهم

من ألان فصاعدا متى ما قال لك قسم عناصر مصفوفة إلى مصفوفتين استخدم نفس هذا الأسلوب فقط الشرط في خطوة رقم خمسة يتغير أو تغير أخر بسيط لكن تبقى الهيكلية تقريبا نفسها.....!

٣.٤- المصفوفات ثنائية الأبعاد:

لا تختلف المصفوفات الثنائية كثيرا عن المصفوفة أحادية الإبعاد .تختلف فقط في العنونة فبدلا من (index) واحد نستخدم اثنان واحد للصفوف وأخر للأعمدة (مثلا مصفوفة (2*3) أي مكونة من ثلاث صفوف وعموديين)

شكل مصفوفة (2*3)	عمود 0	عمود 1
صف 0	(0,0)	**(0,1)**
صف 1	(1,0)	**(1,1)**
صف 2	(2,0)	**(2,1)**

مهم

العنونة تكون (رقم العمود ،رقم الصف) وان العنونة الأعمدة والصفوف تبدأ من **الصفر** وليس من الواحد أي لو أردنا الوصول إلى عنصر في الصف الثاني العمود الأول تكون عنونه (1,0).

والمصفوفة الثنائية أيضا مجموعة خلايا متتالية في الذاكرة تحجز لغرض خزن معلومات معينة في داخلها كأن نخزن في داخلها أرقام أو أحرف وتبقى هذه القيم مخزنة داخل المصفوفة حتى نغلق البرنامج إذا لم نغيرها داخل البرنامج.

المصفوفات الثنائية: يجب الإعلان عن عدد المواقع التي نحتاجها في العمل في بداية البرنامج حتى يحجزها المترجم للمصفوفة ولا يخزن قيم أخرى داخل هذه المواقع تبقى محجوزة فقط لعناصر المصفوفة ويكون الإعلان عليها هكذا

هيكلية

```
Trype of array  arrrayname[row][columns]
```

- ***(arrrayname)*** *هوا اسم المصفوفة التي سنتعامل معه في البرنامج أي* اسم ممكن
- **(row)** هو عدد صفوف المصفوفة ، **(columns)** هو عدد الأعمدة لأننا نتعامل مع مصفوفة ثنائية الإبعاد
- **(array of Trype)** هو نوع المصفوفة التي سوف نعرفها قد تكون حرفية أو رقمية .**لو عرفنا مصفوفة من نوع (integer) جميع عناصرها تكون (integer) ولا يجوز تخزين أحرف في داخلها**

مثال:تعريف مصفوفة من نوع (integer) حجمها (2*3) واسمها (first_array).

هيكلية

```
int   first_array [3][2];
```

لنرسم المصفوفة هذه ذات حجم ثلاث صفوف وعمودان

شكل مصفوفة (2*3)	عمود 0	عمود 1
صف 0	first_array [0][0]	first_array [0][1]
صف 1	first_array [1][0]	first_array [1][1]
صف 2	first_array [2][0]	first_array [2][1]

مهم

كما تلاحظ في العنونة كيف تكون: عمود رقم (0) وهو العمود الأول وعمود رقم (1) وهو العمود الثاني وصف رقم (0) وهو الصف الأول و صف رقم (1) وهو الصف الثاني و صف رقم (2) وهو الصف الثالث.

وتخزين هذه العناصر في الذاكرة يكون نفس طريقة تخزين المصفوفة الأحادية لكن هنا يخزن صف وبعده صف أخر بالتسلسل إلى أن تنتهي الصفوف.

لنفرض أن تخزين أول عنصر في المصفوفة بدء بموقع (18126) ويكون شكل مواقع الذاكرة هكذا

مواقع خلايا الذاكرة		
مواقع عناصر المصفوفة	الموقع	محتواه
	18125	***data***
first_array [0][0]	***18126***	
first_array [0][1]	***18127***	
first_array [1][0]	***18128***	
first_array [1][1]	***18129***	
first_array [2][0]	***18130***	
first_array [2][1]	***18131***	
	18132	*data*

هذه المواقع محجوزة للمصفوفة **first_array**

مهم

لو تلاحظ أن أول عنصر للمصفوفة وهو (**first_array [0][0]**) يحجز الموقع (*18126*) *وتخزن فيه قيمة أول عنصر ويليه بقية العناصر بالتتالي أي* (**first_array [0][1]**) يحجز الموقع (*18127*) وبالتتالي....؟

عنونة عناصر المصفوفة تبدأ من الصفر للصف والعمود كما تلاحظ أول موقع بالمصفوفة هو (**first_array [0][0]**)

هيكلية
```
first_array [index_row][index_columns]
```

- (**index_row**) هو رقم الصف الذي نريد أن نصل إليه ويمثل دائما بالمتغير (i).
- (**index_columns**) هو رقم العمود الذي نريد أن نصل إليه ويمثل دائما بالمتغير (j).

مثال:لو أردنا وضع (79) في المصفوفة في الصف الثاني العمود الأول نكتب هكذا

هيكلية
```
first_array [1][0]=79;
```

لو تلاحظ أن (**index_row=1**) وليس (2) لأنه كما قلنا **تسلسل صفوف المصفوفة تبدأ من الصفر وليس من الواحد** أي الصف الثاني في الحقيقة هوا صف رقم واحد في البرمجة ولاحظ المخطط في الصفحة السابقة

لو تلاحظ أن (**index_columns=0**) وليس (1) لأنه كما قلنا **تسلسل صفوف المصفوفة تبدأ من الصفر وليس من الواحد** أي العمود الأول في الحقيقة هوا عمود رقم صفر في البرمجة ولاحظ المخطط في الصفحة السابقة

لذالك بعد أن وضعنا رقم (79) بالموقع الثالث يصبح شكل المصفوفة في مواقع الذاكرة هكذا

مواقع خلايا الذاكرة		
مواقع عناصر المصفوفة	الموقع	محتواه
	18125	data
first_array [0][0]	18126	
first_array [0][1]	18127	
first_array [1][0]	18128	79
first_array [1][1]	18129	
first_array [2][0]	18130	
first_array [2][1]	18131	
	18132	data

محتويات الموقع بعد الإضافة

هذه المواقع محجوزة للمصفوفة first_array

لو أردنا طباعة محتويات الصف الثالث العمود الثاني

الكود بلغة C++

```
Cout<< first_array [2][1];
```

الكود بلغة C

```
Printf("%d", first_array [2][1]);
```

لو أردنا تعريف مصفوفة من نوع float نفس طريقة مصفوفة السابقة وكذالك بقية الأنواع

هيكلية

```
float   first_array [5][3];
```

نستطيع أعطاء عناصر المصفوفة قيم وقت التعريف هكذا

هيكلية

```
int   first_array [2][2]={{34,26},{43,23} };
```

فيكون شكل المصفوفة في مقطع الذاكرة وهي مخزن فيها العناصر هكذا

مواقع خلايا الذاكرة		
مواقع عناصر المصفوفة	الموقع	محتواه
	18125	data
first_array [0][0]	18126	34
first_array [0][1]	18127	26
first_array [1][0]	18128	43
first_array [1][1]	18129	23

إعطاء قيم لعناصر المصفوفة وقت التعريف يكون بوضع عناصر كل صف بين قوسين ووضع فارزة بين كل صفيين

هذه المواقع محجوزة للمصفوفة first_array ومحتوياتها

مهم

لو أردنا وضع رقم معين في جميع مواقع عناصر المصفوفة وقت التعريف نكتب هكذا

مثلا إذا أردنا وضع رقم صفر في جميع مواقع مصفوفة حجمها (٢*٢)

هيكلية
```
int   first_array [2][2]={0 };
```

مهم

تعريف وإدخال المصفوفة الثنائية: هذه الخطوات ثابتة في تعريف وإدخال أي مصفوفة ثنائية لا تتغير ابد

مثال: مصفوفة حجمها (3*4) من نوع integer تعريفها وإدخالها

الكود بالغة C
```
1.int  first_arry[4][3];
2.for (i=0;i<4;i++)
3.for (j=0;j<3;j++)
4.scanf("%d",&  first_arry[i][j]) ;
```

الكود بالغة ++C
```
1.int  first_arry[4][3];
2.for (i=0;i<4;i++)
3.for (j=0;j<3;j++)
4.cin>>  first_arry[i][j] ;
```

الذي يتغير بين سؤال وأخر فقط المكتوب بخط عريض .

١.في خطوة رقم (١) عرفنا نوع المصفوفة ممكن أن يتبدل نوعها حسب الطلب وحجمها حسب السؤال قال في السؤال (3*4) فكونا مصفوفة عدد صفوفها أربعة وعدد الأعمدة ثلاثة

٢.خطوة رقم (٢) فقط شرط عدد الصفوف يتبدل هنا أربع صفوف كتبنا اصغر من أربعة وإذا خمسة نكتب اصغر من خمسة وكذالك بقية الأرقام

٣.خطوة رقم (٢) فقط شرط عدد الأعمدة يتبدل هنا ثلاث أعمدة كتبنا اصغر من ثلاثة وإذا أربعة نكتب اصغر من أربعة وكذالك بقية الأرقام

٤.خطوة الإدخال أيضا تبقى ثابتة يدخل صف صف بالتسلسل

مهم

لحل أي سؤال يتعلق بالمصفوفات الثنائية نرسمها أولا قبل الحل فلو جاء على سبيل المثال في السؤال مصفوفة حجمها (5*5) نرسم هذه المصفوفة أولا هكذا

القطر الرئيسي

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	**(3,1)**	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

ألان نستطيع حل جميع الأسئلة عن المصفوفة هذه فعلى سبيل المثال **لو قال جد عناصر القطر الرئيسي** وهو الذي نحن واضعون خط عليه وهو منصف المصفوفة ما الذي يمكننا من تميزه عن غيره من الصفوف ؟؟

الذي يميز عناصر القطر الرئيسي عن غيره من العناصر في المصفوفة هو **قيمة الصف تساوي قيمة العمود** لاحظ القيم **((0,0),(1,1),(2,2),(3,3) ,(4,4)** فيكون الشرط هو ثابت في كل مصفوفة نريد إيجاد قطر الرئيسي.

البرمجة

```
If(i==j)
// print the diagonal  هنا نقوم  بطباعة العناصر المصفوفة في قطر الرئيسي
```

قيمة (i) هي عنوان الصف وقيمة (j) هي عنوان العمود

** الحصول على عناصر القطر الثانوي

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	**(3,1)**	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

القطر الثانوي

الذي يميز عناصر القطر الثانوي عن غيرها من العناصر في المصفوفة أن قيمة الصف عند جمعها بقيمة العمود تساوي أربعة لاحظ **(4+0=4),(3+1=4),(2+2=4),(1+3=4),(0+4=4)** فعلا حاصل جمع رقم الصف مع رقم العمود في القطر الثانوي يساوي أربعة لكن يختلف من مصفوفة إلى أخرى هذه القيمة الجمع حسب إبعاد المصفوفة أنت ارسم المصفوفة حسب حجمها وستجد العلاقة كما وجدناها نحن ألان .فيكون الشرط هكذا وهو غير ثابتة قيمة الأربعة

البرمجة

```
If(i+j=4)
```

**الحصول على العناصر فوق القطر الرئيسي

العناصر فوق القطر الرئيسي

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	**(3,1)**	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

الذي يميز عناصر فوق القطر الرئيسي عن غيرها من العناصر في المصفوفة أن قيمة العمود اكبر من قيمة الصف لاحظ التالي **((3,4),(1,4),(1,3),(1,2),(0,4),(0,3),(0,2),(0,1)) فعلا قيم الأعمدة جميعها أعلى من قيم المصفوفة في القطر الثانوي فيكون الشرط هكذا وهو ثابت**

البرمجة

```
If(i<j)
```

**الحصول على العناصر تحت القطر الرئيسي

العناصر تحت القطر الرئيسي

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	**(3,1)**	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

الذي يميز عناصر تحت القطر الرئيسي عن غيرها من العناصر في المصفوفة أن قيمة كل العمود اقل من قيمة كل الصف لاحظ التالي **(1,0),(2,0),(3,0),(4,0),(2,1),(3,1),(4,1),(4,3)) فعلا قيم الأعمدة جميعها أعلى من قيم المصفوفة في القطر الثانوي فيكون الشرط هكذا وهو ثابت**

البرمجة

```
If(i>j)
```

**الحصول على العناصر فوق القطر الثانوي

العناصر فوق القطر الثانو ى

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	(3,1)	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

الذي يميز عناصر فوق القطر الثانوي عن غيرها من العناصر في المصفوفة أن قيمة العمود عند جمعها مع قيمة الصف لا يتجاوز ناتج الجمع ثلاثة بينما بقية العناصر تتجاوز ثلاثة لاحظ التالي
(0,0),(0,1),(0,2),(0,3),(1,0),(1,1),(1,2),(2,0),(2,1),(3,0)) . فيكون الشرط هكذا وهو غير ثابت **من مصفوفة إلى أخرى حسب حجمها**

البرمجة

```
If(i+j<4)
```

**الحصول على العناصر تحت القطر الثانوي

العناصر تحت القطر الثانو ى

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	(3,1)	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

الذي يميز عناصر تحت القطر الثانوي عن غيرها من العناصر في المصفوفة أن قيمة العمود عند جمعها مع قيمة الصف يتجاوز ناتج الجمع فوق الأربعة بينما بقية العناصر لا تتجاوز الأربعة لاحظ التالي **(4,1),(4,2),(4,3),(4,4),(3,3),(3,3),(3,4),(2,3),(2,4),(1,4))**. فيكون الشرط هكذا وهو غير ثابت **من مصفوفة إلى أخرى حسب حجمها**

البرمجة

```
If(i+j>4)
```

- الحصول على العناصر العمود الأول والرابع

عناصر العمود الأول والرابع

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	(3,1)	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

ما يميز عناصر العمود الأول عن غيره من الأعمدة أن قيمة (j=0) ما يميز عناصر العمود الرابع عن غيره من الأعمدة أن قيمة(j=3) فشرط الحصول عليهما وهو ثابت

البرمجة

```
If((j==0)||(j==3))
```

الحصول على عناصر الصف الأول و الثالث والرابع

عناصر الصف الأول والثالث والرابع

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	(3,1)	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

ما يميز عناصر الصف الأول عن غيره من الصفوف أن قيمة (i=0) وما يميز عناصر الصف الثالث عن غيره من الصفوف أن قيمة (i=2) و ما يميز عناصر الصف الرابع عن غيره من الصفوف أن قيمة (i=3) فشرط الحصول عليهما وهو ثابت

البرمجة

```
If((i==0)||(i==2) ||(i==3))
```

مثال: مصفوفة مربعة (5*5) أطبع فقط عناصر القطر الرئيسي ؟

تحليل :نعلم أن شرط الحصول على عناصر القطر الرئيسي في أي مصفوفة هو ثابت وهو **(i==j)** وللكثير من المعلومات راجع المخطط السابق عن الحصول على القطر الرئيسي

البرمجة بلغة c	البرمجة بلغة c++

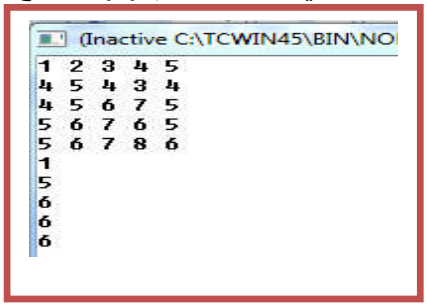
#include<stdio.h>
main()
{1.int i ;
2.int j;
3.int a[5][5];
4.for(i=0;i<5;i++)
5.for(j=0;j<5;j++)
6.scanf("%d",&a[i][j]);
7.for(i=0;i<5;i++)
8.for(j=0;j<5;j++)
9.if(i==j)
10.prin ("%d \n ",a[i][j]);}
	#include<iostream.h>
	main()
	{1.int i;
	2.int j;
	3.int a[5][5];
	4.for(i=0;i<5;i++)
	5.for(j=0;j<5;j++)
	6.cin>>a[i][j];
	7.for(i=0;i<5;i++)
	8.for(j=0;j<5;j++)
	9.if(i==j)
	10.cout<<a[i][j]<<"\n";}

توضيح الخطوات:

١.خطوة رقم (٣ و٤و٥و٦) هي تعريف وإدخال المصفوفة وهي ثابتة في كل برنامج ندخل فيه مصفوفة حسب حجم المصفوفة فقط تتغير قيم توقف العدادان كما قلنا سابقا

٢.خطوة رقم(٧ و ٨) عدادان يمران على جميع عناصر المصفوفة بالتسلسل صف صف لكي نتحقق من الشرط فأي **رقم يقع في القطر الرئيسي** سيكون عندها قيمة العدادان **(i==j)** لذالك سيحقق الشرط في الخطوة رقم (٩) ويطبع الرقم في الخطوة رقم (١٠)

كما في الشكل



معلومة

*جميع الأمثلة التي حليناها في المصفوفات الأحادية نستطيع استخدامها في المصفوفات الثنائية فقط نبدل البعد الواحد إلى بعدين والعداد إلى عدادين

*لو أردنا ضرب جميع عناصر المصفوفة برقم معين مثلا (٢) نضرب هكذا

البرمجة
a[i][j]=2*a[i][j];

وكذالك بقية العمليات جمع عنصر مع كل عناصر المصفوفة أو ضربه

لا تنسى أن تضع عداد للصفوف وعداد لأعمدة المصفوفة قبل هذه الخطوة ليضر عنصر عنصر

مثال: مصفوفة مربعة (5*5) اجمع العناصر فوق القطر الرئيسي وجمع العناصر تحته وجمع العناصر فوق القطر الثانوي وتحته **كل على حدة**

تحليل:ارجع إلى خطوات التحليل السابقة وستفهم الشروط المراد إيجادها وطريقة تحليلها ورسم المصفوفة .

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h>	#include<iostream.h>
main()	main()
{i1.int i,j,sum,sum1,sum2,sum3;	{1.int i,j,sum,sum1,sum2,sum3;
2.sum=sum1=sum2=sum3=0;	2.sum=sum1=sum2=sum3=0;
3.int a[5][5];	3.int a[5][5];
4.for(i=0;i<5;i++)	4.for(i=0;i<5;i++)
5.for(j=0;j<5;j++)	5.for(j=0;j<5;j++)
6.scanf("%d",&a[i][j]);	6.cin>>a[i][j];
7.for(i=0;i<5;i++)	7.for(i=0;i<5;i++)
8.for(j=0;j<5;j++) {	8.for(j=0;j<5;j++) {
9.if(i<j)	9.if(i<j)
10.sum+=a[i][j];	10.sum+=a[i][j];
11.if(i>j)	11.if(i>j)
12.sum1+=a[i][j];	12.sum1+=a[i][j];
13. if((i+j)<4)	13. if((i+j)<4)
14sum2+=a[i][j];	14sum2+=a[i][j];
15if((i+j)>4)	15if((i+j)>4)
16.sum3+=a[i][j];}	16.sum3+=a[i][j];}
17.prin ("\n sum above secondary diagonal= %d ",sum2);	17.cout<<"\n sum above secondary diagonal= "<<sum2;
18. prin ("\n sum above main diagonal=%d ",sum);	18.cout<<"\n sum above main diagonal= "<<sum;
19.prin ("\n sum under main diagonal= %d ",sum1);	19.cout<<"\n sum under main diagonal= "<<sum1;
20.prin ("\n sum under secondary diagonal=%d ",sum3);}	20.cout<<"\n sum under secondary diagonal= "<<sum3;}

توضيح الخطوات:

١.خطوة رقم (٣ و٤و٥و٦) هي تعريف وإدخال المصفوفة وهي ثابتة في كل برنامج ندخل فيه مصفوفة حسب الحجم

٢.خطوة رقم(٧ و٨)عدادان يمران على جميع عناصر المصفوفة بالتسلسل صف صف لكي نتحقق من الشروط

٣.خطوة رقم (٩) هو شرط لجمع العناصر فوق القطر الرئيسي أي أذا جاء أي عنصر ضمن عناصر فوق القطر الرئيسي سينفذ الخطوة رقم (١٠) لكي يجمعه ببقية عناصر فوق القطر الرئيسي

٤.خطوة رقم (١١) هو شرط لجمع العناصر تحت القطر الرئيسي أي إذا جاء أي عنصر ضمن عناصر تحت القطر الرئيسي سينفذ الخطوة رقم (١٢) لكي يجمعه ببقية عناصر تحت القطر الرئيسي

٥.خطوة رقم (١٣) هو شرط لجمع العناصر فوق القطر الثانوي أي إذا جاء أي عنصر ضمن عناصر فوق القطر الثانوي سينفذ الخطوة رقم (١٤) لكي يجمعه ببقية عناصر فوق القطر الثانوي

٦.خطوة رقم (١٥) هو شرط لجمع العناصر تحت القطر الثانوي أي إذا جاء أي عنصر ضمن عناصر تحت القطر الثانوي سينفذ الخطوة رقم (١٦) لكي يجمعه ببقية عناصر تحت القطر الثانوي

مثال: تكوين مصفوفة(٤*٤) كما في الشكل

تحليل: كما قلنا سابقا نرسم المصفوفة في البداية

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	**(3,2)**	(3,3)

```
(Inactive C:\TCWIN45  HUssien

1   1   1   1
1   0   0   1
1   0   0   1
1   1   1   1
```

كما نلاحظ كن الرسم أننا نريد أن نضع قيمة واحد في الصف الأول والصف الرابع والعمود الأول والعمود الرابع والمواقع خلافهما نضع فيها صفر.**وان ما يميز الصف الأول عن باقي الصفوف أن قيمة (i=0) وما يميز الصف الرابع عن بقية الصفوف أن قيمة (i=3) وان ما يميز العمود الأول عن باقي الأعمدة أن قيمة (j=0) وما يميز العمود الرابع عن بقية الأعمدة أن قيمة (j=3)** وسيكون لذالك الشرط يجمع بين هذه الشروط الأربعة كما في البرنامج

البرمجة بلغة c

```
#include<stdio.h>
 main()
{
1.int i,j;
2.int a[4][4]={0};
3.for(i=0;i<4;i++){
4.for(j=0;j<4;j++){
5.if((j==0)||(i==3)||(j==3)||(i==0))
6.a[i][j]=1;
7.printf("%d\t", a[i][j] );}
 8. printf("\n");}
}
```

البرمجة بلغة c++

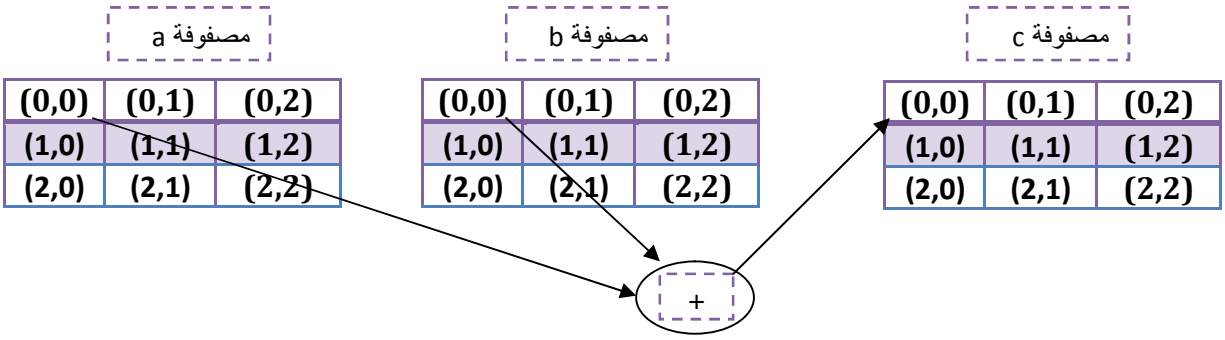
```
#include<iostream.h>
 main()
{
1.int i,j;
2.int a[4][4]={0};
3.for(i=0;i<4;i++){
4.for(j=0;j<4;j++){
5.if((j==0)||(i==3)||(j==3)||(i==0))
6.a[i][j]=1;
7.cout<< a[i][j]<<"\t";}
8.cout<<"\n";}
}
```

توضيح الخطوات:

١. خطوة رقم (٢) عرفنا مصفوفة حجمها (٤*٤) وخزنا صفر في جميع مواقعها
٢. خطوة رقم(٣ و٤) عداد يمر على جميع عناصر المصفوفة صف صف
٣. خطوة رقم(٥) هو شرط اختيار إذا كان الصف الأول أو الصف الرابع أو العمود الأول أو العمود الرابع سوف ينفذ خطوة رقم (٦) ليضع واحد بد الصفر في الموقع
٤. خطوة رقم (٧) طبع محتويات المصفوفة
٥. خطوة رقم (٨) وهي خطوة النزول إلى سطر جديد بعد طباعة صف كامل حتى يكون شكل المصفوفة المطبوعة بالشكل المطلوب ومطابقة لهيكلية المصفوفات الثنائية **ولكي تتأكد من موقعها قبل أن تغلق قوس العبارة التكرارية الخاصة بالعداد (i) ضعها في كل برنامج**

مثال: برنامج لجمع مصفوفتين حجمهما (3*3)

تحليل: لجمع مصفوفتين نجمع العنصر الأول بالمصفوفة الأولى مع العنصر الأول في المصفوفة الثانية وكذالك البقية



هذا الشكل يمثل جمع الموقع الأول والمواقع البقية نفس الشيء الثاني مع الثاني وبالتتابع

البرمجة بلغة c

```
#include<stdio.h>
 main()
 { 1.int i,j;
 2.int a[3][3];
 3.int b[3][3];
4. int c[3][3];
 5.printf("enter first matrixs\n");
6. for(i=0;i<3;i++)
7.for(j=0;j<3;j++)
 8.scanf("%d",&a[i][j]);
9.  printf("enter second matrixs\n");
 10.for(i=0;i<3;i++)
 11.for(j=0;j<3;j++)
12.  scanf("%d",&b[i][j]);
 13.for(i=0;i<3;i++){
14. for(j=0;j<3;j++){
15.c[i][j]=a[i][j]+b[i][j];
16. printf("%d\t",c[i][j]);}
17. printf("\n");}}
```

البرمجة بلغة c++

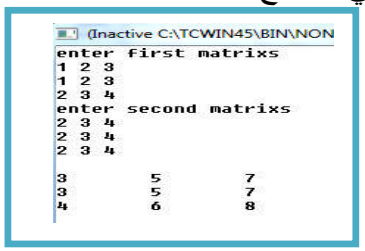
```
#include<iostream.h>
 main()
 { 1.int i,j;
 2.int a[3][3];
 3.int b[3][3];
4. int c[3][3];
 5.cout<<"enter first matrixs\n";
6. for(i=0;i<3;i++)
7.for(j=0;j<3;j++)
 8.cin>>a[i][j];
9. cout<<"enter second matrixs\n";
 10.for(i=0;i<3;i++)
 11.for(j=0;j<3;j++)
12. cin>>b[i][j];
 13.for(i=0;i<3;i++){
14. for(j=0;j<3;j++){
15.c[i][j]=a[i][j]+b[i][j];
16.cout<<c[i][j]<<"\t";}
17.cout<<"\n";}}
```

توضيح الخطوات:

١.خطوة رقم (٦ و٧ و٨) هي عملية إدخال للمصفوفة الأولى .خطوة رقم(١٠ و١١و١٢) إدخال المصفوفة الثانية

٢.خطوة خطوة رقم(١٣ و١٤) هوا عدادا يمر على جميع عناصر المصفوفة صف صف ليجمع في خطوة رقم(١٥) العنصر الأول في المصفوفة الأولى مع العنصر الأول بالمصفوفة الثانية والثاني بالثاني بالتتابع

٣.خطوة رقم (١٦) هو عملية طباعة لعناصر المصفوفة



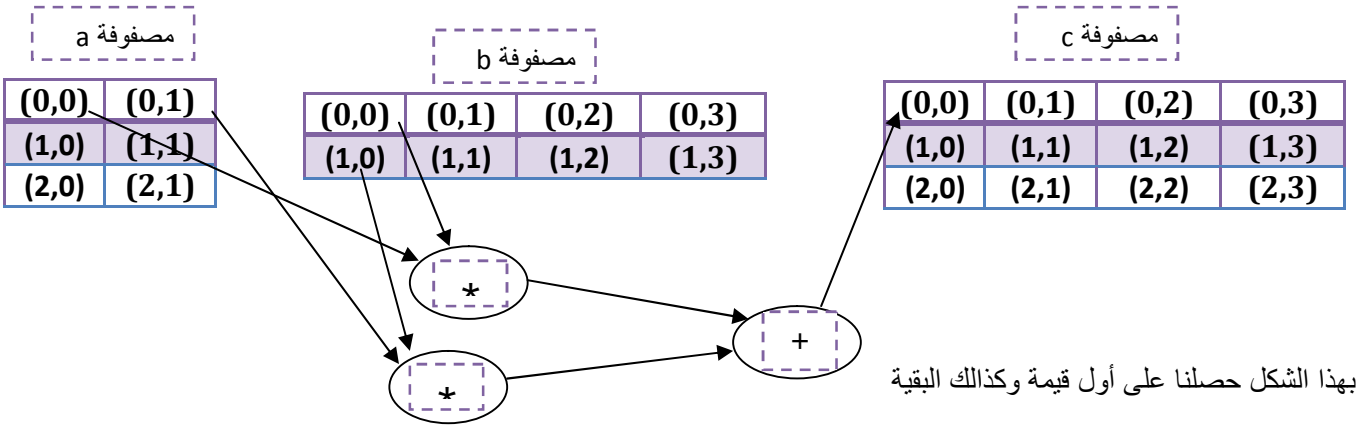
عملية طرح مصفوفتين نفس الخطوات السابقة فقط نبدل خطوة (١٥) بالتالي

البرمجة

```
15.c[i][j]=a[i][j]-b[i][j];
```

مثال: برنامج لضرب مصفوفتين (2*4)* (3*2) .؟

تحليل: لضرب مصفوفتين (2*4)* (3*2) يجب أن يكون الناتج مصفوفة بحجم (3*4) حسب قواعد الضرب



بهذا الشكل حصلنا على أول قيمة وكذالك البقية

البرمجة بلغة c

```
#include<stdio.h>
 main()
 {1.int i,j,k;
2. int a[3][2];
 3.int b[2][4];
4. int c[3][4]={0};  //put zero in every location
5. printf("enter first matrixs\n") ;
 6. for(i=0;i<3;i++)
 7.for(j=0;j<2;j++)
 8. scanf("%d",&a[i][j]);
9.prin   ("enter second matrixs\n") ;
10. for(i=0;i<2;i++)
 11.for(j=0;j<4;j++)
 12. scanf("%d",&b[i][j]);
13.for(i=0;i<3;i++){
14.for(j=0;j<4;j++){
 15.for(k=0;k<2;k++)
16.c[i][j]+=a[i][k]*b[k][j];
 17. prin   ("%d\t",c[i][j]);}
18.prin   ("\n");} }
```

البرمجة بلغة c++

```
#include<iostream.h>
 main()
 {1.int i,j,k;
2. int a[3][2];
 3.int b[2][4];
4. int c[3][4]={0};  //put zero in every location
 5.cout<<"enter first matrixs\n" ;
6. for(i=0;i<3;i++)
 7.for(j=0;j<2;j++)
 8.cin>>a[i][j];
9.cout<<"enter second matrixs\n" ;
10. for(i=0;i<2;i++)
 11.for(j=0;j<4;j++)
 12.cin>>b[i][j];
13.for(i=0;i<3;i++){
14.for(j=0;j<4;j++){
 15.for(k=0;k<2;k++)
 16.c[i][j]+=a[i][k]*b[k][j];
17. cout<<c[i][j]<<"\t" ;}
18.cout<<"\n";} }
```

توضيح الخطوات:

١.خطوة رقم(٤) عرفنا مصفوفة ووضعنا صفر في جميع مواقعها لأننا سنخزن فسها نتيجة الضرب وكما ترى في المخطط يضرب ثم يجمع أي توجد عملية جمع أكثر من مرة لذالك يجب تصفير المواقع حتى لا تؤثر على الجمع

٢.خطوة رقم (١٣ و١٤) هوا التحرك ببعد المصفوفة الجديدة وهي بحجم (3*4) وبعدها خطوة رقم (١٥) هوا البعد المفقود الذي سيفقد في عملية الضرب وخطوة رقم (١٦) هو كما موضح في المخطط

أي إننا متى ما جاءتنا عملية ضرب مصفوفتين نأخذ عدادان بأبعاد مصفوفة ناتجة وعداد بالبعد المفقود وبعدها خطوة رقم(١٦) أي لو ضربنا المصفوفات التالية (3*6)*(6*4) وناتج يكون مصفوفة(3*4) والبعد المفقود (6)

مهم

البرمجة

```
13.for(i=0;i<4;i++){
14.for(j=0;j<3;j++){
 15.for(k=0;k<6;k++)
 16.c[i][j]+=a[i][k]*b[k][j]
```

٤.٤- ترتيب المصفوفات ثنائية الأبعاد:

لا يختلف ترتيب المصفوفات الثنائية عن ترتيب المصفوفات الأحادية في شيء نفس الصيغة نأخذ أول عنصر في المصفوفة ونقارنه مع بقية العناصر والثاني وبالتتالي لذالك راجع ترتيب المصفوفات الأحادية أولا ...

مثال: ترتيب مصفوفة ثنائية الإبعاد (5*5) ترتيب تنازليا

البرمجة بلغة c

```
#include<stdio.h>
 int main()
 {1.int const row=5;
 2.int const col=5;
3. int array[row][col];
 4.int i,j,k,x,l ;
5.printf("Here is the Array befor sorted\n");
6. for ( i=0;i<row;i++)
 7. for ( j=0;j<col;j++)
8.scanf("%d",&array[i][j]);
9.for( k=0;k<row;k++){
10. for( l=0;l<col;l++){
11. for( i=0;i<row;i++){
 12. for ( j=0;j<col;j++){
 13. if (array[i][j] < array[k][l]){
14.x=array[k][l];
 15.array[k][l]=array[i][j];
 16.array[i][j]=x;
 17.}} }} }
18.printf("Here is the Array after sorted\n");
19. for ( i=0;i<row;i++){
 20. for ( j=0;j<row;j++)
21.printf("%d\t",array[i][j]);
22.printf("\n");} }
```

البرمجة بلغة c++

```
#include<iostream.h>
 int main()
 {1.int const row=5;
 2.int const col=5;
3. int array[row][col];
 4.int i,j,k,x,l ;
5.cout<<"Here is the Array befor sorted\n" ;
6. for ( i=0;i<row;i++)
 7. for ( j=0;j<col;j++)
8.cin>>array[i][j] ;
9.for( k=0;k<row;k++){
10. for( l=0;l<col;l++){
11. for( i=0;i<row;i++){
 12. for ( j=0;j<col;j++){
 13. if (array[i][j] < array[k][l]){
14.x=array[k][l];
 15.array[k][l]=array[i][j];
 16.array[i][j]=x;
 17.}} }} }
18. cout<<"Here is the Array after sorted\n" ;
19. for ( i=0;i<row;i++){
 20. for ( j=0;j<row;j++)
21. cout<<array[i][j]<<"\t";
22. cout<<"\n" ;} }
```

توضيح الخطوات:

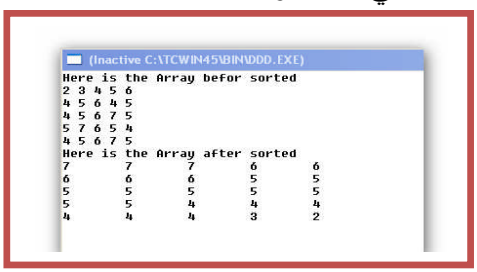
مهم

١.خطوة رقم (١و٢) هو الإعلان عن أبعاد المصفوفة وهذه الصيغة مهمة جدا للمطور ففي البرامج السابقة لو أردت فقط نفس المثال فقط تغير أبعاد المصفوفة لكنت بحاجة لتغير بعد المصفوفة وشروط توقف العدادات في كل الخطوات أما هنا إذا أردنا أن نغير بعد المصفوفة مثلا قال رتب مصفوفة (4*4) فقط نغير قيمة (row=4) و(col=4) ويتحول السؤال كله كما نريد بدون أي خطاء

٢.خطوة رقم(٩ و١٠) عدادان يمران على جميع عناصر المصفوفة لكي يقارن كل عنصر بجميع العناصر التي تليه بواسطة العدادان في الخطوة (١١و١٢)

٣.خطوة رقم(١٣ و١٤و١٥و١٦) هي عملية أبدال بين موقعين شرحت سابقا في المصفوفة الأحادية

٤.خطوة رقم(١٩ و٢٠و٢١و٢٢)طباعة للمصفوفة بعد الترتيب



مثال: ترتيب صفوف مصفوفة ثنائية الإبعاد (5*5) ترتيب تنازليا

يقارن عناصر الصف الواحد ليرتبها ١.الأول مع الجميع

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	(3,1)	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

تحليل:لترتيب المصفوفة كل صف على حدة نحن بحاجة لعداد خارجي يقف على عناصر المصفوفة صف صف وليكن اسمه(k) ويبدأ بترتيب عناصر الصف الواحد بمقارنة العنصر الأول مع جميع العناصر التي تليه في الصف الذي عليه المؤشر العداد ثم بعد أن يرتبه ينتقل إلى الصف التالي.

عندما يكون الترتيب على الصف الأول (k=0)

البرمجة بلغة c

```
#include<stdio.h>
 int main()
 {1.int const row=5;
2. int const col=5;
 3.int array[row][col];
4.int i,j,k,x ;
5.printf("Here is the Array befor sorted\n");
6. for ( i=0;i<row;i++)
 7. for ( j=0;j<col;j++)
8.scanf("%d",&array[i][j]);
 9.for( k=0;k<row;k++)
 10.for( i=0;i<row;i++)
  11.for ( j=0;j<col;j++){
 12. if (array[k][j] <array[k][i]){
 13. x=array[k][j];
 14.array[k][j]=array[k][i];
 15.array[k][i]=x;}}
16.printf("Here is the Array after sorted\n");
 17.for ( i=0;i<row;i++){
18. for ( j=0;j<row;j++)
19.printf("%d\t",array[i][j]);
20.printf("\n");} }
```

البرمجة بلغة c++

```
#include<iostream.h>
 int main()
 {1.int const row=5;
2. int const col=5;
 3.int array[row][col];
4.int i,j,k,x ;
5.cout<<"Here is the Array befor sorted\n" ;
6. for ( i=0;i<row;i++)
 7. for ( j=0;j<col;j++)
8.cin>>array[i][j] ;
 9.for( k=0;k<row;k++)
 10.for( i=0;i<row;i++)
  11.for ( j=0;j<col;j++){
 12. if (array[k][j] <array[k][i]){
 13. x=array[k][j];
 14.array[k][j]=array[k][i];
 15.array[k][i]=x;}}
16. cout<<"Here is the Array after sorted\n" ;
 17.for ( i=0;i<row;i++){
18. for ( j=0;j<row;j++)
19. cout<<array[i][j]<<"\t";
20.cout<<"\n" ;} }
```

مهم

توضيح الخطوات:

ماذا سيحصل في خطوة رقم(15—9)...؟

كيف رتبنا صفوف المصفوفة؟نجد انه يقوم بمقارنة عناصر الصف الواحد ببعضها وترتيبها حيث أن المتغير (k) ينتقل في كل لوب إلى صف جديد بعد أن ينتهي من ترتيب الصف الذي يسبقه وفي المقارنة خطوة رقم(١٢) نثبت الصف بين المصدر والمسار الترتيبي وننقل العداد الخارجي(i) في كل لوب إلى عمود جديد ضمن الصف الواحد ويعمل العداد الداخلي (j)على مقارنة هذا العصر الذي يؤشر عليه (i) في الصف الواحد بكل العناصر في الأعمدة التي تليه في نفس الصف فإذا وجد فيها اصغر منه يبدله

أتمنى أن تكون قد استوعبت الفكرة (ابومشاري)

مثال: ترتيب أعمدة مصفوفة ثنائية الإبعاد (5*5) ترتيب تنازليا

يقارن عناصر العمود الواحد ليرتبها ١.الأول مع الجميع

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
(1,0)	**(1,1)**	**(1,2)**	**(1,3)**	(1,4)
(2,0)	**(2,1)**	**(2,2)**	**(2,3)**	(2,4)
(3,0)	(3,1)	**(3,2)**	**(3,3)**	(3,4)
(4,0)	**(4,1)**	**(4,2)**	**(4,3)**	(4,4)

تحليل:لترتيب المصفوفة كل عمود على حدة نحن بحاجة لعداد خارجي يقف على عناصر المصفوفة عمود عمود وليكن اسمه (k) ويبدأ بترتيب عناصر العمود الواحد بمقارنة العنصر الأول مع جميع العناصر التي تليه في العمود الذي عليه المؤشر العداد ثم بعد إن يرتبه ينتقل إلى العمود التالي.

عندما يكون الترتيب على العمود الأول (k=0)

البرمجة بلغة c

```
#include<stdio.h>
 int main()
 {1.int const row=5;
2. int const col=5;
 3.int array[row][col];
 4.int i,j,k,x,l ;
 5.printf("Here is the Array befor sorted\n");
 6.for ( i=0;i<row;i++)
7.  for ( j=0;j<col;j++)
8.scanf("%d",&array[i][j]);
9. for( k=0;k<row;k++)
 10.for( i=0;i<row;i++)
 11. for ( j=0;j<col;j++) {
12. if (array[j][k] <array[i][k]){
13. x=array[j][k];
 14.array[j][k]=array[i][k];
 15.array[i][k]=x;}}
 16.printf("Here is the Array after sorted\n");
 17.for ( i=0;i<row;i++){
18. for ( j=0;j<row;j++)
19.printf("%d\t",array[i][j]);
20.printf("\n");}}
```

البرمجة بلغة c++

```
#include<iostream.h>
 int main()
 {1.int const row=5;
2. int const col=5;
 3.int array[row][col];
 4.int i,j,k,x,l ;
 5.cout<<"Here is the Array befor sorted\n" ;
 6.for ( i=0;i<row;i++)
7.  for ( j=0;j<col;j++)
8.cin>>array[i][j] ;
9. for( k=0;k<row;k++)
 10.for( i=0;i<row;i++)
 11. for ( j=0;j<col;j++) {
12. if (array[j][k] <array[i][k]){
13. x=array[j][k];
 14.array[j][k]=array[i][k];
 15.array[i][k]=x;}}
 16. cout<<"Here is the Array after sorted\n" ;
 17.for ( i=0;i<row;i++){
18. for ( j=0;j<row;j++)
19. cout<<array[i][j]<<"\t";
20. cout<<"\n" ;}}
```

مهم

توضيح الخطوات:٠ ٠ ٠

ماذا سيحصل في خطوة رقم(15—9)...؟

كيف رتبنا أعمدة المصفوفة؟نجد انه يقوم بمقارنة عناصر العمود الواحد ببعضها وترتيبها حيث أن المتغير (k) ينتقل في كل لوب إلى عمود جديد بعد أن ينتهي من ترتيب العمود الذي يسبقه. وفي المقارنة خطوة رقم(١٢) نثبت العمود بين المصدر والمسار الترتيبي وننقل العداد الخارجي(i) في كل لوب إلى صف جديد ضمن العمود الواحد ويعمل العداد الداخلي (j) على مقارنة هذا العنصر الذي يؤشر عليه (i) في العمود الواحد بكل العناصر في الصفوف التي تليه في نفس العمود الذي عليه المؤشر فإذا وجد فيها اصغر منه يبدله

لو أردنا ترتيب تصاعدي فقط نبدل علامة الأصغر في شرط المقارنة إلى اكبر

٥.٤- النصوص (string):

١.تمثيلها بالمصفوفات الأحادية الأبعاد:

هي مصفوفات مكونه من سلسلة من الحروف وتخزن بنفس طريقة المصفوفة الاعتيادية ويخزن بعد أخر موقع نخزن فيه المصفوفة الحرف ('\٥') للدلالة على أن السلسلة انتهت.
لو أردنا تكوين مصفوفة حرفيه اسمها(***name***) نخزن فيها اسم شخص وليكن(hussien)

هيكلية تعريف

```
Char name[8]="hussien";
```

أو تكتب هكذا لكن هنا يجب وضع ('\٥') في نهاية السلسلة

هيكلية تعريف

```
Char name[8]={'h','u','s','s','i','e','n','\0'};
```

نلاحظ إننا حجزنا ثمانية مواقع ورغم أن الاسم مكون من سبعة أحرف لأنه كما قلنا يضيف ('\0') إلى نهاية السلسلة بالإضافة إلى الأحرف التي أدخلناها (أي دائما نحجز حجم المصفوفة اكبر من ما نحتاجه بواحد)

أي سلسة تخزن في الذاكرة بالتسلسل أي أن مصفوفة name التي تحوي على "hussien" تخزن هكذا إذا افترضنا انه أول موقع يخزن فيه أول حرف هوا (**200**) كما لاحظت أن بقية الأحرف يخزنهم بالتتالي.

مواقع خلايا الذاكرة	عنونة المصفوفة	name[0]	name[1]	name[2]	name[3]	name[4]	name[5]	name[6]	
	عنوان الموقع	**200**	**201**	**202**	**203**	**204**	**205**	**206**	207
	محتواه	***h***	***u***	***s***	***s***	***i***	***e***	***n***	**\0**

*نعلم أن ترقيم المصفوفة في هذه اللغة يبدءا من الصفر أي أن حرف (h) يخزن في موقع صفر وحرف (i) يخزن في موقع ٤

لو أردنا أن نضع حرف (o) بدل حرف (u) وبما انه ثاني موقع في المصفوفة نخزنه هكذا

البرمجة

```
name[1]='o';
```

مواقع خلايا الذاكرة	عنونة المصفوفة	name[0]	name[1]	name[2]	name[3]	name[4]	name[5]	name[6]	
	عنوان الموقع	**200**	**201**	**202**	**203**	**204**	**205**	**206**	207
	محتواه	***h***	***o***	***s***	***s***	***i***	***e***	***n***	**\0**

هذا الحرف الذي تم أبداله

- المكتبة التي نستخدمها مع (string) هي <string.h>

دوال الإدخال والإخراج (string):

في حال إدخال **(string)** من قبل المستخدم .عبارات الإدخال الاعتيادية تكون غير كافية لإدخالها أو غير ملائمة لدرجة 100%. على سبيل المثال دالة الإدخال الاعتيادية ونحاول إدخال **(string) بها ماذا سيحدث**

دالة الإدخال بلغة C++

```
char name[30];
scanf("%s",name);
```

دالة الإدخال بلغة C

```
char name[30];
cin>>name ;
```

سوف يأخذ الجملة المدخلة لكن ليس كلها يأخذها حتى أول فراغ بالإدخال أي لو أدخلنا

الفراغ الأول

Hussien Ahmmed Taleb

لذالك سوف يأخذ المدخلات فقط كلمة (Hussien) وبهمل من الفراغ الأول إلى النهاية

مهم

إذا أدخلنا عدة كلمات فما الذي سوف يحدث للذي مهلته دالة الإدخال الأولى..؟

الذي مهلته دالة الإدخال الأولى لن يذهب سدى **إنهما يبقى ينتظر عملية إدخال ثانية ليكون هو كمدخل لها**(أي في عملية الإدخال الثانية سوف لا يطلب من المستخدم إدخال هو يعتبر ما تبقى من دالة الإدخال الأولى كمدخل لدالة الإدخال الثانية) أي لو رجعنا إلى المثال السابق ووضعنا دالة إدخال أخرى وأدخلنا نفس الجملة كما في المثال

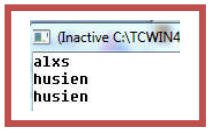
البرمجة بلغة c

```
#include<stdio.h>
 int main()
 {1.char string1[20],string2[20];
2.scanf("%s",&string1);
3 .scanf("%s",&string2);
4.prin   ("%s",string2);
}
```

البرمجة بلغة c++

```
#include<iostream.h>
 int main()
 {1.char string1[20],string2[20];
2.cin>>string1 ;
3 . cin>>string2 ;
4.cout<<string2 ;
}
```

في هذا البرنامج المفروض تنفذ خطوة رقم (٢) يدخل المستخدم (string1) ثم تنفذ خطوة رقم (٣) ويدخل المستخدم (string2) **هذه الكلام صحيح إذا لم يتجاوز المدخلات أكثر من كلمة هكذا**



لو تلاحظ شاشة التنفيذ نفذ خطوة رقم(٢) وطلب من المستخدم إدخال وادخل (alxs) ونفذ خطوة رقم(٣) وطلب من المستخدم إدخال وادخل (hussien) وبعدها نفذ خطوة رابعة وطبع الاسم الثاني

***** أما إذا ادخل المستخدم في خطوة رقم(٢) جملة مكونة من أكثر من كلمة وهي***

فالذي يحدث كما قلنا يأخذ حتى الفراغ الأول ويعتبره كمدخل لدالة الإدخال الأولى وهو(Hussien) ويترك بقية الجملة .وعندما ينفذ خطوة رقم(٣) لا يطلب من المستخدم الإدخال أنما يعتبر ما تبقى من الجملة الأولى وهو(Ahmmed Taleb) كمدخل له ويأخذ أيضا حتى الفراغ الثاني أي سوف يأخذ (Ahmmed) فقط وبهمل البقية وسوف يطبع في خطوة رقم(٤) محتويات (string2)

إذن الحل مع هذه المشاكل توجد دوال إدخال تأخذ الجملة المدخلة كاملة وهي:

(cin.get) تستخدم هذه الدالة في لغة (c++) لإدخال جملة كاملة وشكلها يكون

دالة الإدخال بلغة C++
Cin.get(string,number of input)

١ **.(string)** هي المصفوفة المراد إدخالها ك (string)

٢ ***(number of input).*** هي عدد الحروف المحتملة التي سندخلها من شاشة التنفيذ ودائما اجعلها أكثر من احتياجك **وان لا تتجاوز حجم المصفوفة**.وأننا إذا تجاوزنا الإدخال هذا الرقم سوف سيهمل البقية .

(cin.getline) نفس طريقة الدالة (cin.get) **لكنها تقرئ سطر واحد فقط**

مثال: لو كان لدينا (string1) وحجمها (٢٥) ونريد إدخالها بهذه الدالة فيكون الكود هكذا

دالة الإدخال بلغة C++
Char string1[25] Cin.get(string1,25)

(gets) تستخدم هذه الدالة في لغة (c) لإدخال جملة كاملة فقط نكتبها ونكتب اسم(string) التي نريد أن ندخلها بها

دالة الإدخال بلغة C++
Char string1[25] gets(string1)

دوال الإخراج في لغة (c++) نستخدم نفس دالة الإخراج الاعتيادية تكتب اسم (string) وستطبع سلسلة كاملة

دالة الإدخال بلغة C++
cout<<string1;

دوال الإخراج في لغة (c) نستخدم نفس دالة الإخراج الاعتيادية ونستخدم في دالة الطباعة الرمز(%s) واسم (string) وستطبع سلسلة كاملة في شاشة التنفيذ

دالة الإدخال بلغة C++
Prin ("%s",string1);

*لا نستخدم "%c" لان "%c" تطبع حرف واحد فقط وليس جملة

لنصحح المثال السابق الذي كانت به مشكلة بدوال الإدخال باستخدام هذه الدوال الجديدة (افحص المثال وشاهد الفرق)

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h> int main() {1.char string1[20],string2[20]; 2.gets(string1); 3 . gets(string2); 4.prin ("%s",string2); }	#include<iostream.h> int main() {1.char string1[20],string2[20]; 2.cin.get(string1,20) ; 3 . cin.get(string2,20) ; 4.cout<<string2 ; }

هناك عدة دوال للتعامل مع (string) تقع ضمن مكتبة <string.h> وهي:

١. ***strlen():*** تستخدم هذه الدالة لإيجاد طول السلسة النصية (أي عدد الأحرف في السلسلة وكذالك عدد الفراغات) .

وطريقة استخدامها هي

هيكلية الدالة
strlen(string)

(string) هي المصفوفة المراد إيجاد طولها

مثال:إيجاد طول المصفوفة التالية

البرمجة
char string1[5]="alxs go"; int len; len=strlen(string1); // len=7

يكون طول المصفوفة (٧) لان حتى الفراغ أيضا يعتبر كحرف في حساب الطول السلسلة

مثال: برنامج ندخل سلسلة ويطبعها بالمقلوب .؟

تحليل:لطباعة أي مصفوفة بالمقلوب(أي أخر حرف يطبع أول حرف) يكون بوضع المؤشر على أخر حرف ثم نتنازل إلى أول حرف بالتسلسل فتتم الطباعة ونستطيع معرفة أخر حرف بواسطة دالة معرفة طول السلسة سيعطيك طولها ونطرح من طولها واحد نجد أخر رقم بالمصفوفة لان ترقيم المصفوفة يبدأ من الصفر

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h> #include<string.h> int main() {1.char string1[40]; 2.int i,len; 3 . gets(string1); 4.len=strlen(string1)-1; 5.for(i=len ;i>=0;i--) 6.prin ("%c",string1[i]);}	#include<iostream.h> #include<string.h> int main() {1.char string1[40]; 2.int I,len; 3 . cin.get(string1,40) ; 4.len=strlen(string1)-1; 5.for(i=len ;i>=0;i--) 6.cout<<string1[i];}

توضيح الخطوات:

١.خطوة رقم(١) عرفنا مصفوفة نصية،

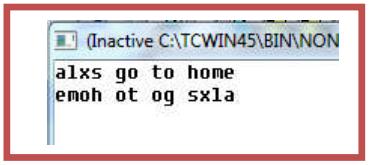
٢.خطوة رقم(٢) عرفنا عداد للمصفوفة ومتغير (len) لكي نخزن فيه طول السلسلة

٣.خطوة رقم (٣) قمنا بإدخال السلسة بواسطة دوال الإدخال

٤.خطوة رقم(٤) خزن طول السلسلة بالمتغير (len) وطرنا من طول السلسلة واحد لان ترقيم المصفوفة يبدأ من الصفر وطول المصفوفة يعطيك عدد الأحرف في المصفوفة لذالك يجب طرحه بواحد

٥.عداد يبدأ بالعد من أخر عنصر بالمصفوفة ويتناقص إلى أول عنصر ويطبع كل عنصر في خطوة(٦)

*** ولو تلاحظ في الطباعة بلغة(c) استخدمنا "%c" لأننا سنطبع حرف حرف وليس سلسلة**



٢. ***strcpy():*** تستخدم هذه الدالة لنسخ جميع محتويات سلسلة إلى سلسلة أخرى وتكون طريقة النسخ انه يبدأ بإضافة عناصر السلسلة الثانية بمكان عناصر السلسلة الأولى التي لها نفس التسلسل بالموقع

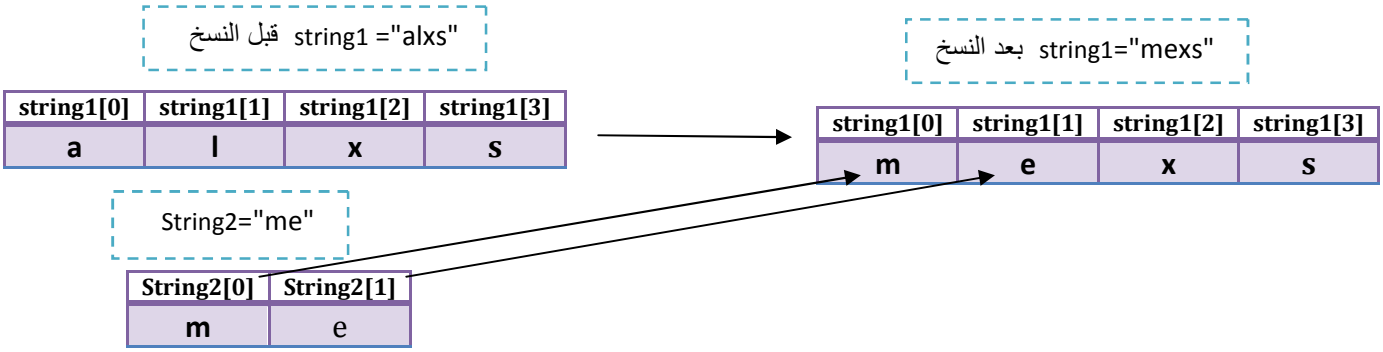
هيكلية الدالة

```
strcpy(string1, string2);
```

مثال: لنسخ جميع محتويات مصفوفة إلى مصفوفة أخرى

البرمجة

```
char string1[5]="alxs ";
char string2[5]="me ";
strcpy(string1, string2);
```



لو تلاحظ أن (String2) بقيت محافظة على محتوياتها نفسها بعد النسخ لأن النسخ يكون منها إلى (string1) وان (string1) تم أبدال محتويات كل موقع بما يكافئه في(String2) وبقيت المواقع التي لا يقابلها قيم من (String2) محتفظة بقيمها

٣. ***strncpy():*** تستخدم هذه الدالة **لنسخ عدد محدد من** محتويات سلسلة إلى سلسلة أخرى وتكون طريقة النسخ انه يبدأ بإضافة عناصر السلسلة الثانية بمكان عناصر السلسلة الأولى التي لها نفس التسلسل بالموقع

هيكلية الدالة

```
strncpy(string1, string2,number of copy);
```

(number of copy) هو عدد الأحرف المراد نسخها من السلسلة الثانية إلى السلسلة الأولى

مثال: لنسخ ثلاث عناصر من محتويات مصفوفة إلى مصفوفة أخرى

البرمجة

```
char string1[5]="alxs ";
char string2[5]="suha muhamed ";
strncpy(string1, string2,3);
```

سوف ينسخ الحروف الثلاثة الأولى من السلسلة الثانية وهي(suh) إلى السلسة الأولى ويكون محتويات (string1=suhs)

٤. ***strcat():*** تستخدم هذه الدالة للإلحاق محتويات سلسلة في نهاية سلسلة أخرى ومحافظا على محتوياتها .

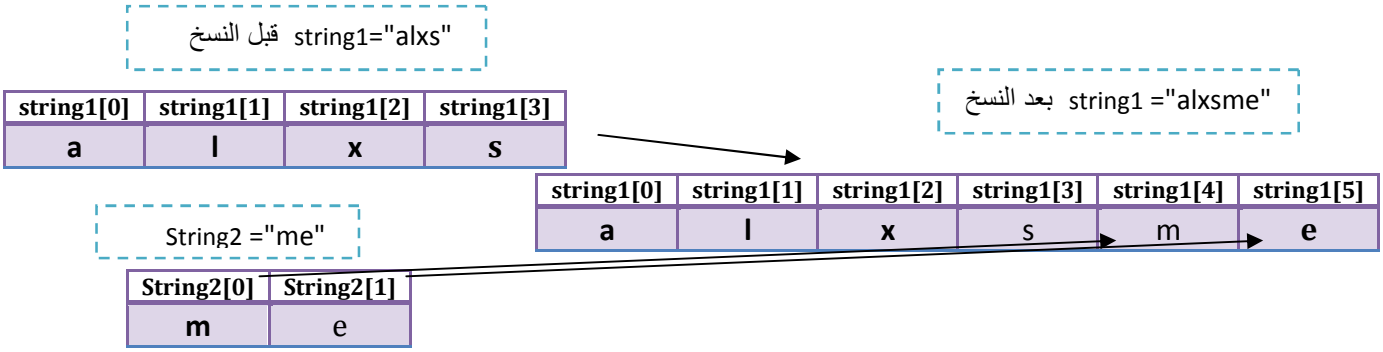
هيكلية الدالة

```
strcat(string1, string2);
```

مثال: لإلحاق جميع محتويات مصفوفة إلى مصفوفة أخرى

البرمجة

```
char string1[7]="alxs ";
char string2[5]="me ";
strcat(string1, string2);
```



لو تلاحظ أن (String2) بقيت محافظة على محتوياتها نفسها بعد الدمج لأن الإلحاق يكون منها إلى (string1) وان (string1) تم إضافة محتويات سلسلة(String2) إلى نهايتها . يجب أن يكون حجم المصفوفة المراد الإضافة إليها مساوي لعدد أحرفها وعدد الأحرف المضافة لو تلاحظ أن في المثال وضعنا حجم(string1) هو(٧) وهي مكونة من أربعة أحرف لأننا سنضيف إليها سلسلة مكونة من حرفان فيصبح طولها سبعة

٥. ***strncat():*** تستخدم هذه الدالة **بإلحاق عدد محدد من** محتويات سلسلة في نهاية سلسلة أخرى وتكون طريقة

هيكلية الدالة

```
strncat(string1, string2,number of copy);
```

(number of copy) هو عدد الأحرف المراد نسخها من السلسلة الثانية إلى السلسلة الأولى

مثال: لإلحاق ثلاث عناصر من محتويات مصفوفة إلى مصفوفة أخرى

البرمجة

```
char string1[5]="alxs ";
char string2[5]="suha muhamed ";
strncat(string1, string2,3);
```

سوف ينسخ الحروف الثلاثة الأولى من السلسلة الثانية وهي(suh) إلى السلسة الأولى ويكون محتويات (string1=alxssuh)

٦. ***strcmp():*** تستخدم هذه الدالة للمقارنة بين سلسلتين وتكون بالشكل التالي

هيكلية الدالة

```
strcmp(string1, string2);
```

هناك ثلاثة نتائج للمقارنة بين سلسلتين وهي.

١. فإذا كانت نتيجة المقارنة صفر فأن (String1) تساوي (String2)

٢. فإذا كانت نتيجة المقارنة اكبر من صفر فأن (String1) اكبر من (String2)

٣. فإذا كانت نتيجة المقارنة اصغر من صفر فأن (String1) اصغر من (String2)

مثال: للمقارنة بين سلسلتين

البرمجة

```
char string1[7]="aa ";
char string2[5]="ab ";
int cmper;
cmper=strcmp(string1, string2) ; //cmper<1
```

كانت نتيجة المتغير (cmper) سالبة لان (String1) اصغر من (String2)

٧. ***strncmp():*** تستخدم هذه الدالة **للمقارنة بين عدد محدد من الأحرف من سلسلتين**

هيكلية الدالة

```
strncmp (string1, string2,number of comper);
```

(number of comper) هو عدد الأحرف المراد مقارنتها من كلا السلسلتين

مثال: لمقارنة أول عنصر من محتويات مصفوفة الأولى والمصفوفة الثانية

البرمجة

```
char string1[5]="hussien ";
char string2[5]="hakmet ";
int cmper;
cmper=strncmp(string1, string2,1) ; //cmper=0
```

كانت نتيجة المتغير (cmper) صفر لان الحرف الأول في (String1) يساوي الحرف الأول في (String2)

مثال: برنامج ندخل سلسلة حرف ويحسب عدد مرات ظهور الحرف (a) في السلسلة

تحليل:بما انه قال سلسلة حروف نستخدم دوال الإدخال الخاصة ب(string) وبعدها نضع شرط بسيط للتحقق إذا كان احد الحروف هو (a) ونزيد عداد في كل مرة يجد فيها حرف (a)

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h> #include<string.h> int main() {1.char string1[40]; 2.int i,len,number_appear=0; 3 . gets(string1); 4.len=strlen(string1)-1; 5.for(i=0;i<=len;i++) 6.If (string1[i]=='a') 7. number_appear=number_appear+1; 8.prin ("number appear of (a)=%d", number_appear);}	#include<iostream.h> #include<string.h> int main() {1.char string1[40]; 2.int i,len,number_appear=0; 3 . cin.get(string1,40) ; 4.len=strlen(string1)-1; 5.for(i=0;i<=len;i++) 6.If (string1[i]=='a') 7. number_appear=number_appear+1; 8.cout<<"number appear of (a)="<< number_appear;}

توضيح الخطوات:

١.خطوة رقم (٥) كونا عداد يبدأ بأول حف بالسلسلة وينتهي بأخر حرف لكي يتحقق من أحرف السلسلة حرف حرف متى ما وجد حرف (a) ستحقق الشرط في الخطوة رقم(٦) وينفذ خطوة رقم (٧) يزيد فيها قيمة العداد بواحد دلالة على انه وجد حرف جديد وكلما يجد الحرف يزيد العداد وفي النهاية تكون قيمة العداد بقدر عدد مرات ظهور الحرف (a) في السلسلة

```
(Inactive C:\TCWIN45\BIN\NONAME01.EXE)
hussien ahmed taleb is ago to his stage
number appear of (a)=4
```

مثال: برنامج ندخل سلسلة حرف ويحسب عدد مرات ظهور أحرف العلة وطباعتها أينما وجدت في السلسلة.؟

تحليل: فكرة هذا السؤال نفس فكرة السؤال السابق فقط الذي يغير شرط التحقق فبدلا من أن يتحقق من حرف واحد سيتحقق من خمس حرف وأينما وجدها يطبعها

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h> #include<string.h> int main() {1.char string1[40]; 2.int i,len,number_appear=0; 3 . gets(string1); 4.len=strlen(string1)-1; 5.for(i=0;i<=len;i++) 6.If ((string1[i]=='a')||(string1[i]=='u')||(string1[i]=='o')||(string1[i]=='i')||(string1[i]=='e')){ 7.prin ("%c",string1[i]); 8. number_appear=number_appear+1;} 9.printf("number appear of vowel =%d\n", number_appear);}	#include<stdio.h> #include<string.h> int main() {1.char string1[40]; 2.int i,len,number_appear=0; 3 . cin.get(string1,40) ; 4.len=strlen(string1)-1; 5.for(i=0;i<=len;i++) 6.If ((string1[i]=='a')||(string1[i]=='u')||(string1[i]=='o')||(string1[i]=='i')||(string1[i]=='e')){ 7.cout<<string1[i]<<"\n"; 8. number_appear=number_appear+1;} 9.cout<<"number appear of vowel="<< number_appear;}

توضيح الخطوات: الشرط في خطوة رقم (٦) يتحقق متى ما جاء أي حرف من حروف العلة الخمسة يطبعه في خطوة رقم (٧) ويزيد قيمة عداد عدد مرات ظهور أحرف العلة في خطوة رقم(٨).

٢.استخدام الحروف في المصفوفات الثنائية:

لا يختلف استخدام الحروف في المصفوفات الثنائية عن استخدام الأرقام في المصفوفات الثنائية تستخدم في الإدخال حرف حرف مكونة مصفوفة حرف وحتى في الطباعة تطبع حرف حرف

طريقة التعريف (على سبيل المثال مصفوفة حرفية ثنائية حجمها (3*3))

هيكلية الدالة
`char first_arraychar [3][3] ;`

وطريقة الإدخال أيضا ثابتة فقط العدادات تتغير حسب حجم المصفوفة وهي

البرمجة بلغة c	البرمجة بلغة c++
1.char first_arrychar [3][3]; 2.for (i=0;i<3;i++) 3.for (j=0;j<3;j++) 4.cin>> first_arrychar [i][j] ;	1. char first_arrychar [3][3]; 2.for (i=0;i<3;i++) 3.for (j=0;j<3;j++) 4.scanf("%c",& first_arry[i][j]) ;

** لاحظ خطوة رقم(٤) في الإدخال استخدمنا الرمز **("%c") في لغة (C) لأننا سندخل حرف حرف وليس سلسلة حروف**

وتدخل بالشكل التالي

a	f	c
d	**e**	l
k	**m**	o

وطريقة الطباعة أيضا ثابتة فقط العدادات تتغير حسب حجم المصفوفة وهي

البرمجة بلغة c	البرمجة بلغة c++
1.for (i=0;i<**3**;i++) **2**.for (j=0;j<3;j++) **3.cout<<** first_arrychar [i][j] ;	**1.for (i=0;i<3;i++)** **2.for (j=0;j<3;j++)** **3.printf("%c", first_arry[i][j]) ;**

** يضأ يجب رسم المصفوفة قبل البدء بالحل بأي سؤال

مثال: برنامج ندخل مصفوفة (4*4) ويحسب عدد أحرف العلة في المصفوفة.

تحليل:نفس طريقة الحل السابقة فقط نغير السلسلة إلى مصفوفة ثنائية الحجم

البرمجة بلغة c++	البرمجة بلغة c

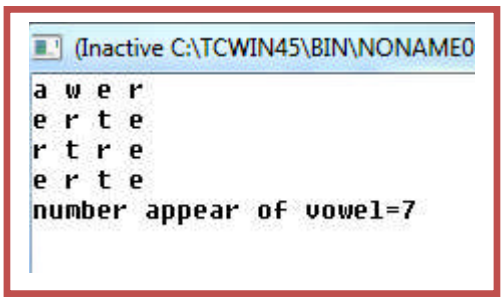
#include<iostream.h>
main()
{1.int i,j,k;
2.int number_appear ;
3. number_appear =0;
4.char vowel [7]={'a','o','u','i','e','n','\o'};
5.char a[4][4];
6.for(i=0;i<4;i++)
7.for(j=0;j<4;j++)
8.cin>>a[i][j];
9.for(k=0;k<6;k++)
10.for(i=0;i<4;i++)
11.for(j=0;j<4;j++)
12.if(a[i][j]== vowel [k])
13.number_appear = number_appear +1;
14.cout<<"number appear of vowel="<< number_appear;}
	#include<stdio.h>
	main()
	{1.int i,j,k;
	2.int number_appear ;
	3. number_appear =0;
	4.char vowel[7]={'a','o','u','i','e','n','\o'};
	5.char a[4][4];
	6.for(i=0;i<4;i++)
	7.for(j=0;j<4;j++)
	8.scanf("%c",& a[i][j]) ;
	9.for(k=0;k<6;k++)
	10.for(i=0;i<4;i++)
	11.for(j=0;j<4;j++)
	12.if(a[i][j]== vowel [k])
	13.number_appear = number_appear +1;
	14.printf("number appear of vowel=%d", number_appear);}

توضيح الخطوات:

١.خطوة رقم (٤) تم تعريف مصفوفة خزنا فيها جميع أحرف العلة

٢.خطوة رقم (٩) هو عداد للمصفوفة التي تحوي أحرف العلة حيث يعمل هذا العداد على مقارنة كل عنصر في مصفوفة (vowel) مع جميع العناصر المخزونة في المصفوفة المدخلة (a) فكلما يجد حرف علة يتحقق الشرط بخطوة رقم(١٢) لكي ينفذ خطوة رقم (١٣) ويزيد قيمة العداد بواحد دالا على إضافة حرف علة

٣.خطوة رقم (١٤) تتم طباعة عدد أحرف العلة كما في الشكل



الفصل الخامس

الدوال (function)

٥.١- الدوال (Function):
٥.٢- دوال (function) بدون قيمة معادة :
٥.٣- الإرسال بالقيمة والإرسال بالمرجع:
٥.٤- الدوال الزائدة
٥.٥- استدعاء الدالة لنفسها (Recursively):
٥.٦- المصفوفات والدوال:

٥.١- الدوال (Function):

تطرقنا في ما مضى على عدد كبير من الدوال الخاصة بلغة (c,c++) وبينا طريقة استخدامها ومنها دالة (pow) التي تستخدم لا يجاد قيمة الرقم المرفوع إلى قوى . وهذه الدوال كلها مصممة من قبل مصممي لغة (c,c++) لكي يسهل العمل على مبرمجين فبدلا أن يكتب المبرمج عدد من الأسطر لإيجاد قيمة رقم المرفوعة إلى قوى

على سبيل المثال إذا أردنا قيمة خمسة مرفوع إلى قوى ثلاثة(5^3) فبدون دالة (pow) نجده هكذا

إيجاد القوى بدون دالة (pow)

```
1.int i,pow1=1;
2.For(i=0 ;i<3 ;i++)
3.pow1=pow1*5;
```

يحلها مباشرة باستخدام الدالة (pow)

إيجاد القوى باستخدام دالة (pow)

```
1.int pow1=pow(5,3)
```

فوجدنا الفرق الطريقة الأولى استخدمنا ثلاث اسطر برمجية وثاني طريقة استخدمنا سطر برمجي واحد وكانت النتيجة نفسها لذالك فائدة الدوال هو تقليل الأسطر البرمجية ومنع تكرار اكواد لأكثر من مرة أي لو احتاجينا على سبيل المثال رفع رقم إلى قوى عشر مرات في برنامجنا في حال دون استخدام الدالة (pow) فسنضطر إلى تكرار الخطوات الثلاث عشر مرات كلما احتجنا إليها التي سوف تزيد من تعقيد البرنامج بينما بواسطة هذه الدالة سوف تقلل الاكواد وتمنع التكرار كلما نحتاجها نستدعيها.إذا هذا الكلام بالنسبة للدوال الجاهزة وأيضا **نستطيع بناء دوال بنفسنا لكي نمنع تكرار اسطر برمجية معينة ونزيد من سهولة تتبع البرنامج كما فعل المبرمجون في بناء دوال**

هيكلية دوال (function)

```
Type  name(parameter1, parameter2,.......)
{
Statement;
Return(type);
}
```

- **(Type)**: هو نوع القيمة التي سوف ترجعها الدالة (function) بعد أن تنتهي من عملها .لان الدوال تكون على نوعين

١.احدها تعيد بعد استدعائها وتكون بشكل التالي عند استدعائها ويجب أن تحتوي على(return) لتعيد قيمة للبرنامج

كود

```
1. reslt =name(var1,var2,.....);
```

وبما أنها تعيد قيمة يجب خزن القيمة المعادة في متغير بنفس نوع القيمة المعادة اي لو كان نوع القيمة المعادة هو (integer) يجب أن يتم تعريف (reslt) على انه متغير من نوع (integer) لتخزن به النتيجة القيمة المرجعة

٢.الثانية لا تعيد أي قيمة للبرنامج تستدعى لتنفذ ما في داخلها وينتهي كل شيء ولا تحتوي في داخلها على (return) لأنها لا تعيد أي قيمة وطريقة استدعائها في البرنامج فقط نكتب اسم الدالة هكذا

كود
```
name(var1,var2,.....);
```

- **(name):** هو اسم الدالة (function) ويمكن أن يكون أي اسم لكن للوضوح اجعل اسم الدالة دالا على عمل دالتك فلو بنيت دالة تجمع رقميين فسمي الدالة (addition) حتى تكون واضحة.
- **(parameter):** هي متغيرات يتم إرسال قيم إلى الدالة لنمثلها داخل الدالة لغرض تنفيذ العرض المطلوب وتقوم الدالة باستقبال المتغيرات المرسلة حسب الترتيب .وكل متغير داخل الدالة يجب تعريفه مثال.

كود
```
Int disp(parameter1, parameter2,.....) { statement};
```

Reslt=disp(var1 , var2 ,.....) استدعاء الدالة من داخل البرنامج

كما تلاحظ في الاستدعاء كان بالتسلسل أسندت قيمة(parameter2= **var2**, parameter1= **var1**)

تسلسل تنفيذ الخطوات في البرنامج

```
# include<iostream.h> //or  #include <stdio.h> for user of C  lanuage
Change_position()
{
6.
7.
8.
عندما انتهى من تنفيذ ماموجود في الدالة واصبح  يعود الى البرنامج الاصلي جاعلا خطوة التنفيذ
التالية هي بعد (٨) وهي (٩) ويستمر البرنامج
}
من هنا يبدء تنفيذ البرنامج خطوة بخطوة (هذه اول خطوة)
Main()
{
( الخطوات الموجودة ضمن هذه الدالة ينفذها تباعا(هذه الارقام هي اسطر برمجية
1.
2.
3.
4.
لاحظ في السطر الخامس احتاج دالة تقع  خارج Main  نقل تنفيذ البرنامج لها اصبح الخطوة(٦)
عندها
5.Change_position()
9.
10
11.
}
```

تسلسل تنفيذ البرنامج يبدأ خطوة خطوة لكن لاحظ عن وصول إلى الخطوة رقم (٥) تم استدعاء دالة لذالك سينقل تنفيذ البرنامج لها لكي ينفذ الخطوات رقم(8—6) التي في داخلها ثم يعود إلى البرنامج حتى ينفذ خطوة رقم (٩و١٠و١١)

- **(Statement)**: هي العملية المراد من الدالة تنفيذها عند استدعائها.

مهم

١. المتغيرات المعرفة داخل الدوال **تنتهي حياتها بانتهاء تنفيذ أخر سطر في الدالة**. أي لو كان متغير(i) في الدالة أصبحت قيمته (i=5) عند انتهاء تنفيذ الدالة فعند استدعاء الدالة مرة أخرى لا تكون قيمته خمسة لان حياته انتهت بانتهاء الاستدعاء السابق وعاد إلى قيمته الأولية قبل التغير.....!

٢. وتكتب الدالة بعد التصريح عن المكتبات مباشرة

٣.الدالة التي تتم كتابتها تعامل داخل البرنامج حالها كحال أي دالة من دوال اللغة

مثال: دالة (function) تقوم بجمع رقميين وتعيد النتيجة إلى البرنامج..؟
تحليل: من السؤال نفهم أن هذه الدالة تحتوي على (parameter) اثنان كل واحد خاص برقم معين وتعيد قيمة من نوع (integer) وهي النتيجة.

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h>	#include<iostream.h>
1.int addi on(int a,int b)	1.int addi on(int a,int b)
{	{
2.int reslt;	2.int reslt;
3.reslt=a+b;	3.reslt=a+b;
4.return(reslt);	4.return(reslt);
}	}
5.main()	5.main()
6.{int reslt;	6.{int reslt;
7.reslt= addi on(3,7);	7.reslt= addi on(3,7);
8.printf("%d\n", reslt);	8.cout<< reslt<<"\n";
9.reslt= addition(2,3);	9.reslt= addition(2,3);
10. printf("%d\n", reslt);	10.cout<< reslt<<"\n";
}	}

توضيح الخطوات:

١.خطوة رقم (١) هي تعريف دالة لجمع متغيرين وتعيد قيمة من نوع (integer)

٢.خطوة رقم (٢) عرفنا متغير جديد وخطوة رقم(٣) جمعنا المتغيرين اللذان تم إرسالهما

مهم

٣.خطوة رقم (٤) تم إعادة هذه القيمة إلى المصفوفة

٤.خطوة رقم (٦) تم تعريف متغير باسم (reslt) لكي يجمل نتيجة الجمع وهناك متغير داخل الدالة(addition) بنفس الاسم لكن لا يؤثر عليه لان المتغيرات داخل الدالة تكون غير معرفة للمتغيرات داخل البرنامج وكذالك المتغيرات داخل البرنامج تكون غير معرفة للمتغيرات داخل الدالة

٥.خطوة رقم(٧) تم استدعاء دالة الجمع وإعطاء قيمة(a=3,b=7) هكذا

كود
```
Int addition(int a,int b)
```

لكي ينفذ خطوة رقم(4—1) جامعاً الرقمين

```
Reslt= addition (3,  7)
```

الذان تم ارسالهما للدالة في خطوة رقم (٣)

٦.خطوة رقم(٨) تم طباعة ناتج الجمع وهو عشرة **(في لغة C استخدما "%d" لا القيمة المعادة هي integer)**

٧. خطوة رقم(٩) تم استدعاء دالة الجمع مرة أخرى وإعطاء قيمة(a=2,b=3) هكذا ويطبع ناتج الجمع في الخطوة رقم(١٠) وناتج الجمع هو خمسة

٥.٢- دوال (function) بدون قيمة معادة :

وهي الدوال التي لا تحتوي على قيمة مرجعة إلى البرنامج.أي تنفذ ما في داخلها ولا تعيد أي قيمة إلى البرنامج **قد تستقبل قيم** لكنها لا تعيد أي قيمة وتعرف هكذا

كود

```
name(parameter1, parameter2,.....) { statement};
```

مثال: دالة (function) تقوم بطباعة رسالة معينة عند استدعائها..؟

تحليل:هذه الدالة تطبع رسالة أي لا ترجع أي قيمة ولا تستقبل أي قيمة

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
1. massageShow()
{
2.prin ("hi Mr.hussien");
}
3.main()
{
4. massageShow()
}
	#include<iostream.h>
	1. massageShow()
	{
	2.cout>>"hi Mr.hussien";;
	}
	3.main()
	{
	4. massageShow()
	}

توضيح الخطوات:

١. خطوة رقم(١) هي دالة لا تستقبل أي متغير ولا تعيد أي متغير لذلك تكتب هكذا
٢. خطوة رقم(٢) هي الرسالة التي سيتم طباعتها عند استدعاء الدالة
٣.خطوة رقم(٤) هي استدعاء لدالة الطباعة لطباعة الرسالة المطلوبة.

مثال : دالة تطبع رقم معين يتم إرساله لها...
تحليل:بما أنها تطبع قيمة ترسل لها أي تستقبل قيمة واحدة ولا تعيد أي قيمة.

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
1. massageShow(int a)
{
2.prin ("the number send is=%d",a);
}
3.main()
{
4. massageShow(3)
}
	#include<iostream.h>
	1. massageShow(int a)
	{
	2.cout<<" the number send is="<<a;
	}
	3.main()
	{
	4. massageShow(3)
	}

توضيح خطوات: خطوة رقم واحد هي دالة تستقبل قيمة واحد لا تعيد أي قيمة وخطوة رقم(٢)تطبع هذه القيمة وخطوة رقم (٤) هو استدعاء لهذه الدالة من داخل البرنامج.

٣.٥- الإرسال بالقيمة والإرسال بالمرجع:

مهم

لإرسال متغيرات إلى دالة يجب أن ترسل بأحد الطريقتين

١. **الإرسال بالقيمة**:ترسل فقط قيمة المتغير إلى الدالة أي إذا تغير قيمة المتغير داخل الدالة لا تتغير قيمته الأصلية داخل البرنامج لأننا أرسلنا فقط قيمته إلى الدالة لمعالجتها .(أي لا تعاد أكثر من قيمة واحدة إلى البرنامج الرئيسي)

مثال: بناء دالة تعمل نفس عمل الدالة (pow) التي تجد قيمة الرقم مرفوع إلى أس (x^n) ولنسميها(powA) معناه هذه الدالة عربية فقط للتميز بينها وبين الأصلية والاثنان يؤديان نفس العمل .

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h>	#include<iostream.h>
1.int powA(int x,int n)	1.int powA(int x,int n)
{	{
2.int i, reslt =1;	2.int i, reslt =1;
3.for(i=0 ;i<n ;i++)	3.for(i=0 ;i<n ;i++)
4. reslt = reslt *x;	4. reslt = reslt *x;
5.return(reslt);	5.return(reslt);
}	}
6.main()	6.main()
7.{int reslt,x,n;	7.{int reslt,x,n;
8.x=3,n=4;	8.x=3,n=4;
9.reslt= powA (x,n);	9.reslt= powA (x,n);
10. printf("powA=%d\n", reslt) ;	10.cout<<" powA="<<reslt<<"\n";
11.x=5,n=2;	11.x=5,n=2;
12.reslt= powA (x,n);	12.reslt= powA (x,n);
13. prin ("powA=%d\n", reslt) ;}	13.cout<<" powA="<<reslt<<"\n";}

توضيح الخطوات:

١.خطو رقم(١ إلى ٥) هي دالة لإيجاد قيمة أي رقم مرفوع إلى أس

٢.خطوة رقم (٨) هو إعطاء قيم للمتغير ولأس المراد إيجاده والمطلوب هنا (x^n) أي انه (3^4)

مهم

٣.خطوة رقم (٩) تم إرسال قيم المتغيرين (x,n) دالة (powA) لإيجاد حل ل(3^4) هكذا

كود

```
Int  powA (int x,int n)
```

3 4

```
Reslt= powA (x ,   n)
```

لو تلاحظ تم إرسال قيم المتغيرين وليس المتغيرين نفسهما وهذا ما يسمى **بالإرسال بالقيمة** (أي أن المتغير(x,n) في الدالة ليس نفس المتغير (x,n) في البرنامج الرئيسي

٤.خطوة رقم(١٠) تم إرسال قيم المتغيرين (x,n) دالة (powA) لإيجاد حل (5^2)

مهم

٢. **الإرسال بالمرجع**:ترسل موقع المتغير إلى الدالة أي أن الدالة تستقبل المتغير نفسه المرسل بنفس الاسم أو بأس ماخر. أي إذا تغير قيمة المتغير داخل الدالة تتغير قيمته الأصلية داخل البرنامج لأننا أرسلنا موقعه إلى الدالة والتغير يكون في محتوى الموقع . وشكلها هكذا

كود

```
Type name(&parameter1, &parameter2,.....) { statement};
```

نضع (&) قبل كل (parameter) نريد أن نعيد التغير في قيمته للبرنامج بمعنى أننا سنستقبل الموقع.
وفائدة الإرسال بالمرجع هو أن الإرسال بلقيمة لا يعيد أكثر من قيمة متغير واحد إلى البرنامج بينما بالإرسال بالمرجع نستطيع أعادة أكثر من قيمة متغير إلى البرنامج

مثال: دالة نرسل لها متغيرين وتقوم بضرب كل واحد منهما بخمسة؟

تحليل: بما إننا نريد ضرب كل واحد منهما بخمسة أي أن الاثنان يتغيران ويعودان إلى الدالة الأصلية بقيم جديدة ونحن نعلم إننا نستطيع إعادة قيمة واحدة في حالة الإرسال بالقيمة لذالك سنستخدم الإرسال بالمرجع

البرمجة بلغة c

```
#include<stdio.h>
1.  mulByFive(int & x,int  &n)
{
2. x=x*5;
3. n=n*5;
}
4.main()
6.{int x,n;
7. int y,z;
8.x=3,n=4;
9. mulByFive  (x,n);
10. prin   ("x=%d\nn=%d\n", x,n) ;
11.y=5,z=2;
12. mulByFive  (x,n);
13. prin   ("y=%d\nz=%d\n", y,z) ;}
```

البرمجة بلغة c++

```
#include<iostream.h>
1.   mulByFive (int & x,int  &n)
{
2. x=x*5;
3. n=n*5;
}
5.main()
 6.{int x,n;
7. int  y,z;
8.x=3,n=4;
9.  mulByFive  (x,n);
10.cout<<" x="<<x<<"\nn="<<n<<"\n";
11.y=5,z=2;
12.  mulByFive (y,z);
13. cout<<" y="<<y<<"\nz="<<z<<"\n";}
```

توضيح الخطوات:

١.خطوة رقم (٩) أرسلنا قيم المتغيرين (x=3,n=4) إلى الدالة وتم استقبالهما بالمرجع وطرب كل واحد منهما بخمسة لاحظ خطوة رقم (١٠) سيطبع قيمهم الجديدة مضروبة بخمسة

كود

```
Int mulByFive(int &x,int &n)
```

x n

```
Reslt= mulByFive (x ,    n)
```

كما نلاحظ من المخطط تم إرسال مواقع المتغيرات

مهم

٢. خطوة رقم (١٢) أرسلنا قيم المتغيرين (y=5,z=5) إلى الدالة وتم استقبالهما بالمرجع وطرب كل واحد منهما بخمسة لاحظ خطوة رقم (١٣) سيطبع قيمهم الجديدة مضروبة بخمسة كال احضنا لا يهتم مهما كان اسم المتغير **لأنه سيستقبل موقعه بأسم أخر ويغير على القيم التي** فيه

كود

```
Int mulByFive(int &x,int &n)
```

y z

```
Reslt= mulByFive (y ,  z)
```

كما نلاحظ من المخطط تم إرسال مواقع المتغيرات وتم التعبير عنهما بالدالة بأسماء جديدة لنفس المواقع

مهم

وباختصار إذا وضعنا (&) قبل أي (parameter). أي تغير في هذا (parameter) سوف يؤثر على قيمته في البرنامج الرئيسي .وإذا لم نضع هذه العلامة يبقى محافظا على قيمته في البرنامج الرئيسي

مهم

إسناد قيم لمتغيرات الدالة (function) : هي قيم يتم إسنادها للمتغيرات في الدالة فإذا لم نذكر هذه المتغيرات في الاستدعاء يتم الاعتماد على هذه القيم وإذا ذكرناها وأعطيناها قيمة يأخذ القيمة التي أعطيناها له.

مثال: دالة تحتوي على قيمة زائدة (دالة لضرب رقمين).؟

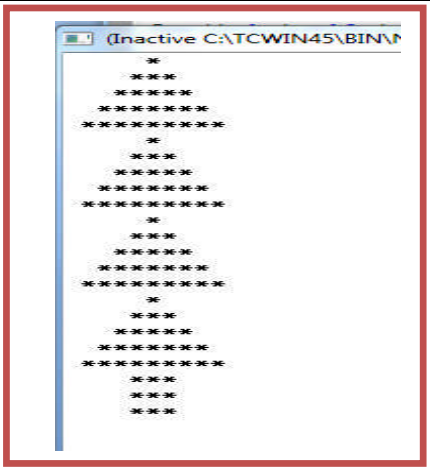
البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h>	#include<iostream.h>
1.int mul (int a,int b=3)	1.int mul (int a,int b=3)
2. {return(a*b);}	2. {return(a*b);}
3.main()	3.main()
4. {printf("mul=%d\n", mul (3));	4. {cout<<"mul="<< mul (3);
5. prin ("mul=%d", mul (3,5));}	5. cout<<"\nmul="<< mul (3,5);}

توضيح الخطوات:

١.خطوة رقم(١) هي دالة تستقبل قيمتين وتكون قيمة (b) هي قيمة زائدة مساوية ل (٣)

٢.خطوة رقم(٤) هو استدعاء للدالة وأعطينا فقط قيمة (a) ولم نذكر المتغير (b) لذالك سيعتمد المتغير (b) على القيمة الزائدة وهي (٣) ويضرب (3*3=9) ويطبع رقم (٩) في شاشة التنفيذ

٣.خطوة رقم (٥) هو استدعاء للدالة وأعطينا قيمة للمتغير (a,b) لذاك سيهمل القيمة الزائدة ويعتمد على قيمة (b=5) ويضرب (3*5=15) ويطبع (١٥) في شاشة التنفيذ



فائدة الدوال في تقليل عدد الاكواد البرمجية وترتيب البرنامج

مثال : برنامج لرسم الشجرة التالية..؟

تحليل:عند تحليلنا لهذه الشجرة نرى أن الجزء المثلث يعاد أربع مرات معناه نفس الكود وبعده كود أخر لقاعدة الشجرة. أي أننا لدينا كودان احدهما يعاد أربع مرات وآخر مرة واحدة ..!

الحل بدون دوال (function).....!

البرمجة بلغة c

```
#include<stdio.h>
main()
{int i,j,k;
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
printf(" " );
for(j=i;j>0;j--)
printf("*") ;
printf("\n") ;
}
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
printf(" " );
for(j=i;j>0;j--)
printf("*") ;
printf("\n") ;
}
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
printf(" " );
for(j=i;j>0;j--)
printf("*") ;
printf("\n") ;
}
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
printf(" " );
for(j=i;j>0;j--)
printf("*") ;
printf("\n") ;
}
for(i=1;i<=6;i+=2){
for(k=4;k>0;k-=1)
printf(" " );
for(j=3;j>0;j--)
printf("*") ;
 printf("\n") ;}}
```

البرمجة بلغة c++

```
#include<iostream.h>
main()
{int i,j,k;
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
cout<<" " ;
for(j=i;j>0;j--)
cout<<"*" ;
cout<<"\n" ;
}
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
cout<<" " ;
for(j=i;j>0;j--)
cout<<"*" ;
cout<<"\n" ;
}
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
cout<<" " ;
for(j=i;j>0;j--)
cout<<"*" ;
cout<<"\n" ;
}
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
cout<<" " ;
for(j=i;j>0;j--)
cout<<"*" ;
cout<<"\n" ;
}
for(i=1;i<=6;i+=2){
for(k=4;k>0;k-=1)
cout<<" " ;
for(j=3;j>0;j--)
cout<<"*" ;
cout<<"\n" ;}}
```

الحل باستخدام دوال (function).....!

تحليل:ولو كتبا الكود المعاد أربع مرات في دالة واستدعيناها أربع مرات وبعدة نكتب كود قاعدة الشجرة لتبسط البرنامج كثيرا وأصبح واضح وسهل.

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
Draw_tree()
{ int i,j,k;
for(i=1;i<=10;i+=2){
for(k=i;k<10;k+=2)
printf(" ") ;
for(j=i;j>0;j--)
printf("*") ;
printf("\n");}}
main()
{ int i,j,k;
Draw_tree();
Draw_tree();
Draw_tree();
Draw_tree();
for(i=1;i<=6;i+=2){
for(k=4;k>0;k-=1)
printf(" ") ;
for(j=3;j>0;j--)
printf("*") ;
printf("\n");}}
	#include<iostream.h>
	Draw_tree()
	{ int i,j,k;
	for(i=1;i<=10;i+=2){
	for(k=i;k<10;k+=2)
	cout<<" " ;
	for(j=i;j>0;j--)
	cout<<"*" ;
	cout<<"\n" ;}}
	main()
	{ int i,j,k;
	Draw_tree();
	Draw_tree();
	Draw_tree();
	Draw_tree();
	for(i=1;i<=6;i+=2){
	for(k=4;k>0;k-=1)
	cout<<" " ;
	for(j=3;j>0;j--)
	cout<<"*" ;
	cout<<"\n" ;}}

لو تلاحظ كم تبسط الكود وكم أصبح البرنامج واضح عندما وضعنا الجزء المكرر أربع مرات في دالة واستدعيناه أربع مرات فكون شجرة وبقية قاعدة الشجرة كتبنا كودها وحده.

مهم

المتغيْرات التي تعرف تحت تعريف المكتبات تسمى **متغيرات عامة** تكون معرفة بالنسبة إلى جميع أجزاء البرنامج أي على سبيل المثال لو عرفنا متغير اسمه (item) يكون هذا المتغير معرف بالنسبة إلى جميع الدوال أو إلى البرنامج الرئيسي على خلاف **المتغيرات الخاصة** التي تعرف داخل الدوال تكون فقط معرفة نسبة إلى الدالة

توضيح

```
#Include<        >
 Global variable  ←————————— متغيرات عامة معرفة إلى جميع أجزاء البرنامج

Main()
{
 Local variable ←————————— متغيرات خاصة تكون معرفة فقط داخل الدالة ( main)
}
```

وهذه المتغيرات لا تنتهي حياتها إلا بانتهاء البرنامج وتبقى محافظة على قيمها الجديدة ولا تعود لقيمتها البدائية.

نستطيع تعريف واستخدام أكثر من دالة في برنامج واحد..

مهم

٤.٥- الدوال الزائدة :

هي مجموعة دوال لها نفس الاسم وتختلف في القيمة المعادة أو تختلف في نوع (parameter) المستقبل للدالة. فعند استدعاء احد هذه الدوال **وبما أنها جميعا بنفس الاسم لذالك سوف يستدعي المترجم الدالة التي تستقبل اقرب نوع للمتغير الذي أرسلته لها أو نفس النوع.**

مثال: برنامج يحوي على دوال الآتية احدها تجمع الرقمين والأخرى تقسمهم والأخرى تضربهم ولهم نفس الاسم؟

البرمجة بلغة c

```
#include<stdio.h>
1.int opera on (int a, int b)
2. {return(a*b);}
3.float opera on (float a, float b)
4. {return(a/b);}
5. float opera on (float a , int b)
6. {return(a+b);}
7.main()
8. {int a=3,b=2;
9.float c=2.5,d=4.5;
10. printf("reslt=%d\n", operation (a,b));
11. printf("reslt=%f\n", operation (c,d) );
12. printf("reslt=%f\n", operation (c,a) );
}
```

البرمجة بلغة c++

```
#include<iostream.h>
1.int opera on (int a, int b)
2. {return(a*b);}
3. float operation (float a , int b)
4. {return(a+b);}
5.float operation (float a, float b)
6. {return(a/b);}
7.main()
8. {int a=3,b=2;
9.float c=2.5,d=4.5;
10.cout<<"reslt="<< opera on (a,b);
11. cout<<"\n reslt="<< operation (c,d);
12. cout<<"\n reslt="<< operation (c,a);
}
```

توضيح الخطوات:

١.خطوة رقم (١٠) تم إرسال متغيرين (a,b)من نوع integer لذالك ستستقبله الدالة في الخطوة رقم (١) وتضرب الرقميين لان هذه الدالة تستقبل المتغيرين من نوع integer

كود

```
1.int opera on (int a, int b)
```

3 2

```
operation ( a , b)
```

٢.خطوة رقم (١١) تم إرسال متغيرين (c,d)من نوع float لذالك ستستقبله الدالة في الخطوة رقم (5) وتقسم الرقميين لان هذه الدالة تستقبل المتغيرين من نوع float

كود

```
5.float operation (float a, float b)
```

2.5 4.5

```
operation ( x , n)
```

٣.خطوة رقم (١٢) تم إرسال متغيرين (a)من نوع integer , (c) من نوع float لذالك ستستقبله الدالة في الخطوة رقم (3) وتجمع الرقميين لان هذه الدالة تستقبل متغير من نوع integer ومتغير من نوع float

مهم

٥.٥- استدعاء الدالة لنفسها (Recursively):

وتسمى أيضا (Recursive Function) أي استدعاء الدالة لنفسها لمرة أو أكثر من مرة.فتكون شكلها معروف أنها من داخل دالة (function) نقوم باستدعائها لنفسها(أي يذكر اسمها في داخلها) ويكون شكلها هكذا

هيكلية دوال (Recursive function)

```
Type  name(parameter1, parameter2,.....)
{
Statement;
Return( name(parameter1, parameter2,.....) );
}
```

مثال: بناء الدالة التالية (X^n) أي دالة (pow) باستخدام أسلوب (Recursive Function)

تحليل:عندما يذكر (Recursive Function) يجب أن ترك أي فكرة في ذهننا لحل السؤال بدون استدعاء الدالة لنفسها.وكما نرى أن رفع الرقم لأس معي معناه ضرب الرقم بنفسه بمقدار قيمة الأس مثال على ذالك

كود

```
4^3=4*4*4
```

أذن سنكون دالة تعيد استدعاء لنفسها بمقدار قيمة الأس المرفوع له الرقم وبكل استدعاء تضرب الرقم في نفسه

البرمجة بلغة c

```
#include<stdio.h>
1.int power(int x,int n )
{
2.if (n>0)
3.return(x*power(x,n-1 ));
4.else
5.return 1;
}
6.main() {
7.printf("%d",power(4,3));}
```

البرمجة بلغة c++

```
#include<iostream.h>
1.int power(int x,int n )
{
2.if (n>0)
3.return(x*power(x,n-1 ));
4.else
5.return 1;
}
6.main() {
7.cout<<power(4,3);}
```

توضيح الخطوات:

١.خطوة رقم (٧) أردنا أيجاد قيمة (4^3) لذالك سيستدعي الدالة في الخطوة رقم (١) جاعلا قيمة (x=4,n=3) ثم ينفذ خطوة رقم (٢) لان(n>0) وينفذ بعده خطوة رقم (٣) لكي يعيد الرقم مضروب باستدعاء الدالة لنفسها مرسلة لها الرقم والأس منقص منه واحد ويستمر بالاستدعاء الذاتي إلى أن تصبح قيمة الأس (n=0) يعيد عندها واحد ويخرج من الاستدعاء الذاتي .

توضيح ماذا سيحصل في خطوات رقم(١ إلى ٥) في حال كان (x=4,n=3)

توضيح ماذا سيحصل في خطوات رقم(١ إلى ٥) في حال كان (x=4,n=3) (فقط الخطوات التي سوف تنفذ بكل استدعاء)
الاستدعاء الأول
1.power(x=4,n=3)
2. n=3 is large than zero
3. return(4*power(4,3-1))
الاستدعاء الثاني
1.power(x=4,n=2)
2. n=2 is large than zero
3. return(4*4*power(4,2-1))
الاستدعاء الثالث
1.power(x=4,n=1)
2. n=1 is large than zero
3. return(4*4*4*power(4,1-1))
الاستدعاء الرابع
1.power(x=4,n=0)
4. n=0 is equal to zero
5. return(4*4*4*1)

استدعت الدالة لنفسها أربع مرات ؟ والنتيجة هي(4*4*4*1=64) أذن النتيجة صحيحة..،كتبنا فقط الخطوات التي سوف تنفذ بكل استدعاء للدالة والتي لم تنفذ لم نكتبها

مثال: أيجاد مفكوك الرقم باستخدام أسلوب (Recursive Function)..؟

تحليل:عندما يذكر (Recursive Function) يجب أن ترك أي فكرة في ذهننا لحل السؤال بدون استدعاء الدالة لنفسها.وكما نرى أن المفكوك هو ناتج من حاصل ضرب الرقم بالأرقام التي اقل منه وصولا إلى الواحد

كود
5!=5*4*3*2*1

أذن سنكون دالة تعيد استدعاء لنفسها بمقدار قيمة الرقم مثلا مفكوك خمسة ستستدعي الدالة نفسها خمس مرات وفي كل مرة تستدعي الدالة لنفسها نطرح من الرقم المرسل واحد ونضربه ببقية الأرقام هكذا

كود
N!=N*(n-1)!

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h>	#include<iostream.h>
1.int fact (int x)	1.int fact (int x)
{	{
2.if (x>1)	2.if (x>1)
3.return(x* fact (x-1));	3.return(x* fact (x-1));
4.else	4.else
5.return 1;	5.return 1;
}	}
6.main() {	6.main() {
7.prin ("%d",fact(4));}	7.cout<<fact(4);}

توضيح الخطوات:

- خطوة رقم (٧) استدعينا دالة المفكوك وأردنا إيجاد مفكوك الرقم أربعة فعند الدخول للدالة نرى وجود خطوة رقم(٣) هذه الخطوة تضرب الرقم بمفكوك الأرقام التي اقل منه هكذا(n*(n-1)!) وتستمر بالضرب إلى أن يصل إلى الرقم صفر

توضيح ماذا سيحصل في خطوات رقم(١ إلى ٥) في حال كان (x=4,n=3)

توضيح ماذا سيحصل في خطوات رقم(١ إلى ٥) في حال كان (x=4) (فقط الخطوات التي سوف تنفذ بكل استدعاء)
الاستدعاء الأول
1.fact(x=4)
2. x=4 is large than one
3. return(4* fact (4-1))
الاستدعاء الثاني
1. fact (x=3)
2. x=3 is large than one
3. return(4*3*power(3-1))
الاستدعاء الثالث
1. fact (x=2)
2. x=2 is large than one
3. return(4*3*2*power(2-1))
الاستدعاء الرابع
1. fact (x=1)
4. x=1 is equal to one
5. return(4*3*2*1)

استدعت الدالة لنفسها أربع مرات ؟ والنتيجة هي(4*3*2*1=24) أذن النتيجة صحيحة..،كتبنا فقط الخطوات التي سوف تنفذ بكل استدعاء للدالة والتي لم تنفذ لم نكتبها

مهم

أذا كانت الدالة مكونة من سطر برمجي واحد تسمى دوال سطريه (inline function) وتكتب هكذا

هيكلية دوال (inline function)
inlineType name(parameter1, parameter2,.....)
{Statement;}

مهم

٦.٥- المصفوفات والدوال :

يمكن إرسال مصفوفات أحادية الإبعاد إلى الدوال بذكر اسمها فقط بدون أبعاد .

مثال:لو كان لدينا مصفوفة اسمها (a) ونريد إرسالها إلى دالة اسمها (name)

إرسال مصفوفة أحادية إلى الدوال (function)

```
name( a );
```

وطريقة استقبال المصفوفة الأحادية في الدوال نضع فقط أقواس المصفوفة بدون إبعاد

إرسال مصفوفة أحادية إلى الدوال (function)

```
Type  name(type a[] );
```

- **(type)** : هو نوع المصفوفة المرسلة

يمكن إرسال مصفوفات ثنائية الأبعاد إلى الدوال فقط بذكر اسمها بدون أبعاد .

مثال:لو كان لدينا مصفوفة اسمها (a) وإبعادها (2*3) ونريد إرسالها إلى دالة اسمها (name)

إرسال مصفوفة ثنائية إلى الدوال (function)

```
name( a );
```

وطريقة استقبال المصفوفة ثنائية الأبعاد في الدوال نذكر فقط البعد الثاني هكذا

إرسال مصفوفة ثنائية إلى الدوال (function)

```
Type  name(type a[][3] );
```

- المصفوفات عند إرسالها بهذه الطرق إلى الدوال **ترسل بالقيمة وليس بالمرجع**.إذا إي تغير على عناصر المصفوفة في الدالة سوف لا يؤثر على قيم العناصر في البرنامج الرئيسي.أي ترسل نسخة من المصفوفة وليس المصفوفة نفسها.

مثال: خمس مصفوفات أحادية الإبعاد حجمها (7) جد جمع كل مصفوفة واكبر عدد بكل مصفوفة..؟

تحليل: كما نرى انه يريد ناتج جمع كل مصفوفة واكبر عدد فيكون البرنامج كبير جدا إذا لم نستعمل الدوال (function) لأنه لدل مصفوفة يجب كتابة كود يجد اكبر رقم ويجمع عناصر المصفوفة لكن مع الدوال (function) نكون دالة تجد مجموع عناصر المصفوفة ودالة تجد اكبر رقم ونمرر كل مصفوفة لهاتان الدالتان

البرمجة بلغة c

```
#include<stdio.h>
1.int  maxN(int  array1[])
2.{ int i, max;
 3.max=array1[0];
4.for (i=0;i<7;i++)
5.if (array1[i] > max )
6.max=array1[i];
7.return(max);}
8.int  sumN(int  array1[])
9.{ int i, sum=0;
 10.for (i=0;i<7;i++)
 11.sum=sum+ array1[i];
12.return(sum);}
13.main()
14.{ int i, a[7],b[7],c[7],d[7],e[7];
15.printf( "enter element (1)  array=");
16.for (i=0;i<7;i++)
17.scanf("%d",&a[i]);
18.printf("max=%d\tsum=%d\n",maxN(a),sumN(a) );
19. printf( "enter element (2)  array=");
20.for (i=0;i<7;i++)
21. scanf("%d",&b[i]);
22. printf("max=%d\tsum=%d\n",maxN(b),sumN(b) );
23. printf( "enter element (3)  array=");
24.for (i=0;i<7;i++)
25. scanf("%d",&c[i]);
26. printf("max=%d\tsum=%d\n",maxN(c),sumN(c) );
27. printf( "enter element (4)  array=");
28.for (i=0;i<7;i++)
29. scanf("%d",&d[i]);
30. printf("max=%d\tsum=%d\n",maxN(d),sumN(d) );
31. printf(  "enter element (5)  array=");
32.for (i=0;i<7;i++)
33. scanf("%d",&e[i]);
34. printf("max=%d\tsum=%d\n",maxN(e),sumN(e) );}
```

البرمجة بلغة c++

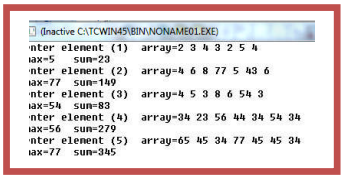
```
#include<iostream.h>
1.int  maxN(int  array1[])
2.{ int i, max;
 3.max=array1[0];
4.for (i=0;i<7;i++)
5.if (array1[i] > max )
6.max=array1[i];
7.return(max);}
8.int  sumN(int  array1[])
9.{ int i, sum=0;
 10.for (i=0;i<7;i++)
 11.sum=sum+ array1[i];
12.return(sum);}
13.main()
14.{ int i, a[7],b[7],c[7],d[7],e[7];
15.cout<< "enter element (1)  array=";
16.for (i=0;i<7;i++)
17.cin>>a[i];
18.cout<<"max="<<maxN(a)<<"\tsum="<<sumN(a)<<"\n";
19.cout<< "enter element (2)  array=";
20.for (i=0;i<7;i++)
21.cin>>b[i];
22.cout<<"max="<<maxN(b)<<"\tsum="<<sumN(b)<<"\n";
23.cout<< "enter element (3)  array=";
24.for (i=0;i<7;i++)
25.cin>>c[i];
26.cout<<"max="<<maxN(c)<<"\tsum="<<sumN(c)<<"\n";
27.cout<< "enter element (4)  array=";
28.for (i=0;i<7;i++)
29.cin>>d[i];
30.cout<<"max="<<maxN(d)<<"\tsum="<<sumN(d)<<"\n";
31.cout<< "enter element (5)  array=";
32.for (i=0;i<7;i++)
33.cin>>e[i];
34.cout<<"max="<<maxN(e)<<"\tsum="<<sumN(e)<<"\n";}
```

توضيح الخطوات:

١.خطوة رقم (١) هي دالة تستقبل المصفوفة وتقوم بإيجاد اكبر رقم لو تلاحظ الخطوات (٢ إلى ٧) هي خطوات إيجاد اكبر رقم نفسها التي شرحناها في المصفوفة الأحادية وكيفية أيجاد اكبر رقم فقط وضعناها في دالة

٢.خطوة رقم(٨) هي دالة لجمع عناصر المصفوفة

٣.خطوة رقم(١٦ و١٧) هي إدخال المصفوفة (a) وخطوة رقم (١٨) هي استدعاء دالة اكبر رقم ودالة جمع عناصر المصفوفة وطباعة الرقم وبقية الخطوات تتكرر نفس العملية بالنسبة لبقية المصفوفات



** ألان أصبح لدينا دالة متى استدعيناها تجد اكبر رقم.

مثال: برنامج لحساب الحرف الأكثر تكرار وعدد مرات تكراره. في خمس سلاسل نصية كل على حدة .؟

تحليل:لإيجاد الحرف الأكثر تكرار ضمن إي سلسلة يجب خزن جميع الأحرف في مصفوفة وحساب عدد مرات ظهور كل حرف في السلسلة وطباعة الحرف الذي يظهر أكثر من غيره.

البرمجة بلغة c

```
#include<stdio.h>
#include<string.h>
1.int i,m, max,fou;
2.char a[45], charSaved[255];
 3.repeat(char string[],int len)
4.{ int  equavelentnumber[255]={0};
 5. for(i=0;i<255;i++)
6.charSaved[i]=char(i);
7.for(m=0;m<255;m++){
8.for(i=0;i<len-1;i++)
9.if (( charSaved [m]== string [i]) &&   ( string [i] !=' '))
10.equavelentnumber [m]+=  1 ;}
11.max= equavelentnumber [0];
12.for(i=0;i<255;i++)
13.if ( equavelentnumber [i]>max){
14.max= equavelentnumber [i];
15.fou =i;}
16.printf("charcter more repeat=%c\n", charSaved [ fou ]);
17.printf("it repeat=%d\n", max );   }
18.main()
19.{int count1;
20.for ( count1=1; count1<6; count1++){
21.prin   ("enter the S   ng(%d):  ", count1);
22.gets(a );
23.repeat(a,strlen(a));}}
```

البرمجة بلغة c++

```
#include<iostream.h>
#include<string.h>
1.int i,m, max,fou;
2.char a[45], charSaved[255];
 3.repeat(char string[],int len)
4.{ int  equavelentnumber[255]={0};
 5. for(i=0;i<255;i++)
6.charSaved[i]=char(i);
7.for(m=0;m<255;m++){
8.for(i=0;i<len-1;i++)
9.if (( charSaved [m]== string [i]) &&   ( string [i] !=' '))
10.equavelentnumber [m]+=  1 ;}
11.max= equavelentnumber [0];
12.for(i=0;i<255;i++)
13.if ( equavelentnumber [i]>max){
14.max= equavelentnumber [i];
15.fou =i;}
16.cout<<"charcter more repeat "<< charSaved [ fou ]<<" \n";
17.cout<<"it repeat="<<max <<"\n";   }
18.main()
19.{int count1;
20.for ( count1=1; count1<6; count1++){
21.cout<<"enter the Sting( "<<count1<<"): ";
22.cin.getline(a );
23.repeat(a,strlen(a));}}
```

توضيح الخطوات:

١.خطوة رقم(١) عرفنا متغيرات عامة معرفة لجميع أجزاء البرنامج
٢.خطوة رقم (٢) عرفنا مصفوفة (charSaved) لنخزن فيها جميع الأحرف حتى نحسب عدد مرات ظهور كل حرف
٣.خطوة رقم (٣)هي دالة تستقبل السلسلة وطولها وتطبع الحرف الأكثر تكرار وعدد مرات تكراره
٤.خطوة رقم(٤) عرفنا مصفوفة (equavelentnumber) لنخزن فيها عدد مرات ظهور كل حرف
٥.خطوة رقم (٥ و٦) نخزن جميع الأحرف والرموز في مصفوفة اسمها (charSaved)
٦.خطوة رقم (٧ و٨و٩و١٠) نحسب عدد مرات ظهور كل حرف في السلسلة التي ادخلها المستخدم ونخزن في مصفوفة (equavelentnumber) وتكون هذه المصفوفة مكافئ لمواقع كل رمز أو حرف في مصفوفة (charSaved)
٧.خطوة رقم(١١ إلى ١٧) نحسب الحرف الذي ظهر أكثر مرة ونطبع و نطبع عدد مرات ظهوره
٨.خطوة رقم (٢٠ إلى ٢٣) هو ادخل المصفوفات وإرسالها إلى الدالة

**لا يجوز تعريف متغير (متغير عام) واستخدامه كعداد في البرنامج الرئيسي وهو مستخدم داخل احد الدوال كعداد أو تتغير قيمته سوف لا ينفذ البرنامج لأنه سوف يجعل عبارة التكرار في غموض

```
ive C:\TCWIN45\BIN\NONAME01.EXE)
the Sting(1):  hussien ahmmed taleb
er more repeat=e
eat=3
the Sting(2):  he go to home
er more repeat=o
eat=3
```

مثال: برنامج لترتيب ثلاث مصفوفات ثنائية الإبعاد (5*5) تصاعديا.؟

تحليل:لترتيب ثلاث مصفوفات ثنائية نستخدم نفس طريقة ترتيب المصفوفات الثنائية ونضعها في دالة(function) ونستدعيها ثلاث مرات.

البرمجة بلغة c

```
#include<stdio.h>
1.int const row=5;
 2.int const col=5;
4.int i,j,k,x,l ;
3.Sort2D(int array[][col])
{
4.for( k=0;k<row;k++){
5. for( l=0;l<col;l++){
6. for( i=0;i<row;i++){
 7. for ( j=0;j<col;j++){
 8. if (array[i][j] > array[k][l]){
9.x=array[k][l];
 10.array[k][l]=array[i][j];
 11.array[i][j]=x;
 12.}} }} }
13. printf("Here is the Array after sorted\n") ;
14. for ( i=0;i<row;i++){
 15. for ( j=0;j<row;j++)
16. printf("%d\t",array[i][j] );
17.  printf("\n" );}}
18. main()
 19.{  int array1[row][col];
20. int array2[row][col];
21. int array3[row][col];
22. printf("Here is the Array(1)  befor sorted\n") ;
23. for ( i=0;i<row;i++)
 24. for ( j=0;j<col;j++)
25.scanf("%d",&array1[i][j] );
26.Sort2D(array1);
27. printf("Here is the Array (2) befor sorted\n") ;
28. for ( i=0;i<row;i++)
 29. for ( j=0;j<col;j++)
30. scanf("%d",&array2[i][j] );
31.Sort2D(array2);
32. printf("Here is the Array (3)  befor sorted\n" );
33. for ( i=0;i<row;i++)
 34. for ( j=0;j<col;j++)
35. scanf("%d",&array3[i][j] );
36.Sort2D(array3);}
```

البرمجة بلغة c++

```
#include<iostream.h>
1.int const row=5;
 2.int const col=5;
4.int i,j,k,x,l ;
3.Sort2D(int array[][col])
{
4.for( k=0;k<row;k++){
5. for( l=0;l<col;l++){
6. for( i=0;i<row;i++){
 7. for ( j=0;j<col;j++){
 8. if (array[i][j] >array[k][l]){
9.x=array[k][l];
 10.array[k][l]=array[i][j];
 11.array[i][j]=x;
 12.}} }} }
13. cout<<"Here is the Array after sorted\n" ;
14. for ( i=0;i<row;i++){
 15. for ( j=0;j<row;j++)
16. cout<<array[i][j]<<"\t";
17. cout<<"\n" ;}}
18. main()
 19.{  int array1[row][col];
20. int array2[row][col];
21. int array3[row][col];
22.cout<<"Here is the Array(1)  befor sorted\n" ;
23. for ( i=0;i<row;i++)
 24. for ( j=0;j<col;j++)
25.cin>>array1[i][j] ;
26.Sort2D(array1);
27.cout<<"Here is the Array (2) befor sorted\n" ;
28. for ( i=0;i<row;i++)
 29. for ( j=0;j<col;j++)
30.cin>>array1[i][j] ;
31.Sort2D(array2);
32.cout<<"Here is the Array (3)  befor sorted\n" ;
33. for ( i=0;i<row;i++)
 34. for ( j=0;j<col;j++)
35.cin>>array3[i][j] ;
36.Sort2D(array3);}
```

توضيح الخطوات: خطوة رقم(٣) هي دالة لترتيب عناصر مصفوفة تصاعديا
ونلاحظ خطوة رقم(٢٢ إلى ٢٥) هي إدخال للمصفوفة الأولى وخطوة رقم(٢٦) هي إرسالها لترتيبها
ولخطوات البقية هي إدخال بقية المصفوفات وترتيبها.

مثال: برنامج لإيجاد أحرف العلة وعددها في ثلاث مصفوفات ثنائية الإبعاد (4*4) كل على حدة .؟

تحليل: إيجاد أحرف العلة في مصفوفات ثنائية لأكثر من واحدة نحتاج إلى دوال (function) .

البرمجة بلغة c

```
#include<stdio.h>
1.char  vowelchar [7]={'a','o','u','i','e','n','\o'};
2..int i,j,k;
3.int indexofprintarray=1;
4.vowel( char a[][4])
5.{ printf("\nvowel Char in array(%d)\n
",indexofprintarray) ;
6.indexofprintarray=indexofprintarray+1;
7.int number_appear  ;
8. number_appear =0;
9.for(k=0;k<6;k++)
10.for(i=0;i<4;i++)
11.for(j=0;j<4;j++)
12.if(a[i][j]== vowelchar [k])
13.{printf( "\n%d",a[i][j]);
14.number_appear = number_appear +1;}
15.printf("\nnumber appear of vowel=%d",
number_appear);}}
 16.main()
17.{char a[4][4],b[4][4], c[4][4];
18.printf("\nenter (1) array:\n");
19..for(i=0;i<4;i++)
20.for(j=0;j<4;j++)
21.scanf("%d",&a[i][j]);
22. printf("\nenter (2) array:\n");
23.for(i=0;i<4;i++)
24.for(j=0;j<4;j++)
25. scanf("%d",&b[i][j]);
26. printf("\nenter (3) array:\n");
27.for(i=0;i<4;i++)
28.for(j=0;j<4;j++)
29. scanf("%d",&c[i][j]);
30.vowel(  a );
31.vowel( b);
32.vowel( c);}
```

البرمجة بلغة c++

```
#include<iostream.h>
1.char  vowelchar [7]={'a','o','u','i','e','n','\o'};
2..int i,j,k;
3.int indexofprintarray=1;
4.vowel( char a[][4])
5.{ cout<<"\nvowel Char in
array("<<indexofprintarray<<")\n";
6.indexofprintarray=indexofprintarray+1;
7.int number_appear  ;
8. number_appear =0;
9.for(k=0;k<6;k++)
10.for(i=0;i<4;i++)
11.for(j=0;j<4;j++)
12.if(a[i][j]== vowelchar [k])
13.{cout<< "\n"<<a[i][j];
14.number_appear = number_appear +1;}
15.cout<<"\nnumber appear of vowel="<<
number_appear;}}
 16.main()
17.{char a[4][4],b[4][4], c[4][4];
18.cout<<"\nenter (1) array:\n";
19..for(i=0;i<4;i++)
20.for(j=0;j<4;j++)
21.cin>>a[i][j];
22.cout<<"\nenter (2) array:\n";
23.for(i=0;i<4;i++)
24.for(j=0;j<4;j++)
25.cin>>b[i][j];
26.cout<<"\nenter (3) array:\n";
27.for(i=0;i<4;i++)
28.for(j=0;j<4;j++)
29.cin>>c[i][j];
30.vowel(  a );
31.vowel( b);
32.vowel( c);}
```

توضيح الخطوات: خطوة رقم (٣) عرفنا متغير (indexofprintarray) كمتغير عام ونلاحظ انه كلما نستدعي الدالة (vowel) في الخطوة رقم (٤) ستزداد قيمته بواحد في خطوة رقم (٦) لأنه كما قلنا **المتغير العام لا يموت يبقى حيا حتى نهاية البرنامج** ولا يرجع إلى قيمه الابتدائية (أي بعد ثلاث استدعائات لخطوة رقم(٣٠و٣١و٣٢) تصبح قيمته أربعة) ونحن احتاجيناه حتى في كل استدعاء يطبع رقم المصفوفة الجاري البحث فيها في خطوة رقم(٥)

وبقية الخطوات واضحة ومشروحة سابقا

الفصل السادس

المؤشرات (pointer)

٦.١- المؤشرات:

٦.٢- المؤشرات والمتغيرات:

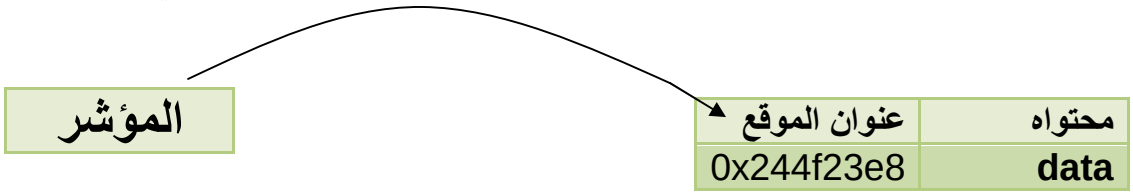
٦.٣- الدالة (NEW) :

٦.٤- المؤشرات والمصفوفات:

٦.٥- المؤشرات والدوال (function)

٦.١- المؤشرات:

هو إشارة أو تأشير على موقع في الذاكرة. عرفنا سابقا إن كل متغير أو كل متغير يخزن في الذاكرة بموقع ذو عنوان معين والمؤشر سوف يؤشر على هذا العنوان ويمكنا من الوصول إلى القيمة المخزنة في داخله.



المؤشر يؤشر على عنوان المتغير في الذاكرة ويحمل قيمة هذا العنوان وهو هنا (0x244f23e8)

٦.٢- المؤشرات والمتغيرات:

المتغيرات تخزن في الذاكرة ويمكن أن نؤشر على قيمها بواسطة (pointer) ونتلاعب بقيمة هذا الموقع بواسطته وتكون طريقة تعريف (pointer) مع المتغيرات هكذا.

هيكلية المؤشر

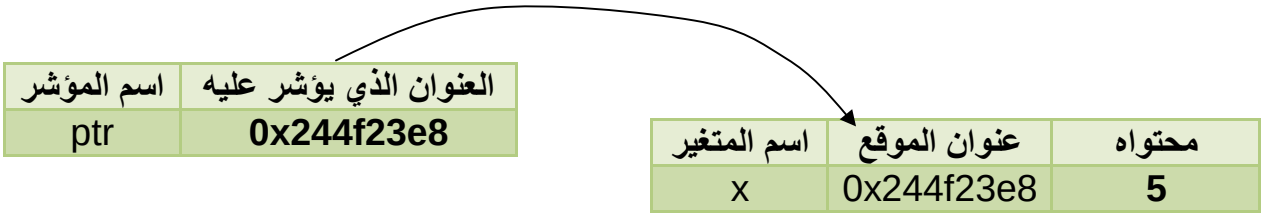
```
Type *ptr=&var
```

- **(ptr)** : هو اسم المؤشر وقد يكون أي اسم يعرفه المستخدم ويجب وضع علامة النجمة (*) قبله
- **(Type)**: هو نوع المؤشر ويكون نوعه نفس نوع المتغير الذي يؤشر عليه
- **(var)**: هو اسم المتغير الذي نريد أن يؤشر عليه المؤشر ويجب وضع علامة (&) قبله عندما نجعل المؤشر يؤشر عليه لان وضع هذه العلامة قبل أي متغير معناه المطلوب عنوان المتغير وليس قيمة المتغير نفسه والمؤشرات تؤشر على العناوين المتغيرات فلو وضعنا هذه العلامة أمام أي متغير في الطباعة سيطبع موقع المتغير وليس قيمته.

مثال: لو كان لدينا متغير (x=5) ويؤشر عليه مؤشر (ptr) بالشكل التالي فيكون المخطط هكذا

كود

```
int x=5;
int *ptr=&x;
```



- الموقع الذي يؤشر عليه (ptr) هو (0x244f23e8) ومحتواه هو (٥)
- للوصول على عنوان الموقع نكتب **(ptr)** أو نكتب (&x) لان الاثنان أصبح لهما نفس الموقع.
- للوصول على محتويات الموقع نكتب **(*ptr)** أو نكتب (x) لان الاثنان أصبح لهما نفس القيمة.

مثال: تأشير على موقع متغير وطباعة قيمته.

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
1.{int x=5;
2.int *ptr=&x;
3.prin ("location Ptr=%d",ptr);
4.printf("\nlocation var=%d ",&x);
5.prin ("\nvalue Ptr=%d ",*ptr);
6.prin ("\n value var=%d",x);
}
	#include<iostream.h>
	main()
	1.{int x=5;
	2.int *ptr=&x;
	3.cout<<"loca on Ptr="<<ptr;
	4.cout<<"\nlocation var="<<&x;
	5.cout<<"\nvalue Ptr="<<*ptr;
	6.cout<<"\n value var="<<x;
	}

توضيح الخطوات:

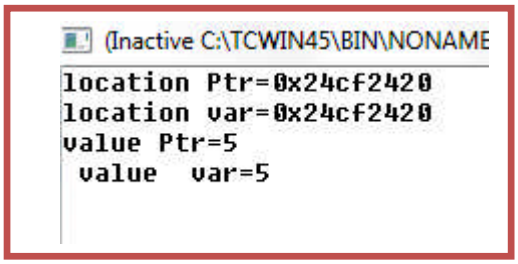
١.خطوة رقم (٢) جعلنا المؤشر(ptr) يؤشر على موقع المتغير (x)

٢.خطوة رقم (٣) طبعنا عنوان الموقع الذي يؤشر عليه المؤشر (ptr) وهو نفس عنوان موقع المتغير (x) لأنهما يؤشران على نفس الموقع.أي أننا إذا كتبنا فقط اسم المؤشر سيطبع الموقع الذي يؤشر عليه

٣.خطوة رقم (٤) طبعنا عنوان المتغير (x) لأننا إذا وضعنا علامة (&) قبل أي متغير سيطبع موقعه

٤.خطوة رقم (٥) طبعنا القيمة التي يؤشر عليها المؤشر(ptr) وهي نفس قيمة المتغير (x) لأنهما يؤشران على نفس الموقع. أي أننا إذا كتبنا نجمة قبل اسم المؤشر(ptr) سيطبع محتوى الموقع الذي يؤشر عليه

٥.خطوة رقم (٦) سيطبع قيمة المتغير (x) شاهد شاشة التنفيذ



مثال : تغير محتويات المتغير (g) بواسطة المؤشر.؟

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
1.{int g=5;
2.int *ptr=&g;
3. *ptr=32;
4.printf("\nx=%d ",g);}
	#include<iostream.h>
	main()
	1.{int g=5;
	2.int *ptr=&g;
	3. *ptr=32;
	4.cout<<"\nx="<<g;}

توضيح الخطوات:

١.خطوة رقم(٢) جعلنا المؤشر يشير إلى موقع المتغير (g)

٢.خطوة رقم (٣) وضعنا قيمة جديدة في الموقع الذي يؤشر عليه المؤشر (*ptr) وهو موقع المتغير (g)

مهم

٣.٦- الدالة (NEW) : هي دالة تستخدم لحجز مكان في الذاكرة لمؤشر معين لان المؤشرات بطبيعتها سوف تؤشر على مواقع متغيرات ولن تحجز مواقع أما مع هذا الإيعاز نستطيع حجز موقع للمؤشر وتعريفها بالشكل التالي

هيكلية المؤشر
Type *ptr=new type[size]

- **(ptr)** : هو اسم المؤشر وقد يكون أي اسم يعرفه المستخدم ويجب وضع علامة النجمة (*) قبله
- **(Type)**: هو نوع المؤشر ويكون نوعه نفس نوع المتغير الذي يؤشر عليه
- **(size)**: هو الحجم أو عدد الموقع الذي سوف نحجزه للمؤشر في الذاكرة .

مثال : تكوين مؤشر جديد وحجز مكان جديد له وإبدال بين محتوى المؤشر ومحتويات المتغير (g).؟

البرمجة بلغة c++	البرمجة بلغة c

#include<iostream.h>
main()
1.{int g=15,item;
2.int *ptr=new int [1];
3. *ptr=32;
4.item=*ptr;
5.*ptr=g;
6.g=item;
7.cout<<"\ng="<<g<<"\n*ptr="<<*ptr;}
	#include<stdio.h>
	main()
	1.{int g=15,item;
	2.int *ptr=new int [1];
	3. *ptr=32;
	4.item=*ptr;
	5.*ptr=g;
	6.g=item;
	7.printf("\ng=%d\n*ptr=%d",g,*ptr);}

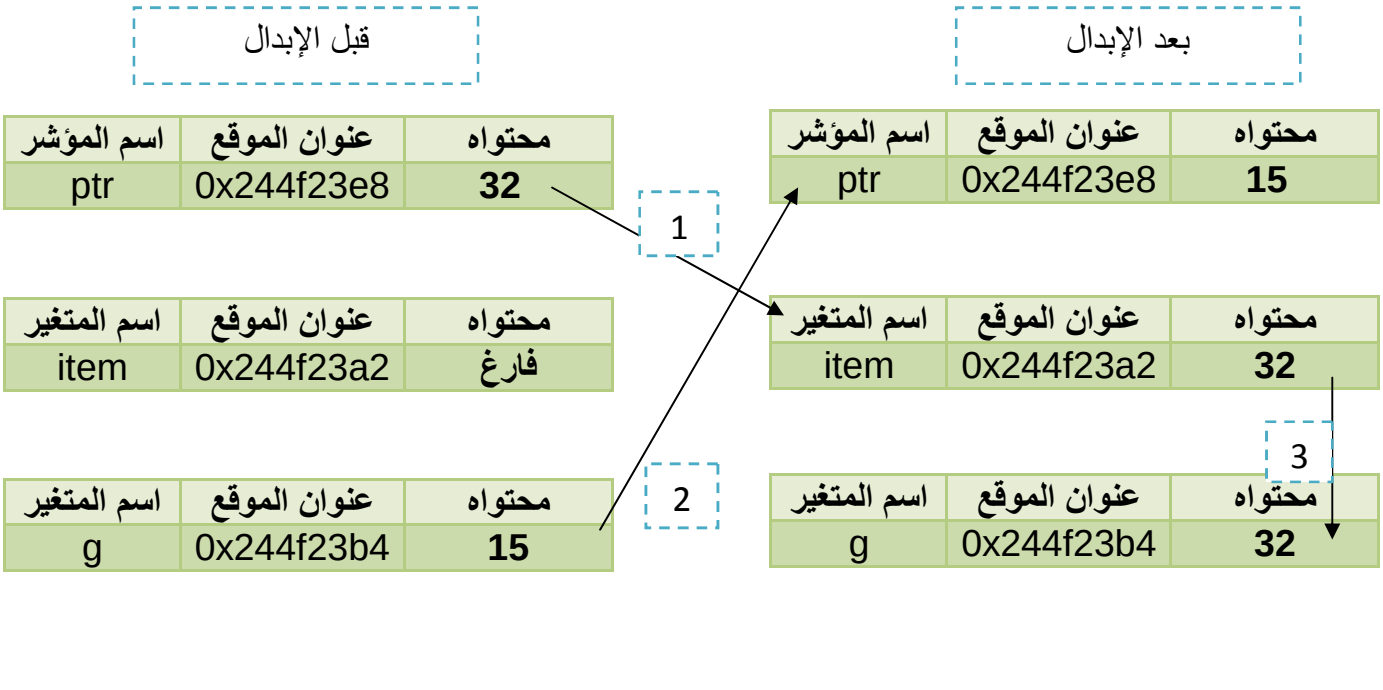
توضيح الخطوات:

١.خطوة رقم(٢) جعلنا المؤشر يشير إلى موقع جديد .وخطوة لرقم(٣) خزنا بالموقع قيمة (٣٢)

٢.خطوة رقم (٤) وضعنا قيمة المؤشر (ptr*) في متغير مؤقت للإبدال

٣.خطوة رقم (٥)وضعنا قيمة المتغير (g) في الموقع الذي يؤشر عليه المؤشر (ptr*)

٤.خطوة رقم (٦) وضعنا قيمة المؤشر التي خزناها في متغير مؤقت إلى متغير(g)



٤.٦- المؤشرات والمصفوفات:

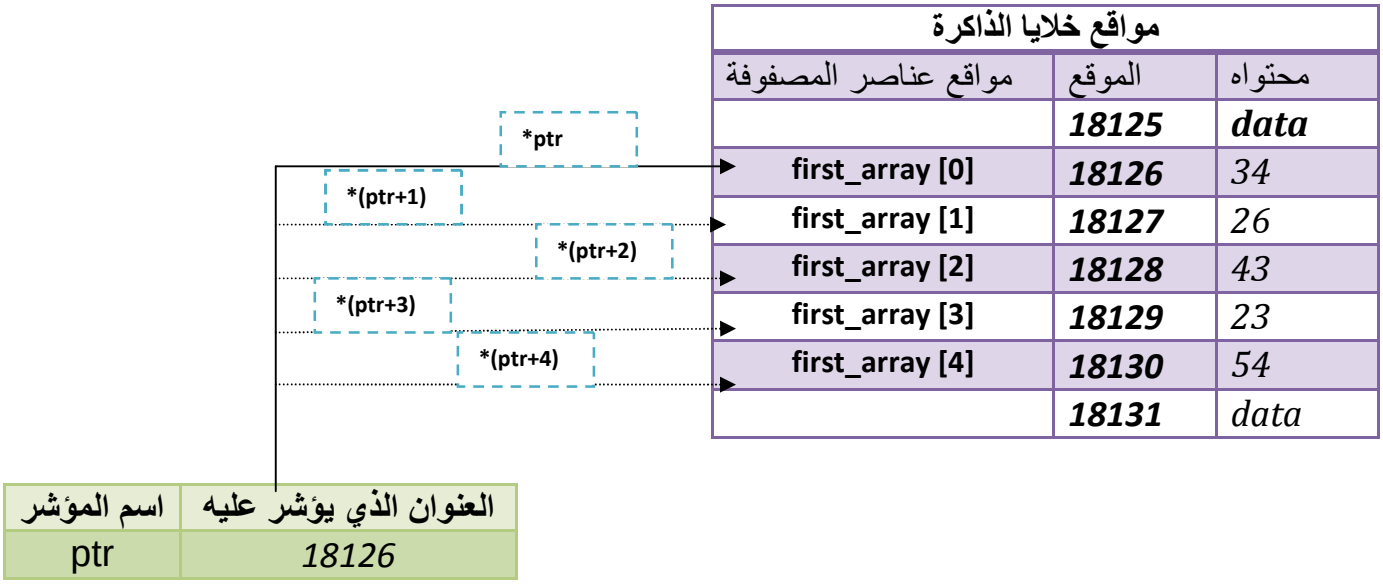
مهم

المصفوفات الأحادية والمؤشرات: عرفنا أن المصفوفة الأحادية هي مجموعة من المواقع المتتالية المحجوزة في الذاكرة ويمكن أن نستخدم المؤشر مع المصفوفة الأحادية وجعله يؤشر على احد القيم وسهولة تمريره على جميع العناصر فقط نزيد قيمة عنوان المؤشر بواحد فينتقل المؤشر ليؤشر على الموقع التالي الذي يليه.

مثال توضيحي :لو عرفنا المصفوفة التالية حجمها خمسة عناصر وجعلنا المؤشر يؤشر على أول عنصر بالمصفوفة

كود
```
Int  first_array[5]={34,26,43,23,54};
int *ptr=& first_array[0];
```

وافترضنا أن أول عنصر في المصفوفة خزن بموقع (***18126***)



- لاحظ أن المؤشر(**ptr**) يؤشر على (**first_array [0]**) أي على الموقع (*18126*)
- إذا أردنا أن يؤشر المؤشر (ptr) على الموقع الثاني بالمصفوفة نزيد قيمة الموقع الذي يؤشر عليه المؤشر (ptr) بمقدار واحد هكذا

كود
```
ptr+1→18126+1→18127
```

ليصبح المؤشر يؤشر على الموقع (18127) وهو عنوان ثاني موقع بالمصفوفة

- إذا أردنا أن يؤشر المؤشر (ptr) على الموقع الرابع بالمصفوفة نزيد قيمة الموقع الذي يؤشر عليه المؤشر (ptr) بمقدار ثلاثة هكذا . ليصبح المؤشر يؤشر على الموقع (18129) وهو عنوان رابع موقع بالمصفوفة

كود
```
ptr+1→18126+3→18129
```

مثال: مصفوفة مكونة من خمسة عناصر أضف مقدار (١٤) لكل عنصر باستخدام المؤشرات..؟

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
1.{ int i, first_array[5]={34,26,43,23,54};
2.int *ptr=& first_array[0];
3. for(i=0;i<5;i++){
4. *(ptr+i)=*(ptr+i)+14;
4.printf("\n first_array[%d]=%d",i,*(ptr+i));}}
	#include<iostream.h>
	main()
	1.{ int i, first_array[5]={34,26,43,23,54};
	2.int *ptr=& first_array[0];
	3. for(i=0;i<5;i++){
	4. *(ptr+i)=*(ptr+i)+14;
	5.cout<<"\n first_array["<<i<<"]="<<*(ptr+i);}}

توضيح الخطوات:

١.خطوة رقم (٢) جعلنا المؤشر **(ptr)** يؤشر على عنوان أول عنصر في المصفوفة

٢.خطوة رقم (٣) هو عداد يعد من (4—0) ويكرر خطوة رقم (٤ و٥) في كل عدة

٣.خطوة رقم (٤) هو إضافة مقدار (١٤) لكل موقع من مواقع المصفوفة **فمثلا عند الإضافة في الموقع الثالث** تكون قيمة (i=2) فتكون خطوة رقم(٤) هكذا

كود
4. *(ptr+i)=*(ptr+i)+14;→*(ptr+2)=*(ptr+2)+14;

وبما أن المؤشر في خطوة رقم (٢) يؤشر على أول عنصر في المصفوفة فعد إضافة قيمة (٢) إلى عنوان الموقع سوف يؤشر على ثالث موقع بالمصفوفة.

مثال: مصفوفة مكونة من ثلاث عناصر أجمعها باستخدام المؤشرات..؟

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
1.{ int sum=0, first_array[3]={ 43,23,54};
2.int *ptr=& first_array[0];
3. sum+=*ptr++;
4. sum+=*ptr++;
5. sum+=*ptr++;
6.printf("\n sum=%d ",sum);}
	#include<iostream.h>
	main()
	1.{ int sum=0, first_array[3]={ 43,23,54};
	2.int *ptr=& first_array[0];
	3. sum+=*ptr++;
	4. sum+=*ptr++;
	5. sum+=*ptr++;
	6.cout<<"\n sum="<<sum;}

توضيح الخطوات:

١.خطوة رقم (٢) جعلنا المؤشر **(ptr)** يؤشر على أول عنصر في المصفوفة

٢.خطوة رقم (٣) أضفنا قيمة أول موقع إلى قيمة المتغير (sum) وزودنا قيمة الوقع بمقدار واحد ليؤشر على العنصر الثاني ***(لو تلاحظ الزيادة بعد الجمع إي يجمع قيمة الموقع الأول ثم ينقل المؤشر للموقع الثاني)***

٣. خطوة رقم (٤) أضفنا قيمة ثاني موقع إلى قيمة المتغير (sum) وزودنا قيمة الوقع بمقدار واحد ليؤشر على العنصر الثالث

٤. خطوة رقم (٥) أضفنا قيمة ثالث موقع إلى قيمة المتغير (sum) وزودنا قيمة الوقع بمقدار واحد

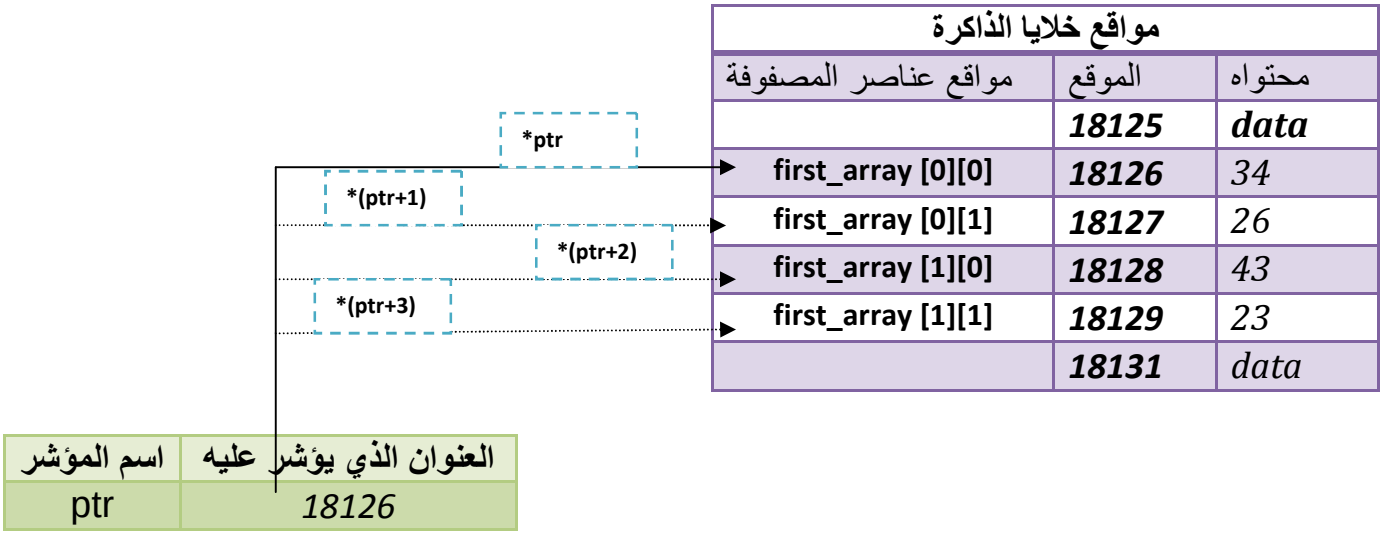
مهم

المصفوفات الثنائية والمؤشرات: عرفنا أن المصفوفة الثنائية هي مجموعة من المواقع المتتالية المحجوزة في الذاكرة ويمكن أن نستخدم المؤشر مع المصفوفة الثنائية وجعله يؤشر على احد القيم وسهولة تمريره على جميع العناصر فقط نزيد قيمة عنوان المؤشر بواحد فينتقل المؤشر ليؤشر على الموقع التالي الذي يليه.

مثال توضيحي:لو عرفنا المصفوفة التالية حجمها (2*2) وجعلنا المؤشر يؤشر على أول عنصر بالمصفوفة

كود
```
int  first_array[2][2]={{34,26},{43,23} };
int *ptr=& first_array[0][0];
```

وافترضنا أن أول عنصر في المصفوفة خزن بموقع (***18126***)



- لاحظ أن المؤشر(**ptr**) يؤشر على (**first_array [0][0]**) أي على الموقع (*18126*)
- إذا أردنا أن يؤشر المؤشر (ptr) على الموقع الثاني بالمصفوفة (أي الصف الأول العمود الثاني) نزيد قيمة الموقع الذي يؤشر عليه المؤشر (ptr) بمقدار واحد هكذا

كود
```
ptr+1→18126+1→18127
```

ليصبح المؤشر يؤشر على الموقع (18127) وهو عنوان ثاني موقع بالمصفوفة

- إذا أردنا أن يؤشر المؤشر (ptr) على الموقع الرابع بالمصفوفة (أي الصف الثاني العمود الثاني) نزيد قيمة الموقع الذي يؤشر عليه المؤشر (ptr) بمقدار ثلاثة هكذا . ليصبح المؤشر يؤشر على الموقع (18129) وهو عنوان رابع موقع بالمصفوفة

كود
```
ptr+1→18126+3→18129
```

مهم

- **تعامل المصفوفة الثنائية نفس معاملة المصفوفة الأحادية بالمؤشرات لان كلاهما عبارة عن خلايا متسلسلة محجوزة بالذاكرة**

مثال: مصفوفة حجمها (2*2) أضرب كل عنصر بمقدار (٤) باستخدام المؤشرات..؟

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
1.{ int i, j,first_array[2][2]={{34,26},{43,23} };
2.int *ptr=& first_array[0][0];
3. for(i=0;i<2*2;i++)
4. *(ptr+i)=*(ptr+i)*4;
5.for(i=0;i<2 ;i++){
6.for(j=0;j<2 ;j++)
7.printf("%d\t", first_array[i][j]);
8.printf("\n");}}
	#include<iostream.h>
	main()
	1.{ int i, j,first_array[2][2]={{34,26},{43,23} };
	2.int *ptr=& first_array[0][0];
	3. for(i=0;i<2*2;i++)
	4. *(ptr+i)=*(ptr+i)*4;
	5.for(i=0;i<2 ;i++){
	6.for(j=0;j<2 ;j++)
	7.cout<< first_array[i][j];
	8.cout<<"\n";}}

توضيح الخطوات:

١.خطوة رقم (٢) جعلنا المؤشر **(ptr)** يؤشر على عنوان أول عنصر في المصفوفة

٢.خطوة رقم (٣) هو عداد يعد من (4—0) ويكرر خطوة رقم (٤ و٥) في كل عدة

٣.خطوة رقم (٤) هو ضرب مقدار (٤) لكل موقع من مواقع المصفوفة **فمثلا عند الإضافة في الموقع الثالث** تكون قيمة (i=2) فتكون خطوة رقم(٤) هكذا

كود
4. *(ptr+i)=*(ptr+i)*4;→*(ptr+2)=*(ptr+2)*4;

وبما أن المؤشر في خطوة رقم (٢) يؤشر على أول عنصر في المصفوفة فعد إضافة قيمة (٢) إلى عنوان الموقع سوف يؤشر على ثالث موقع بالمصفوفة. خطوة رقم (٥ و٦و٧و٨) هو طباعة لعناصر المصفوفة بعد الضرب

مثال: مصفوفة حجمها (2*2) أجمعها باستخدام المؤشرات..؟

البرمجة بلغة c	البرمجة بلغة c++

#include<stdio.h>
main()
1.{ int sum=0, first_array[2][2]={{34,26},{43,23} };
2.int *ptr=& first_array[0][0];
3. sum+=*ptr++;
4. sum+=*ptr++;
5. sum+=*ptr++;
6. sum+=*ptr++;
7.printf("\n sum=%d ",sum);}
	#include<iostream.h>
	main()
	1.{ int sum=0, first_array[2][2]={{34,26},{43,23} };
	2.int *ptr=& first_array[0][0];
	3. sum+=*ptr++;
	4. sum+=*ptr++;
	5. sum+=*ptr++;
	6. sum+=*ptr++;
	7.cout<<"\n sum="<<sum;}

توضيح الخطوات:

١.خطوة رقم (٣) أضفنا قيمة أول موقع (0,0) إلى قيمة المتغير (sum) وزودنا قيمة الوقع بمقدار واحد ليؤشر على العنصر الثاني (0,1) ***(لو تلاحظ الزيادة بعد الجمع إي يجمع قيمة الموقع الأول ثم ينقل المؤشر للموقع الثاني)***

٢. خطوة رقم (٤) أضفنا قيمة ثاني موقع (0,1) إلى قيمة المتغير (sum) وزودنا قيمة الوقع بمقدار واحد ليؤشر على العنصر الثالث (1,0)

٣. خطوة رقم (٥) أضفنا قيمة ثالث(1,0) موقع إلى قيمة المتغير (sum) وزودنا قيمة الوقع بمقدار واحد ليؤشر على العنصر الرابع (1,1)

٤. خطوة رقم (٦) أضفنا قيمة رابع موقع (1,1) إلى قيمة المتغير (sum) وزودنا قيمة الوقع بمقدار واحد .

مهم

٥.٦- المؤشرات والدوال (function)

١. **الدوال والمتغيرات:** عرفنا طريقة إرسال متغير بالقيمة كيف وبالمرجع كيف تكون. المؤشرات تمكنك من إرسال المتغير بالمرجع أي إرسال موقع المتغير ***وهذه تفيد إذا كنا نريد أن نعيد أكثر من متغير إلى البرنامج الرئيسي*** ونعلم إن الإرسال بالمرجع يكون أي تغير على المتغير في الدوال يؤثر على قيمته في البرنامج الرئيسي

لو كان لدينا متغير اسمه (a) ونريد إرساله إلى دالة اسمها (name) .

إرسال مصفوفة مؤشر إلى الدوال (function)
name(&a);

- وضع علامة (&) قبله معناه أننا أرسلنا عنوان أول موقع .

وطريقة استقبال المتغير في الدوال نعرف مؤشر من نفس نوع المتغير المرسل لكي يؤشر على موقعه .

استقبال مصفوفة مؤشر إلى الدوال (function)
Type name(type *ptr);

- **(type)**: هو نوع المتغير المرسل
- ألان أصبح هذا المؤشر **(ptr)** يؤشر على عنوان المتغير.

مثال: تكوين دالة تستقبل وحرف صغير وتحوله إلى حرف كبير..؟

تحليل: لتحويل حرف من صغير إلى كبير نعلم أن الفرق بين أسكي كود كل حرف صغير ونضيره الكبير هو (٣٢) لذالك لتحويل إلى حرف كبير نحول الحرف إلى أسكي كود ونطرح منه (٣٢) ونرجع نحوله إلى حرف فيتحول إلى حرف كبير

الحل باستخدام المؤشرات.....!

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h>	#include<iostream.h>
1.inline toBigLeter (char *ptr)	1.inline toBigLeter (char *ptr)
2.{*ptr=int(*ptr)-32;}	2.{*ptr=int(*ptr)-32;}
3.main()	3.main()
4. { char inputchar;	4. { char inputchar;
5.scanf("%c",&inputchar);	5.cin>> inputchar;
6.toBigLeter (& inputchar);	6.toBigLeter (& inputchar);
7.prin ("Big to it is=%c", inputchar);}	7.cout<<"Big to it is="<< inputchar;}

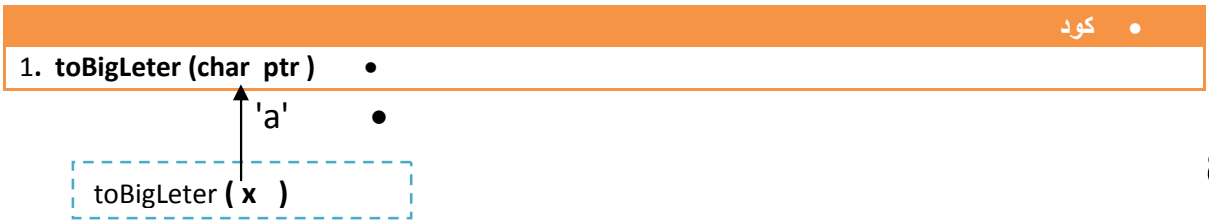
الحل بدون استخدام المؤشرات.....!

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h> 1. inline toBigLeter (char charrec) 2.{ charrec =int(charrec)-32;} 3.main() 4. { char inputchar; 5.scanf("%c",&inputchar); 6.toBigLeter (inputchar); 7.prin ("Big to it is=%c", inputchar);}	#include<iostream.h> 1.inline toBigLeter (char charrec) 2.{ charrec =int(charrec)-32;} 3.main() 4. { char inputchar; 5.cin>> inputchar; 6.toBigLeter (inputchar); 7.cout<<"Big to it is="<< inputchar;}

توضيح الخطوات بالنسبة للحلين:

- خطوة رقم (١) نسبة إلى **الحل باستخدام المؤشرات**: إعلان عند دالة سطريه تستقبل موقع الحرف وتكبر الحرف في خطوة رقم (٢)
- خطوة رقم (١) نسبة إلى **الحل بدون استخدام المؤشرات**: إعلان عند دالة سطريه تستقبل الحرف وتكبر الحرف في خطوة رقم (٢)
- خطوة رقم (٦) نسبة إلى **الحل باستخدام المؤشرات**: إرسال عنوان المتغير (inputchar) إلى الدالة هكذا

- خطوة رقم (٦) نسبة إلى **الحل بدون استخدام المؤشرات**: إرسال قيمة المتغير (inputchar) إلى الدالة يكون هكذا (إذا قام المستخدم بإدخال الحرف a مثلا فيكون الإرسال هكذا)



- خطوة رقم (٧) نسبة إلى **الحل باستخدام المؤشرات**: ***سوف يطبع الحرف بعد التكبير لأننا في المؤشرات نتعامل مع موقع المتغير فأي تغير في الدالة على الموقع يغير في قيمه المتغير في البرنامج الرئيسي***

- خطوة رقم (٦) نسبة إلى **الحل بدون استخدام المؤشرات**: ***سوف يطبع الحرف بدون أي تكبير نفس الحرف المدخل سوف يطبعه لأننا أرسلنا نسخة من المتغير إلى الدالة ولم نرسل المتغير نفسه فأي تغير على هذه النسخة لا يؤثر بقيمه المتغير في البرنامج الرئيسي***

٢. **الدوال والمصفوفات الأحادية:** عرفا سابقا طريقة التعامل مع المصفوفات في الدوال (function) وعرفنا طريقة الإرسال إلى الدالة وهي إرسال نسخة من المصفوفة وليس المصفوفة الأصلية أي كان إرسال بالقيمة. **أما مع المؤشرات يكون إرسال بالمرجع** أي أن أي تغير على المصفوفة داخل أي دالة (function) سوف يؤثر على المصفوفة الأصلية في البرنامج الرئيسي التي أرسلت إلى الدالة للمعالجة لأن المؤشرات تتعامل مع مواقع الذاكرة أي مواقع المتغيرات وليس نسخة منه

ترسل المصفوفة الأحادية إلى الدالة بتحديد عنوان الموقع المرسل كأن يكون أننا نرسل عنوان أول موقع وفي الدالة عندما يعرف أول موقع يستطيع التنقل إلى باقي المواقع بزيادة قيمة المؤشر بواحد كل مرة . هكذا ترسل

مثال:لو كان لدينا مصفوفة اسمها (a) ونريد إرسالها إلى دالة اسمها (name) .

إرسال مصفوفة مؤشر إلى الدوال (function)

```
name( &a [0]);
```

- في هذه الطريقة أرسلنا عنوان أول موقع بوضع علامة (&) قبله .

وطريقة استقبال المصفوفة الأحادية في الدوال نعرف مؤشر من نفس نوع المصفوفة المرسلة .

استقبال مصفوفة مؤشر إلى الدوال (function)

```
Type  name(type *ptr);
```

- **(type):** هو نوع المصفوفة المرسلة
- ألان أصبح هذا المؤشر **(ptr)** يؤشر على عنوان أول موقع بالمصفوفة

** في إرسال المصفوفة إلى الدوال قد نرسل عنوان أول موقع أو نرسل عنوان أخر موقع أو أي موقع حسب ما نحتاجه في برنامجنا

مثال : تكوين دالة تعكس تسلسل أحرف ثلاث مصفوفات بالعكس وتطبعهم بعد العكس .؟

تحليل: بما انه يريد عكس الأحرف في المصفوفة باستخدام الدوال فيجب الإرسال بالمرجع حتى عندما يقلب أحرف السلسلة وعندما ينتهي من (function) ويعود للبرنامج الرئيسي تعكس المصفوفات أيضا في البرنامج الرئيسي .

البرمجة بلغة c

```
#include<stdio.h>
#include<string.h>
1.int i;
2.Reverse(char *string,int len)
3.{char item;
4.for(i=len; i>len/2;i--){
5.item=* (string);
6.*(string )=*(string -i+(len-i));
7.  *(string -i+(len-i))= item;
 8. string --;} }
9.main()
10. { char string1[55], string2[55], string3[55];
11.int len;
12.prin   ("enter string (1):  ");
 13.gets ( string1 );
14.len=strlen(string1)-1;
15.Reverse( &string1[len], len);
16.prin   ("%s",string1 );
17.prin   ("\n enter string (2): ");
 18.gets ( string2);
19.len=strlen(string2)-1;
20.Reverse( &string2[len], len);
21.prin   ("%s",string2 );
22.prin   ("\n enter string (3): ");
 23.gets ( string3 );
24.len=strlen(string3)-1;
25.Reverse( &string3[len], len);
26.prin   ("%s",string3);}
```

البرمجة بلغة c++

```
#include<iostream.h>
#include<string.h>
1.int i;
2.Reverse(char *string,int len)
3.{char item;
4.for(i=len; i>len/2;i--){
5.item=* (string);
6.*(string )=*(string -i+(len-i));
7.  *(string -i+(len-i))= item;
 8. string --;} }
9.main()
10. { char string1[55], string2[55], string3[55];
11.int len;
12.cout<<"enter string (1):  ";
13.cin.getline( string1,55);
14.len=strlen(string1)-1;
15.Reverse( &string1[len], len);
16.cout<< string1 ;
17.cout<<"\nenter string (2):  ";
18.cin.getline( string2,55);
19.len=strlen(string2)-1;
20.Reverse( &string2[len], len);
21.cout<< string2 ;
22.cout<<"\n enter string (3):  ";
23.cin.getline( string3,55);
24.len=strlen(string3)-1;
25.Reverse( &string3[len], len);
26.cout<< string3 ;}
```

توضيح الخطوات:

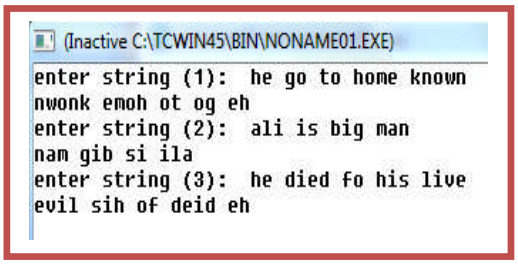
١.خطوة رقم (٢)هي دالة تستقبل أخر موقع بالسلسلة وطول هذه السلسلة

٢.خطوة رقم (٤) هو عداد يبدأ بالعد من أخر موقع إلى المنتصف حتى يبدل المواقع الأول بالأخير والثاني بالقبل الأخير في خطوات رقم (٥ و٦و٧و٨) ويستمر بالإبدال

٣.خطوة رقم (١٣) هي إدخال السلسلة الأولى وخطوة (١٤) حساب طولها

٤.خطوة رقم (١٥) هي إرسال أخر موقع بالسلسلة وطول السلسلة

بقية الخطوات واضحة ومكررة.هذه صورة من شاشة التنفيذ



مهم

٣. **الدوال والمصفوفات الثنائية:** عرفا سابقا طريقة التعامل مع المصفوفات في الدوال (function) وعرفنا طريقة الإرسال إلى الدالة وهي إرسال نسخة من المصفوفة الثنائية وليس المصفوفة الأصلية أي كان إرسال بالقيمة. **أما مع المؤشرات يكون إرسال بالمرجع** أي أن أي تغير على المصفوفة داخل أي دالة (function) سوف يؤثر على المصفوفة الأصلية في البرنامج الرئيسي التي أرسلت إلى الدالة للمعالجة لأن المؤشرات تتعامل مع مواقع الذاكرة أي مواقع المتغيرات وليس نسخة منه

ترسل المصفوفة الثنائية إلى الدالة بتحديد عنوان الموقع المرسل كأن يكون أننا نرسل عنوان أول موقع وفي الدالة عندما يعرف أول موقع يستطيع التنقل إلى باقي المواقع بزيادة قيمة المؤشر بواحد كل مرة . هكذا ترسل

مثال:لو كان لدينا مصفوفة اسمها (a) حجمها (4*4) ونريد إرسالها إلى دالة اسمها (name) .

إرسال مؤشر مصفوفة الثنائية إلى الدوال (function)

```
name( &a [0][0]);
```

- في هذه الطريقة أرسلنا عنوان أول موقع بوضع علامة (&) قبله .

وطريقة استقبال المصفوفة الثنائية في الدوال نعرف مؤشر من نفس نوع المصفوفة المرسلة .

استقبال مؤشر مصفوفة الثنائية إلى الدوال (function)

```
Type  name(type *ptr);
```

- **(type)**: هو نوع المصفوفة المرسلة
- ألان أصبح هذا المؤشر **(ptr)** يؤشر على عنوان أول موقع بالمصفوفة

** في إرسال المصفوفة إلى الدوال قد نرسل عنوان أول موقع أو نرسل عنوان أخر موقع أو أي موقع حسب ما نحتاجه في برنامجنا

مثال : تكوين دالة تضع واحد مكان العدد الأولي في مصفوفة (4*4) وصفر مكان العدد الغير أولي .؟

تحليل:بما إننا نريد تحويل المصفوفة إلى أصفار و واحدات نرسلها كمؤشر إلى داله وهناك العدد الأولي نضع مكانه واحد والغير أولي نضع صفر

البرمجة بلغة c	البرمجة بلغة c++
#include<stdio.h>	#include<iostream.h>
1.int i,j;	1.int i,j;
2.int row=4;	2.int row=4;
3.int col=4;	3.int col=4;
4.prime2d (int *string)	4.prime2d (int *string)
5.{int prime=1;	5.{int prime=1;
6.for(i=0; i<row*col ;i++){	6.for(i=0; i<row*col ;i++){
7.prime=1;	7.prime=1;
8.for(j=2;j<*string;j++)	8.for(j=2;j<*string;j++)
9.if(*string % j==0)	9.if(*string % j==0)
11.prime=0;	11.prime=0;
12.*string= prime;	12.*string= prime;
13. string ++;} }	13. string ++;} }
14.main()	14.main()
15.{ int string1[4][4];	15.{ int string1[4][4];
16. for(i=0; i<row ;i++)	16. for(i=0; i<row ;i++)
17.for(j=0; j<col ;j++)	17.for(j=0; j<col ;j++)
18.scanf("%d",& string1[i][j]);	18.cin>> string1[i][j];
19.prime2d(&string1[0] [0]);	19.prime2d(&string1[0] [0]);
20.for(i=0; i<row ;i++){	20.for(i=0; i<row ;i++){
21.for(j=0; j<col ;j++)	21.for(j=0; j<col ;j++)
22.printf("%d\t", string1[i][j]);	22.cout<< string1[i][j]<<"\t";
23.printf("\n");}}	23.cout<<"\n";}}

توضيح الخطوات:

١.خطوة رقم (٤) دالة تستقبل عنوان أول عنصر بالمصفوفة

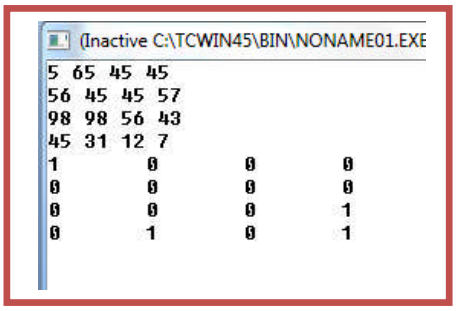
٢.خطوة رقم (٥) هو متغير إذا كان الرقم الذي عليه المؤشر عدد أولي يبقى واحد وذا كان الرقم غير أولي يتحول إلى صفر لكي يخزن بدل قيمة العنصر

٣.خطوة رقم (٦) هوا عداد يمر على جميع عناصر المصفوفة لكي يتحقق من العناصر عنصر عنصر في خطوات رقم (٨و٧ و٩و١٠و١١و١٢) هل العنصر عدد أولي أم لا

٤.خطوة رقم(١٣) لكي ينقل المؤشر على العنصر التالي بعد أن يتحقق من العنصر السابق

٥.خطوة رقم(١٩) هي إرسال عنوان أول عنصر بالمصفوفة بالمصفوفة إلى الدالة

شاهد شاشة التنفيذ



مهم

مصفوفة أحادية غير محدودة الحجم: تعلمنا سابقا أن المصفوفات حجمها ثابت ويجب تعريفه ولا يمكن كتابة مصفوفة دون تحديد حجمها **أما مع المؤشرات نستطيع** مع دالة (new) تكوين مصفوفة غير محدودة الحجم يحدد حجمها المستخدم وقت التنفيذ **.** حيث أن الحجم الذي نحجزه بقدر حجم المصفوفة المطلوب

لو أردنا أن نحجز مصفوفة حجمها (٩) عناصر من نوع integer وقت التنفيذ نحجز بشكل التالي

حجز مصفوفة وقت التنفيذ

```
int *Array=new int [9];
```

أي كأنما نقول المؤشر (*Array) يؤشر على مكان في الذاكرة حجمه تسعة

مثال : برنامج يطلب من المستخدم تحديد حجم المصفوفة وقت التنفيذ وبعدها يطلب منه أن يدخل عناصر ثم يجمع العناصر ويجد المعدل ؟

البرمجة بلغة c

```
#include <stdio.h>
void main( )
1.{int sizearray,j,sum,avg;
2.sum=0;
3.prin   ("who size the arrray\n");
4.scanf("%d",& sizearray );
5.int *Array=new int [ sizearray ];
6.prin   ("enter the array\n");
7.for ( j=0;j<sizearray; j++)
8. scanf("%d",&Array[j]);
9.for (j=0;j<sizearray; j++)
10. sum=sum+Array[j];
11. avg=sum/sizearray;
12.prin   ("sum=%d\navg=%d",sum,avg); }
```

البرمجة بلغة c++

```
#include <iostream.h>
void main( )
1.{int sizearray,j,sum,avg;
2.sum=0;
3.cout<<"who size the arrray\n" ;
4.cin>> sizearray  ;
5.int *Array=new int [ sizearray ];
6.cout<<"enter the array\n" ;
7.for ( j=0;j<sizearray; j++)
8. cin>> Array[j] ;
9.for (j=0;j<sizearray; j++)
10. sum=sum+Array[j];
11. avg=sum/sizearray;
12. cout<<"sum="<<sum<<"\navg= "<<avg; }
```

توضيح الخطوات :

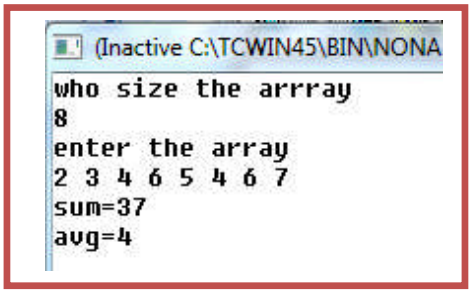
١.خطوة رقم (٤) يطلب من المستخدم إدخال حجم المصفوفة لكي يضع الحجم الذي يدخله المستخدم في خطوة رقم (٥) إي مثلا إذا ادخل المستخدم الرقم خمسة ستكون الخطوة رقم (٤) بشكل التالي

حجز مصفوفة وقت التنفيذ

```
int *Array=new int [5];
```

إي حجز خمسة مواقع في الذاكرة

لاحظ الشكل التالي لهذا الإدخال في شاشة التنفيذ .



مهم

مصفوفة ثنائية غير محدودة الأبعاد: نستطيع مع دالة (new) تكوين مصفوفة ثنائية غير محدودة الأبعاد يحدد أبعادها المستخدم وقت التنفيذ .

لو أردنا أن نحجز مصفوفة حجمها (3*5) من نوع integer وقت التنفيذ نحجز بشكل التالي

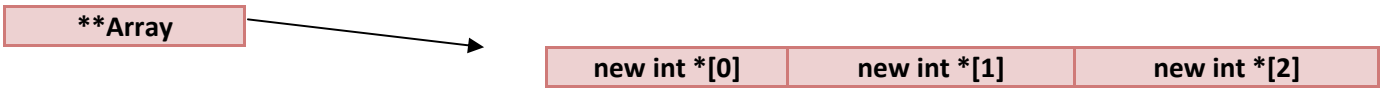
حجز مصفوفة وقت التنفيذ

```
1.int k;
2.int **Array=new int *[row];
 3.for (k=0 ; k<  row  ; k++)
 4.Array[k]=new int[columns];
```

- **(row)**: هو عدد الصفوف
- **(columns)**: هو عدد الأعمدة

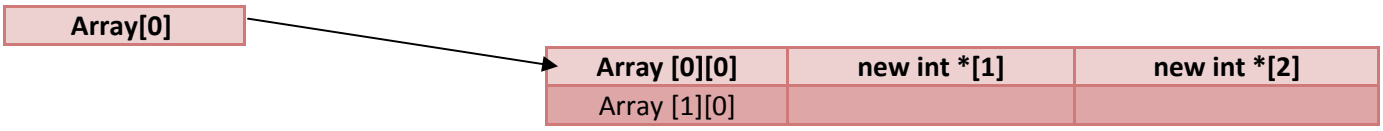
تكون طريق الحجز بشكل التالي مثلا لمصفوفة (2*3):

١.في خطوة رقم (٢) يؤشر المؤشر (Array) على مصفوفة مؤشرات حجمها بقدر عدد الصفوف أي (٣)

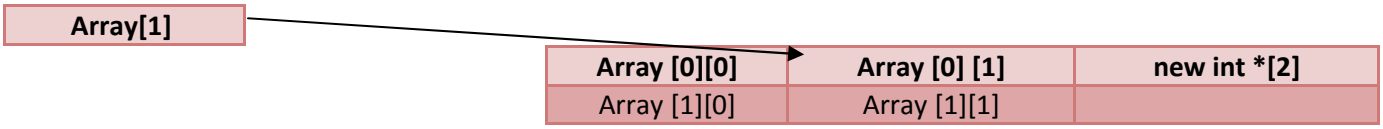


٢.خطوة رقم (٣) يبدأ بالتحرك على كل صف يؤشر عليه المؤشر (Array) ويكون له أعمدة في خطوة رقم (٤)

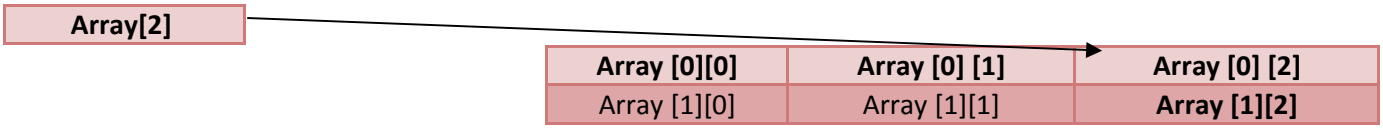
A. عندما تكون قيمة (k=0) سيؤشر المؤشر (Array) على أول موقع بصف ويكون العمود له طوله (٢)



B. عندما تكون قيمة (k=1) سيؤشر المؤشر (Array) على ثاني موقع بصف ويكون العمود له طوله (٢)



C. عندما تكون قيمة (k=2) سيؤشر المؤشر (Array) على ثالث موقع بصف ويكون العمود له طوله (٢)



مثال : مصفوفة ثنائية غير محدودة الحجم يحدد حجمها المستخدم وقت التنفيذ ويجمع عناصر القطر الرئيسي ؟

البرمجة بلغة c	البرمجة بلغة c++
#include <stdio.h>	#include <iostream.h>
void main ( )	void main ( )
1.{ int i,j,k,sum,rowN,colN;	1.{ int i,j,k,sum,rowN,colN;
2. sum=0;	2. sum=0;
3.scanf("%d%d",&rowN,&colN);	3.cin>>rowN>>colN ;
4.int **Array=new int *[rowN];	4.int **Array=new int *[rowN];
5.for (k=0 ; k< rowN ; k++)	5.for (k=0 ; k< rowN ; k++)
6.Array[k]=new int[colN];	6.Array[k]=new int[colN];
7.for (i=0 ; i< rowN ; i++)	7.for (i=0 ; i< rowN ; i++)
8.for (j=0; j< colN ; j++)	8.for (j=0; j< colN ; j++)
9. scanf("%d",&Array[i][j]);	9. cin>>Array[i][j] ;
10.for (i=0 ; i< rowN ; i++)	10.for (i=0 ; i< rowN ; i++)
11. for (j=0; j< colN ; j++)	11. for (j=0; j< colN ; j++)
12.if (i==j)	12.if (i==j)
13.sum=sum+ Array[i][j];	13.sum=sum+ Array[i][j];
14. printf(""sum="%d",sum); }	14. cout<<"sum="<<sum ; }

توضيح الخطوات:

١. خطوة رقم (٣) هي إدخال عدد صفوف المصفوفة وعدد أعمدتها
٢. خطوة رقم (٤ و٥و٦) تكوين المصفوفة
٣. خطوة رقم (٧ و٨و٩) هي إدخال المصفوفة بالإبعاد التي حددناها
٤. خطوة رقم (١٠ و١١و١٢و١٣) هي أيجاد عناصر القطر الرئيسي وجمعها
٥. خطو رقم (١٤) طباعة ناتج الجمع

لو أدخلنا مصفوفة حجمها (5*5) من شاشة التنفيذ بشكل التالي

```
(Inactive C:\TCWIN45\BIN\NO
5 5
5 6 4 3 4
5 6 7 8 9
6 7 8 9 7
6 7 5 4 5
6 7 8 6 5
sum=28
```